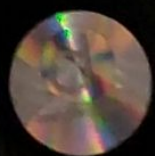




Desta Sandya Prasvita, S.Kom., M.Kom.
Mayanda Mega Santoni, S.Kom., M.Kom.
Nurul Chamidah, S.Kom., M.Kom.

PENGOLAHAN
CITRA DIGITAL
MENGUNAKAN
OPENCV-PYTHON



**PENGOLAHAN CITRA DIGITAL
MENGUNAKAN OPENCV-PYTHON**

PENGOLAHAN CITRA DIGITAL MENGUNAKAN OPENCV-PYTHON

Desta Sandya Prasvita, S.Kom., M.Kom.

Mayanda Mega Santoni, S.Kom., M.Kom.

Nurul Chamidah, S.Kom., M.Kom.



Cerdas, Bahagia, Mulia, Lintas Generasi.

DAFTAR ISI

KATA PENGANTAR PENERBIT	v
DAFTAR ISI	vi
DAFTAR GAMBAR.....	xi
 BAB I PENGENALAN OPENCV-PYTHON.....	1
Materi	1
Tujuan Pembelajaran.....	1
A. Penyajian	1
1. Instalasi dan Konfigurasi OpenCV-Python	2
2. Memulai dengan Citra	5
B. Latihan	8
C. Lembar Kerja Praktikum	9
Referensi	10
 BAB II OPERASI DASAR PADA CITRA DIGITAL	11
Materi	11
Tujuan Pembelajaran.....	11
A. Penyajian	11
1. Operasi Dasar pada Citra.....	11
2. Operasi Aritmatika pada Citra.....	14
B. Latihan	15
C. Lembar Kerja Praktikum	22
Referensi	23
 BAB III RUANG WARNA PENGOLAHAN CITRA	
DIGITAL.....	24
Materi	24
Tujuan Pembelajaran.....	24
A. Penyajian	24
1. Konsep Warna	24

	2. Model Warna.....	25
	3. Transformasi Ruang Warna pada Citra Digital.....	28
B.	Latihan.....	31
C.	Lembar Kerja Praktikum	34
	Referensi	35
BAB IV	PENGOLAHAN TITIK	36
	Materi.....	36
	Tujuan Pembelajaran	36
A.	Penyajian	36
	1. Image Enhancement.....	36
	2. Point Processing	37
B.	Latihan.....	43
C.	Lembar Kerja Praktikum	45
	Referensi	46
BAB V	HISTOGRAM	47
	Materi:.....	47
	Tujuan Pembelajaran:	47
A.	Penyajian	47
	1. Histogram.....	47
	2. Perhitungan Histogram dan Plot Histogram.....	49
	3. Contrast Stretching/Histogram Stretching	51
	4. Histogram Equalization.....	52
	5. Histogram Warna	52
B.	Latihan.....	54
C.	Lembar Kerja Praktikum	57
	Referensi	57
BAB VI	<i>IMAGE ENHANCEMENT DENGAN MASK</i>	
	PROCESSING.....	58
	Materi.....	58
	Tujuan Pembelajaran	58
A.	Penyajian	58
	1. Mask Processing	58
	2. Filter Spasial	60

	3. Jenis Filter Spasial.....	61
	B. Latihan.....	64
	C. Lembar Kerja Praktikum	68
	Referensi	69
BAB VII	RESTORASI CITRA.....	70
	Materi:.....	70
	Tujuan Pembelajaran:.....	70
	A. Penyajian	70
	1. Restorasi Citra	70
	2. Noise.....	71
	B. Latihan.....	75
	C. Lembar Kerja Praktikum	87
	Referensi	88
BAB VIII	SEGMENTASI CITRA (<i>IMAGE SEGMENTATION</i>).....	89
	Materi:.....	89
	Tujuan Pembelajaran:.....	89
	A. Penyajian	89
	1. Segmentasi Citra.....	89
	2. Thresholding.....	89
	3. Deteksi Tepi (<i>Edge Detection</i>)	90
	B. Latihan.....	96
	C. Lembar Kerja Praktikum	98
	Referensi	98
BAB IX	MORFOLOGI.....	99
	Materi:.....	99
	Tujuan Pembelajaran:.....	99
	A. Penyajian	99
	1. Definisi dan Fungsi Morfologi	99
	2. Dilasi	100
	3. Erosi	101
	4. Opening dan Closing	102

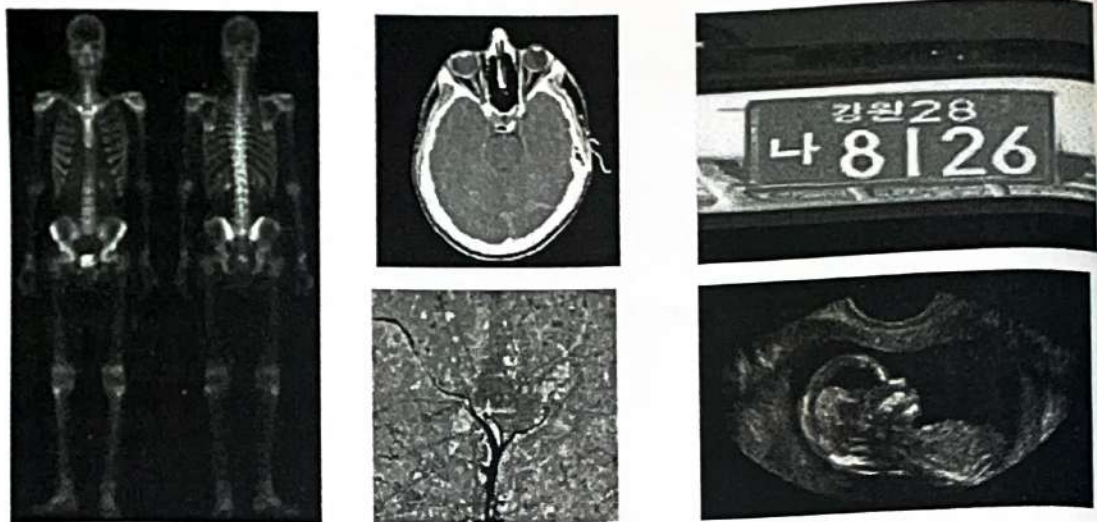
Gambar 27.	Histogram Ideal	51
Gambar 28.	Kontras pada Citra	51
Gambar 29.	Histogram Warna pada <i>Channel</i> RGB.....	51
Gambar 30.	SutanSjahrirLowContrast.jpg.	51
Gambar 31.	Contoh Jendela Ketetanggan	51
Gambar 32.	Mask Berukuran 3x3.....	51
Gambar 33.	Contoh Piksel Citra.....	51
Gambar 34.	Perbandingan Gambar Frekuensi Rendah dan Frekuensi Tinggi.....	61
Gambar 35.	Kernel dari LPF	61
Gambar 36.	Monas.jpg.	63
Gambar 37.	Ki Hadjar Dewantara, Bapak Pendidikan Nasional.....	69
Gambar 38.	Citra Salt and Pepper Noise.....	72
Gambar 39.	Citra <i>Gaussian Noise</i>	73
Gambar 40.	Citra <i>Speckle Noise</i>	74
Gambar 41.	Citra <i>Periodic Noise</i>	74
Gambar 42.	Citra Asli.....	78
Gambar 43.	Citra Hasil Penambahan <i>Noise</i>	79
Gambar 44.	Citra Hasil Restorasi Pada <i>Salt and Pepper Noise</i>	80
Gambar 45.	Citra Hasil Restorasi pada <i>Gaussian Noise</i>	81
Gambar 46.	Citra Hasil Restorasi pada <i>Speckle Noise</i>	82
Gambar 47.	Citra Input dengan <i>Periodic Noise</i>	86
Gambar 48.	Citra Hasil <i>Low Pass Filter Noise</i> Menggunakan DFT.....	86
Gambar 49.	Citra Hasil <i>High Pass Filter Noise</i> Menggunakan DFT.....	87
Gambar 50.	Citra Hasil Restorasi <i>Periodic Noise</i> Menggunakan DFT.....	87
Gambar 51.	Sutomo.jpg.....	88
Gambar 52.	Roberts Operator.....	90
Gambar 53.	Prewitt Operator	91
Gambar 54.	Sobel Operator	92
Gambar 55.	<i>Non-Maximum Suppression</i>	94
Gambar 56.	<i>Hysteresis Thresholding</i>	95

DAFTAR GAMBAR

Gambar 1.	Contoh Penerapan Citra Digital dalam Berbagai Bidang.	2
Gambar 2.	Tampilan Anaconda Navigator.....	3
Gambar 3.	RGB Image.....	5
Gambar 4.	<i>Grayscale Image</i>	6
Gambar 5.	Citra Output Latihan 1.....	8
Gambar 6.	Citra "PangeranDiponogoro.jpg".	10
Gambar 7.	Output Citra pada Latihan No. 2.	18
Gambar 8.	Output Citra pada Latihan No. 3.	19
Gambar 9.	LogoUPNVJ.png.	21
Gambar 10.	upn.jpg.....	21
Gambar 11.	Hasil Penggabungan.....	21
Gambar 12.	Soekarno.jpg.....	22
Gambar 13.	Hatta.jpg.	22
Gambar 14.	Keluaran yang Diharapkan dari Penggabungan Kedua Citra.....	23
Gambar 15.	Spektrum Warna.....	25
Gambar 16.	CRT <i>Display</i>	26
Gambar 17.	Representasi Ruang Warna pada HSV.....	27
Gambar 18.	Citra RGB dan Citra YCbCr dari Setiap Channel-nya.	28
Gambar 19.	Output Citra Latihan No. 1.....	33
Gambar 20.	Output Citra Latihan No. 2.....	34
Gambar 21.	<i>Image Enhancement</i>	36
Gambar 22.	Gambaran <i>Point Processing</i>	37
Gambar 23.	Citra Output Latihan No.1.....	43
Gambar 24.	Foto Jendral Sudirman "jenderalsoedirman.jpg".	44
Gambar 25.	Histogram <i>Citra Grayscale 8-Bit</i>	48
Gambar 26.	<i>Exposure</i> pada Citra	48

	B. Latihan.....	104
	C. Lembar Kerja Praktikum	108
	Referensi	108
BAB X	EKSTRAKSI CIRI TEKSTUR	109
	Materi:.....	109
	Tujuan Pembelajaran:	109
	A. Penyajian	109
	1. Konsep Tekstur pada Citra.....	109
	2. Local Binary Pattern	110
	B. Latihan.....	115
	C. Lembar Kerja Praktikum	117
	Referensi	118
BAB XI	KLASIFIKASI CITRA	119
	Materi:.....	119
	Tujuan Pembelajaran:	119
	A. Penyajian	119
	1. <i>Image Classification</i> dan <i>Image Recognition</i>	119
	2. Memahami k-Nearest Neighbor (kNN).....	121
	3. Contoh Perhitungan kNN	122
	B. Latihan.....	123
	C. Lembar Kerja Praktikum	129
	Referensi	129
BAB XII	EVALUASI HASIL KLASIFIKASI CITRA	130
	Materi:.....	130
	Tujuan Pembelajaran:	130
	A. Penyajian	130
	1. Akurasi Klasifikasi	132
	2. Evaluasi Kinerja Model.....	134
	B. Latihan.....	135
	C. Lembar Kerja Praktikum	137
	Referensi	137

di berbagai bidang, seperti bidang pertanian dan perkebunan, kedokteran, industri, militer, geologi, transportasi, dan lain-lain. Gambar 1 ini adalah contoh penerapan citra digital dalam berbagai bidang.



Gambar 1. Contoh Penerapan Citra Digital dalam Berbagai Bidang.

1. Instalasi dan Konfigurasi OpenCV-Python

Salah satu *library* yang mendukung dalam pembuatan aplikasi pengolahan citra digital adalah OpenCV. OpenCV adalah *library* (perpustakaan) *open source* yang digunakan untuk mengolah gambar dan video. OpenCV rilis pertama pada tahun 2000 oleh Gary Bradsky di Intel. Pada 2005, OpenCV digunakan pada kendaraan Stanley dan memenangkan DARPA *Grand Challenge*, kompetisi untuk kendaraan otonom Amerika [2]. Saat ini, OpenCV mendukung banyak algoritma yang terkait dengan *computer vision* dan pembelajaran mesin (*machine learning*) dan terus berkembang dari hari ke hari. OpenCV dapat berjalan di berbagai bahasa pemrograman, seperti C, C++, Java, Python, dan juga *support* di berbagai *platform* seperti Windows, Linux, Mac OS, iOS dan Android.

Ada beberapa *tools* dan *library* yang perlu disiapkan untuk dapat menjalankan OpenCV-Python, di antaranya adalah Python, Anaconda, numpy, matplotlib, dan tentunya *library* OpenCV itu sendiri. Tahapan mempersiapkan openCV di Python:

BAB

I

Pengenalan OpenCV-Python

Materi

- Instalasi dan Konfigurasi OpenCV-Python
- Memulai dengan Citra

Tujuan Pembelajaran

- Mahasiswa dapat melakukan instalasi OpenCV-Python dan mulai menjalankan program sederhana dengan bahasa pemrograman Python dan *library* OpenCV (membaca, menampilkan, dan menyimpan gambar).

A. Penyajian

Suatu citra didefinisikan sebagai fungsi dua dimensi $f(x,y)$, di mana x dan y adalah koordinat bidang, dan fungsi f pada koordinat (x, y) disebut intensitas atau tingkat abu-abu dari gambar pada titik itu. Jika nilai intensitas f pada koordinat x, y memiliki nilai diskrit yang terbatas maka citra tersebut bisa kita anggap sebagai citra digital. Citra digital disusun dari elemen-elemen dengan nilai pada lokasi tertentu, elemen tersebut disebut sebagai piksel. Piksel adalah istilah yang banyak digunakan untuk menunjukkan unsur atau elemen penyusun gambar digital [1].

Pengolahan citra digital adalah salah satu cabang dari ilmu Komputer yang saat ini menjadi cukup populer. Pengolahan citra digital meliputi mulai dari akuisisi citra, peningkatan kualitas citra (*image enhancement*), perbaikan citra (*image restoration*), transformasi citra, segmentasi, ekstraksi fitur (*feature extraction*), dan pengenalan pola (*pattern recognition*). Cabang dari ilmu ini menjadi populer karena mulai banyak aplikasi yang dapat mengenali objek secara otomatis berbasis citra

Gambar 57.	Citra Output Hasil Segmentasi tanpa Dilakukan Morfologi.	105
Gambar 58.	Citra Output Latihan No.2.....	107
Gambar 59.	Foto Burung Jalak Bali.....	108
Gambar 60.	<i>Circular Neighborhood</i> Delapan <i>Sampling Points</i>	110
Gambar 61.	Contoh perhitungan nilai LBP.....	111
Gambar 62.	Tekstur <i>Uniform Patterns</i>	112
Gambar 63.	Keluaran Kode Program untuk Menampilkan Tekstur <i>Uniform Pattern</i> LBP	114
Gambar 64.	Keluaran Kode Program pada Latihan No.1.	117
Gambar 65.	Contoh Motif Batik Kawung dan Sidomukti.....	118
Gambar 66.	<i>Image Classification</i>	120
Gambar 67.	<i>Image Recognition</i>	120
Gambar 68.	Visualisasi tentang Klasifikasi menggunakan kNN	121
Gambar 69.	Contoh Data Citra Daun.....	126
Gambar 70.	Alur Kerja Klasifikasi Citra Digital.	131
Gambar 71.	<i>5-Fold Cross Validation</i>	135
Gambar 72.	Alur dari Design Eksperimen untuk Klasifikasi Citra Batik.	137

d. OpenCV

Anda dapat meng-installnya melalui Anaconda Navigator atau melalui cmd. Jika meng-install melalui cmd dapat menggunakan kode sebagai berikut:

```
pip install opencv-Python
```

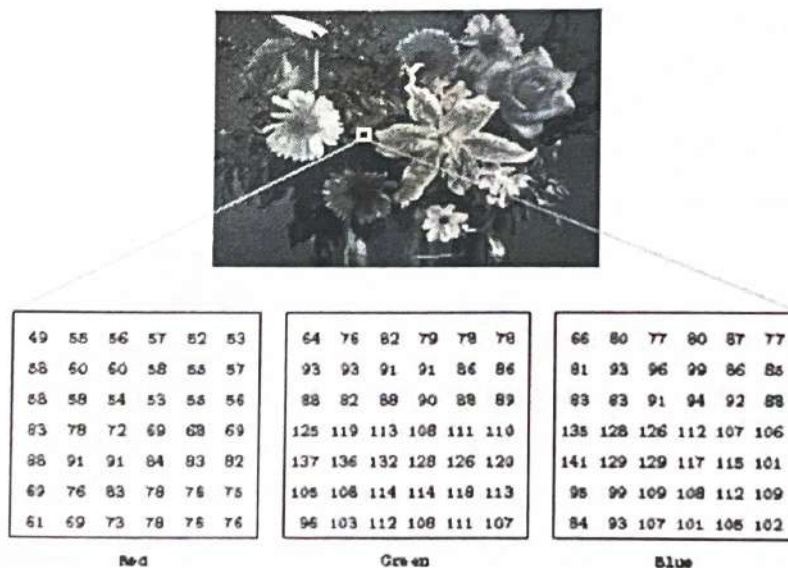
2. Memulai dengan Citra

a. Tipe Dasar Citra Digital

Terdapat 2 tipe dasar dari sebuah citra, antara lain *true color image* dan *grayscale image*. Berikut ini penjelasan dari masing-masing tipe dasar citra.

1) True Color/RGB (Red Green Blue)

Pada tipe citra ini, setiap piksel mempunyai warna tertentu yaitu jumlah warna merah, hijau dan biru di dalamnya. Masing-masing komponen warna berkisaran dari 0-255, maka total komponen warna untuk setiap citra $255^3 = 16.777.216$ kemungkinan perbedaan citra dalam sebuah *image*. Setiap warna direpresentasikan ke dalam 8 *bits* sehingga RGB Image disebut juga *24-bit colour images*.



Gambar 3. RGB Image.

matematika, logika, manipulasi bentuk, pengurutan, pemilihan, I/O aljabar linier dasar, operasi statistik dasar, dan banyak lagi [3]. Fiturnya hampir sama dengan MATLAB dalam mengelola *array* baik dua dimensi ataupun multidimensi. Numpy merupakan salah satu *library* yang digunakan oleh *library* lain seperti OpenCV untuk *computer vision* dan juga Scikit-Learn untuk keperluan analisis data [4].

Cara instalasi *library* Numpy yaitu dapat dilakukan melalui Anaconda Navigator atau melalui cmd. Jika meng-*install* melalui cmd dapat menggunakan kode sebagai berikut:

```
pip install numpy
```

Pada buku ini tidak membahas *library* Numpy secara mendalam, untuk belajar lebih mendalam tentang *library* Numpy dapat kunjungi beberapa *website* berikut ini:

- <https://numpy.org/doc/> [3].

c. Matplotlib

Matplotlib adalah *library* banyak digunakan oleh *data science* untuk menyajikan datanya ke dalam visual yang lebih baik. Dalam pengolahan citra digital, *library* ini bersifat opsional namun disarankan untuk memudahkan visualisasi.

Cara instalasi *library* Matplotlib yaitu dapat dilakukan melalui Anaconda Navigator atau melalui cmd. Jika meng-*install* melalui cmd dapat menggunakan kode sebagai berikut:

```
pip install matplotlib
```

Jika Anda ingin belajar lebih mendalam tentang *library* matplotlib dapat mengunjungi beberapa website di bawah ini:

- <https://matplotlib.org/tutorials/index.html> [5].

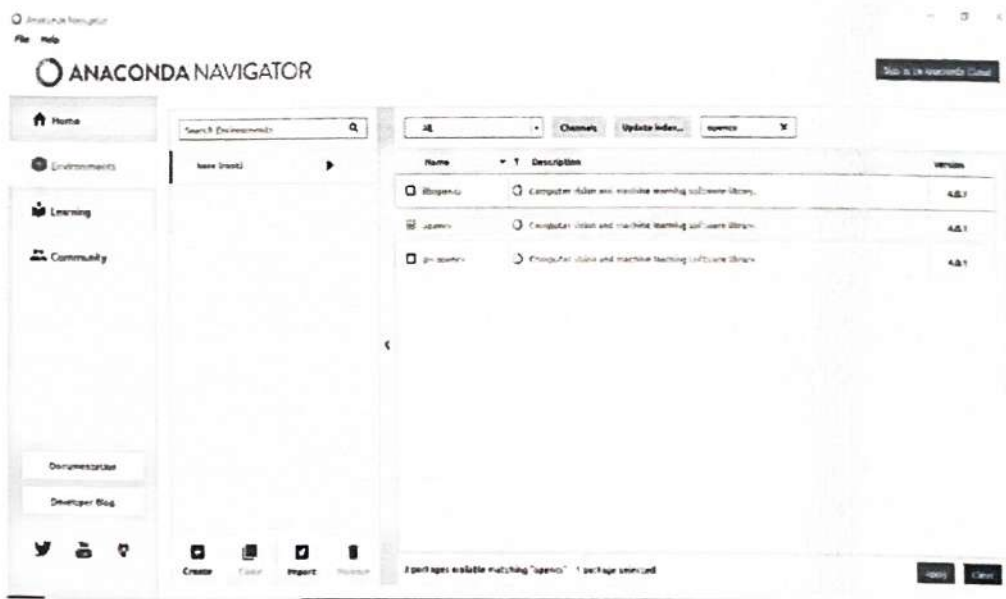
a. Unduh dan *install* Python melalui Anaconda

Anaconda merupakan *package management system* dan *environment management system* yang akan memudahkan pemasangan package dan mengelolanya. Ini *open source* dan dapat bekerja di Linux, OS X dan Windows. Awalnya Anaconda dibuat untuk membuat package Python, sekarang dapat digunakan untuk *software* lainnya.

Setelah Anaconda berhasil di-*install*, Anda dapat memastikan bahwa Python sudah ter-*install* dengan menggunakan Anaconda Prompt dan ketikkan kode sebagai berikut:

```
Python--version
```

Dengan menggunakan Anaconda, Anda juga dapat dengan mudah meng-*install packages* atau *environments* yang dibutuhkan. Untuk meng-*install package* dapat dibuka Anaconda Navigator lalu pilih menu *environments*.



Gambar 2. Tampilan Anaconda Navigator.

b. Numpy

Numpy adalah *library* (pustaka) Python yang menyediakan objek *array* multidimensi, serta operasi cepat pada *array*, termasuk

B. Latihan

1. Buatlah program dengan bahasa pemrograman Python *library* OpenCV untuk membaca satu buah gambar 'upn.jpg'. Pembacaan gambar dilakukan dua kali, di mana untuk pembacaan gambar pertama gambar ditampilkan dengan mode gambar RGB dan gambar kedua ditampilkan dengan mode *grayscale*. Setelah gambar tersebut dibaca, tampilkan kedua gambar tersebut dalam dua jendela berbeda dan simpan gambar mode *grayscale* dengan nama file 'upn_grayscale.jpg'.

```
import numpy as np
import cv2 as cv

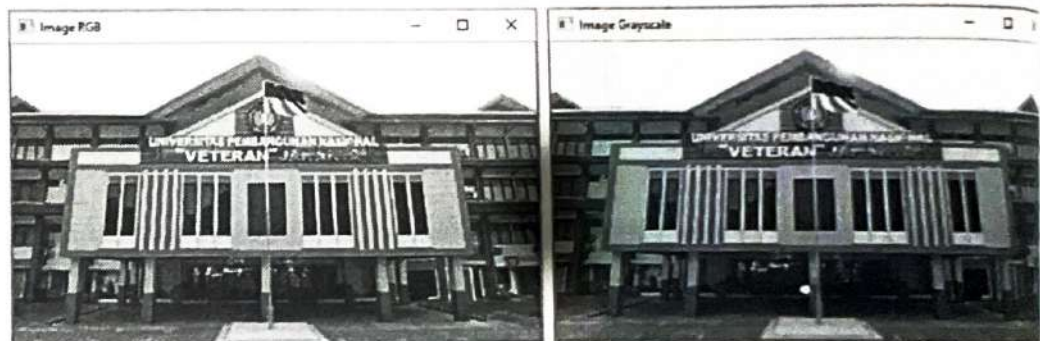
img_rgb = cv.imread('upn.jpg', cv.IMREAD_COLOR)
img_gray = cv.imread('upn.jpg', cv.IMREAD_GRAYSCALE)

cv.imwrite("upn_grayscale.jpg", img_gray)

cv.namedWindow('Image RGB', cv.WINDOW_NORMAL)
cv.imshow('Image RGB', img_rgb)
cv.namedWindow('Image Grayscale', cv.WINDOW_NORMAL)
cv.imshow('Image Grayscale', img_gray)

cv.waitKey(0)
cv.destroyAllWindows()
```

Keluaran:



Gambar 5. Citra Output Latihan 1.

Flag	Keterangan
cv.IMREAD_COLOR	Merupakan flag default. Berfungsi memuat gambar berwarna. Transparansi gambar akan diabaikan.
cv.IMREAD_GRAYSCALE	Membuat gambar dalam mode <i>grayscale</i> .

2) Menampilkan Gambar

Menggunakan fungsi **cv.imshow ()** untuk menampilkan gambar di jendela. Jendela secara otomatis sesuai dengan ukuran gambar. Argumen pertama adalah nama jendela yang bertipe data string. Argumen kedua adalah nama variabel dari citra kita. Anda dapat membuat jendela sebanyak yang Anda inginkan, tetapi dengan nama jendela yang berbeda.

```
cv.imshow(winname, mat)
```

Keterangan parameter

winname : nama jendela yang digunakan untuk menampilkan gambar

mat : citra yang ingin ditampilkan

3) Menyimpan gambar

Gunakan fungsi **cv.imwrite ()** untuk menyimpan gambar. Argumen pertama adalah nama file, argumen kedua adalah gambar yang ingin Anda simpan.

```
retval = cv.imwrite(filename, img)
```

Keterangan:

filename : nama file, tulis nama file beserta dengan format gambar

(contoh: "img.jpg")

img : gambar yang ingin disimpan

2) *Grayscale*

Setiap piksel adalah warna abu-abu, biasanya dari 0 (hitam) sampai 255 (putih). Setiap piksel direpresentasikan oleh 8 bits, atau 1 *byte*.



Gambar 4. *Grayscale Image*.

b. Proses Dasar Pengolahan Citra Digital

Ada tiga proses dasar dalam pengolahan citra digital, yaitu membaca (*read*), menampilkan (*show*), dan menyimpannya kembali (*write*). Ketiga proses tersebut akan sering kita temui dalam pembuatan program pengolahan citra digital.

1) Membaca Gambar

Gunakan fungsi `cv.imread()` untuk membaca gambar. Gambar harus berada di direktori kerja atau dengan menuliskan *path* atau direktori gambar secara lengkap.

```
img = cv.imread(filename, flags)
```

Keterangan Parameter:

filename : file gambar yang ingin dibaca, jika gambar tidak berada dalam direktori kerja yang sama maka sertakan *path* lengkap.

flags : Argumen kedua adalah *flag* yang menentukan cara gambar harus dibaca. Berikut ini. *Flag* yang dapat digunakan:

BAB

II

OPERASI DASAR PADA CITRA DIGITAL

Materi

- Operasi Dasar pada Citra
- Operasi Aritmatika pada Citra

Tujuan Pembelajaran

- Dengan mempelajari bab ini Mahasiswa dapat mengetahui dan mampu menerapkan operasi-operasi dasar dan operasi aritmatika pada citra digital.

A. Penyajian

1. Operasi Dasar pada Citra

Ada empat operasi dasar dalam pengolahan citra digital yaitu 1) mengakses dan memodifikasi piksel, 2) mengakses properti pada citra, 3) menetapkan *Region of Interest* (ROI), dan 4) memecah (*splitting*) dan menggabungkan (*merging*) citra [1]. Berikut ini penjelasan dari keempat operasi dasar pada citra digital.

a. Mengakses nilai dan memodifikasi piksel

Citra disusun oleh piksel-piksel dengan satu atau lebih layer atau *channel*. Piksel yang dalam Bahasa Inggris disebut *pixel* (*picture element*) adalah unsur gambar atau representasi sebuah titik terkecil dalam sebuah gambar grafis [2]. Operasi paling dasar yang akan sering dilakukan adalah mendapatkan nilai dan memodifikasi nilai pada posisi piksel dan *channel* tertentu. Anda dapat mengakses nilai piksel dengan koordinat baris dan kolomnya. Untuk gambar BGR (tiga *channels*), mengembalikan nilai matriks biru (B), hijau (G),



Gambar 6. Citra "PangeranDiponogoro.jpg".

Referensi

- [1] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.
- [2] A. Mordvintsev and A. K., "OpenCV-Python Tutorial Documentation." p. 265, 2014.
- [3] NumPy community, "NumPy Documentation," 2020. <https://numpy.org/doc/>.
- [4] M. Arslan, "5 Library Python untuk Data Science 2017", <https://www.codepolitan.com/>,
<https://www.codepolitan.com/5-library-Python-untuk-data-science-59b774b6cad97>.
- [5] J. Hunter, D. Dale, E. Firing, and M. Droettboom, "Matplotlib Documentation," 2020. <https://matplotlib.org/tutorials/index.html>

Keterangan:

cv.waitKey(0)

Fungsi yang digunakan untuk memberikan batas waktu *keyboard* dalam memproses baris kode selanjutnya. Argumennya adalah waktu dalam *milliseconds*. Artinya, jika diinputkan argument 200 dan tidak ada tombol *keyboard* yang ditekan maka baris kode selanjutnya belum diproses sampai waktu ke 200 *milliseconds*. Untuk nilai argument 0, artinya baris kode selanjutnya akan diproses sampai kita menekan sembarang tombol.

cv.destroyAllWindows()

Fungsi untuk menutup semua jendela yang telah dibuat. Jika Anda ingin menutup jendela tertentu, gunakan fungsi `cv.destroyWindow()` di mana Anda memberikan nama jendela yang tepat sebagai argumen.

cv.namedWindow()

Flag default adalah `cv.WINDOW_AUTOSIZE`, jendela tidak dapat diubah ukurannya. *Flag* `cv.WINDOW_NORMAL`, dapat mengubah ukuran jendela.

C. Lembar Kerja Praktikum

1. Buatlah program untuk membaca gambar pahlawan nasional dari Yogyakarta Pangeran Diponegoro "PangeranDiponogoro.jpg" dengan menggunakan OpenCV-Python. Ketentuan dalam membaca gambar adalah dengan flag mode *grayscale*. Tampilkan hasil pembacaan gambar tersebut ke dalam jendela yang diberi nama "Pangeran Diponegoro-11 November 1785-Yogyakarta" dan simpan gambar mode *grayscale* tersebut dengan nama file 'PangeranDiponogoroGrayscale.jpg'.

```
imgRgb[:, :, 2] = 0
```

2. Operasi Aritmatika pada Citra

a. *Image Addition*

Anda dapat menambahkan dua gambar dengan fungsi OpenCV, `cv.add()` atau hanya dengan operasi numpy, `res = img1 + img2`. Kedua gambar harus memiliki kedalaman (*depth*) dan tipe data gambar yang sama, atau gambar kedua hanya bisa menjadi nilai skalar. Ada perbedaan antara *addition* OpenCV dan penambahan Numpy. Penambahan OpenCV adalah operasi jenuh sementara penambahan Numpy adalah operasi modulo. *Image Addition* juga digunakan untuk menambah tingkat kecerahan pada suatu citra jika citra ditambahkan dengan suatu nilai skalar.

Image addition dengan menggunakan openCV:

```
res = cv.add(img1, img2)
```

Image addition dengan menggunakan operasi numpy:

```
res = img1 + img2
```

b. *Image Subtraction*

Sama seperti penambahan pada gambar, *image subtraction* juga dapat dilakukan dengan openCV dan numpy. *Image subtraction* juga digunakan untuk mengurangi tingkat kecerahan pada suatu citra jika citra ditambahkan dengan suatu nilai skalar.

Image subtraction dengan menggunakan openCV:

```
res = cv.subtract (img1, img2)
```

ROI diperoleh dengan menggunakan *library* Numpy. Berikut ini merupakan kode yang digunakan untuk menerapkan ROI.

```
imgROI = img[x1:x2, y1:y2]
```

d. Memecah (*Splitting*) dan Menggabungkan (*Merging*) Citra

Terkadang Anda perlu bekerja secara terpisah pada setiap *channel* gambar B, G, R. Dalam hal ini, Anda perlu membagi gambar BGR menjadi satu saluran. Dalam kasus lain, Anda mungkin perlu menggabungkan masing-masing saluran ini ke gambar BGR.

Untuk memecah (*splitting*) gambar Anda dapat melakukannya hanya dengan:

```
b,g,r = cv.split(img)
```

atau:

```
b = img[:, :, 0]  
g = img[:, :, 1]  
r = img[:, :, 2]
```

Untuk menggabungkan (*merge*) gambar Anda dapat melakukannya hanya dengan:

```
imgMerge = cv.merge((b,g,r))
```

Untuk menjadikan semua piksel di *channel* merah ke nol, Anda tidak perlu memecah *channel* terlebih dahulu. Pengindeksan dengan menggunakan numpy akan lebih cepat dan mudah.

dan merah (R). Untuk citra grayscale (satu *channel*), mengembalikan nilai intensitas yang sesuai.

Berikut ini merupakan fungsi pada OpenCV untuk mengembalikan nilai pada piksel atau koordinat x,y. fungsi di bawah ini akan mengembalikan variabel pixelVal akan menampung nilai dari matriks citra di piksel atau koordinat x,y (jika mode RGB/BGR akan terdiri dari 3 nilai).

```
pixelVal = img[x,y]
```

Berikut ini merupakan *code* untuk memodifikasi nilai pada piksel atau koordinat x,y. *Code* di bawah ini artinya matriks citra pada koordinat x,y akan memiliki nilai 255 untuk setiap *channel* (B=255, G=255, dan R=255).

```
img[x,y] = [255,255,255]
```

b. Mengakses properti citra

Citra yang telah dibaca menggunakan fungsi pada OpenCV memiliki properti yang dapat kita akses untuk kebutuhan tertentu. Properti pada citra antara lain:

- Bentuk citra : Ukuran baris, kolom, dan *channel* diakses menggunakan *script* **img.shape**
- Ukuran citra : Total jumlah piksel diakses menggunakan *script* **img.size**.
- Tipe data citra: Tipe data dari citra yang dapat diakses menggunakan *script* **img.dtype**

c. Menetapkan *Region of Interest* (ROI)

Terkadang, Anda harus bermain dengan wilayah gambar tertentu. Misalnya, untuk deteksi mata dalam gambar dan deteksi wajah dari keseluruhan citra. Ketika wajah diperoleh, kami memilih wilayah wajah saja dan mencari mata di dalamnya daripada mencari seluruh gambar. Ini meningkatkan akurasi dan kinerja.

2. Buatlah program dengan OpenCv-Python untuk mengakses dan memodifikasi piksel serta *split* dan *merge* citra.

```
import numpy as np
import cv2 as cv

imgRgb = cv.imread('upn.jpg')
imgGrayscale = cv.imread('upn.jpg', cv.IMREAD_GRAYSCALE)

pxRgb = imgRgb[100,100]
blue = imgRgb[100,100,0]
imgRgb[100,100] = [255,255,255]
pxGrayscale = imgGrayscale[100,100]
imgGrayscale[100,100] = 255

print(pxRgb)
print(blue)
print(imgRgb[100,100])
print(pxGrayscale)
print(imgGrayscale[100,100])

b,g,r = cv.split(imgRgb)
imgRgbMerge = cv.merge((b,g,r))
b = imgRgb[:, :, 0]
imgRgb[:, :, 2] = 0

cv.namedWindow('Image imgRgb Layer Blue', cv.WINDOW_NORMAL)
cv.imshow('Image imgRgb Layer Blue', b)
cv.namedWindow('Image imgRgb dengan Layer Red=0',
cv.WINDOW_NORMAL)
cv.imshow('Image imgRgb dengan Layer Red=0', imgRgb)
cv.waitKey(0)
cv.destroyAllWindows()
```

Keluaran:

```
[255 255 255]
111
[255 255 255]
125
255
```

```
print("\nGrayscale Image')  
print ( imgGrayscale.shape )  
print( imgGrayscale.size )  
print( imgGrayscale.dtype )
```

Keluaran:

RGB Image
(284, 500, 3)
426000
uint8

Grayscale Image
(284, 500)
142000
uint8

Keterangan:

Bentuk (*shape*) citra BGR/RGB keluarannya 3 nilai yaitu (284, 500, 3) di mana 284 adalah ukuran tinggi/baris dari citra, 500 adalah ukuran lebar/kolom dari citra, dan 3 adalah jumlah *channel* atau layer dari citra. Sedangkan bentuk atau *shape* dari citra grayscale keluarannya adalah 2 nilai yaitu (284, 500), tidak ada jumlah *channel* atau layer karena jumlah *channel* pada citra grayscale hanya 1 *channel* saja. Size dari citra BGR juga berbeda dari citra grayscale, di mana untuk citra BGR adalah 426000 (284 x 500 x 3) dan citra grayscale adalah 142000 (284 x 500). Untuk tipe data kedua citra tersebut adalah sama yaitu bertipe data integer dengan nilai perpixselnya adalah 8 bit (0-255).

Image subtraction dengan menggunakan operasi numpy:

```
res = img1 + img2
```

c. *Bitwise Operations*

Operasi bitwise pada citra adalah melakukan pengoperasian satu atau lebih gambar menggunakan operator logika pada citra. Operasi bitwise antara lain AND, OR, NOT, dan XOR.

Operasi bitwise	Code OpenCV
AND	<code>dst = cv.bitwise_and(src1, src2, mask)</code>
OR	<code>dst = cv.bitwise_not(src1, src2, mask)</code>
NOT	<code>dst = cv.bitwise_or(src1, src2, mask)</code>
XOR	<code>dst = cv.bitwise_xor(src1, src2, mask)</code>

Keterangan:

src1 : input array pertama atau skalar.

src2 : input array kedua atau skalar.

mask : opsional, array saluran tunggal 8-bit, yang menentukan elemen array output yang akan diubah.

B. Latihan

1. Membuat program dengan OpenCV-Python untuk melihat perbedaan properti dari citra BGR/RGB dengan citra *grayscale*.

```
import numpy as np
import cv2 as cv

imgRgb = cv.imread('upn.jpg')
imgGrayscale = cv.imread('upn.jpg', cv.IMREAD_GRAYSCALE)

print('\nRGB Image')
print ( imgRgb.shape )
print( imgRgb.size )
print( imgRgb.dtype )
```

error atau coba-coba sampai mendapatkan posisi tiang bendera yang tepat.

- Selanjutnya kopikan tiang bendera tersebut ke posisi lain dengan kode `imgRgb[25:280, 20:80] = tiang`. Nilai `[25:280, 20:80]` juga diperoleh dari trial and error.

4. Membuat program untuk menggabungkan dua buah citra. Citra yang digunakan adalah logo UPN Veteran Jakarta dan foto Kampus UPN Veteran Jakarta.

```
import numpy as np
import cv2 as cv

imgUpn = cv.imread('upn.jpg')
imgUpnLogo = cv.imread('LogoUPNVJ.png')

# meletakkan logo di sudut kiri atas, dan menyimpannya membuat ROI
rows,cols,channels = imgUpnLogo.shape
roi = imgUpn[0:rows, 0:cols ]

# Buat sebuah mask dari logo dan mask inverse logo
img2gray = cv.cvtColor(imgUpnLogo,cv.COLOR_BGR2GRAY)
ret, mask = cv.threshold(img2gray, 10, 255, cv.THRESH_BINARY)
mask_inv = cv.bitwise_not(mask)

# area black-out dari logo dalam ROI
img1_bg = cv.bitwise_and(roi,roi,mask = mask_inv)

# ambil hanya region dari logo dalam gambar logo.
img2_fg = cv.bitwise_and(imgUpnLogo,imgUpnLogo,mask = mask)

# letakan logo dalam ROI dan modifikasi citra utama
dst = cv.add(img1_bg, img2_fg)
imgUpn[0:rows, 0:cols ] = dst

cv.imshow('Logo UPNVJ', imgUpn)
cv.waitKey(0)
cv.destroyAllWindows()
```

3. Lakukan penduplikasian gambar bendera pada citra upn.jpg sedemikian sehingga bendera terlihat menjadi 2 pada posisi yang berbeda.

```
import numpy as np
import cv2 as cv

imgRgb = cv.imread('upn.jpg')

tiang = imgRgb[25:280, 230:290]
imgRgb[25:280, 20:80] = tiang

cv.imshow('Gambar', imgRgb)
cv.waitKey(0)
cv.destroyAllWindows()
```

Keluaran:



Gambar 8. Output Citra pada Latihan No. 3.

Keterangan:

- Untuk kode `tiang = imgRgb[25:280, 230:290]` artinya mengambil region of interest citra `imgRGB` di posisi baris 25-280 dan posisi kolom 230-290. Nilai `[25:280, 230:290]` diperoleh dari trial and



Gambar 7. Output Citra pada Latihan No. 2.

Keterangan:

- $pxRgb = imgRgb[100,100]$ yaitu mengakses matriks citra $imgRgb$ pada posisi koordinat piksel 100,100 di semua channel (B, G, dan R).
- $blue = imgRgb[100,100,0]$ yaitu mengakses matriks citra $imgRgb$ pada posisi koordinat piksel 100,100 hanya di channel blue saja.
- $imgRgb[100,100] = [255,255,255]$ yaitu mengubah nilai matriks citra $imgRgb$ pada posisi koordinat piksel 100,100 untuk channel $B=255$, $G=255$, dan $R=255$.
- $b = imgRgb[:, :, 0]$ yaitu mengakses matriks citra $imgRgb$ di semua baris dan semua kolom pada channel ke-0 (channel *blue*).
- $imgRgb[:, :, 2] = 0$ yaitu memberikan nilai 0 untuk matriks citra $imgRgb$ di semua baris dan semua kolom pada channel ke-2 (channel *red*).



Gambar 14. Keluaran yang Diharapkan dari Penggabungan Kedua Citra.

Referensi

- [1] A. Mordvintsev and A. K, "OpenCV-Python Tutorials Documentation." p. 265, 2014.
- [2] Wikipedia, "Piksel," 2019. <https://id.wikipedia.org/wiki/Piksel>.

C. Lembar Kerja Praktikum

1. Baca 2 buah gambar dengan format *.jpg, yaitu foto Presiden pertama Indonesia Ir. Soekarno dan Wakil Presiden pertama Indonesia Dr. Drs. H. Mohammad Hatta. Ubah mode kedua gambar tersebut menjadi *grayscale* dan tampung ke dalam 2 variabel yang berbeda.



Gambar 12. Soekarno.jpg.



Gambar 13. Hatta.jpg.

2. Jika kedua memiliki ukuran yang berbeda, maka sesuaikan ukuran kedua gambar tersebut sedemikian sehingga memiliki ukuran piksel yang sama, yaitu dengan cara mengambil *shape* (*rows* dan *cols*) dari gambar yang lebih kecil dan dapatkan ROI gambar yang lebih besar sesuai dengan *shape* gambar yang lebih kecil.
3. Lakukan operasi terhadap kedua gambar tersebut (yang ukurannya sudah sama atau sesuai):
 - a. *Image Addition*
 - b. *Image Subtraction*
4. Tampilkan empat citra:
 - a. Citra *grayscale* pertama
 - b. Citra *grayscale* kedua
 - c. Citra hasil *image addition*
 - d. Citra hasil *image subtraction*
5. Jelaskan hasil dari operasi *Image Addition* dan *Image Subtraction*.
6. Buatlah program dengan OpenCV-Python untuk menggabungkan kedua gambar Ir. Soekarno dan Dr. Drs. H. Mohammad Hatta sedemikian sehingga gambar menjadi sebagai berikut:

Keluaran:



Gambar 9. LogoUPNVJ.png.

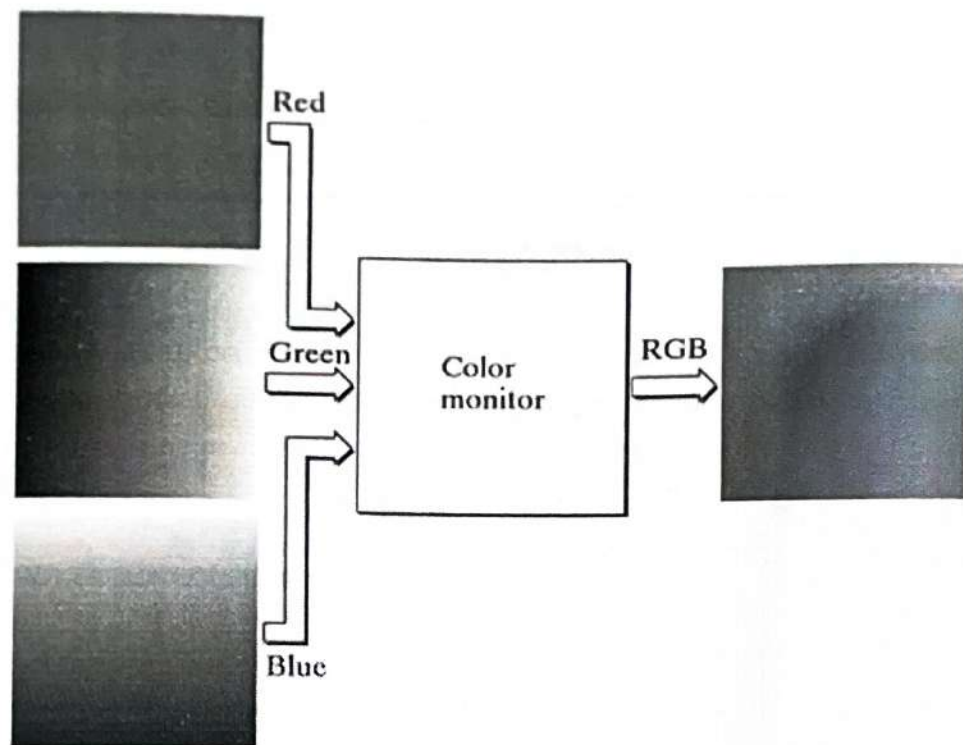


Gambar 10. upn.jpg.



Gambar 11. Hasil Penggabungan.

gambar di bawah ini. Jumlah bit yang digunakan untuk mewakili setiap piksel dalam ruang RGB disebut kedalaman piksel (pixel depth). Pada citra RGB di masing-masing channel merah, hijau, dan biru menampung 8-bit per-piksel. Maka setiap titik pada gambar akan menampung 3×8 bit, atau 24 bit per-titik pada gambar RGB. Istilah gambar full-color ini sering disebut sebagai gambar warna RGB 24-bit. RGB digunakan untuk menghasilkan warna pada CRT, monitor dan display.

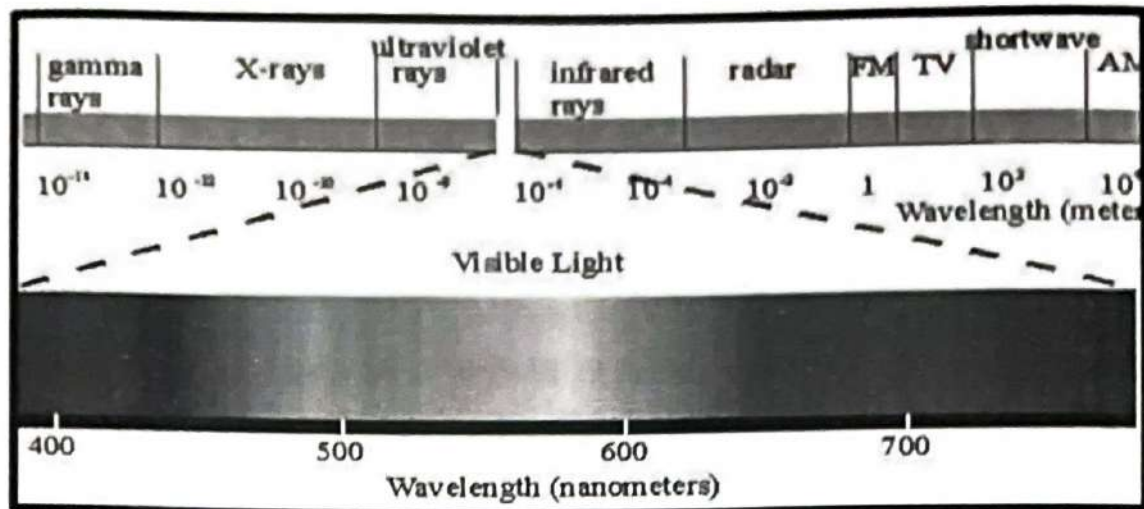


Gambar 16. CRT Display.

b. Model Warna CMY/CMYK

Model warna CMYK (cyan, magenta, yellow, dan key) adalah proses pencampuran pigmen yang biasanya digunakan percetakan. Tinta elemen cyan, magenta, yellow, dan lebih lengkapnya ditambah elemen black. Keempat elemen tersebut dicampurkan dengan komposisi tertentu dan akurat sehingga menghasilkan warna tepat seperti yang diinginkan. Sistem CMY(K) juga digunakan oleh banyak printer warna dan mesin fotokopi. Maka dari itu diperlukan

500 hingga 570 nm, sedangkan warna biru memiliki panjang gelombang 460 nm. [1].



Gambar 15. Spektrum Warna

Cahaya yang kita lihat adalah bukan cahaya dengan satu panjang gelombang melainkan kumpulan panjang gelombang tertentu. Warna terbentuk dari kumpulan gelombang dengan panjang gelombang yang berbeda-beda. Ini bisa diartikan bahwa warna yang kita lihat adalah kombinasi dari beberapa elemen dasar warna.

2. Model Warna

Cara penyajian campuran elemen dasar untuk menghasilkan warna dinamakan dengan Color Space atau Ruang Warna. Beberapa macam ruang warna dalam pengolahan citra digital, dan yang paling umum ditemui adalah ruang warna RGB, yaitu model ruang warna yang memiliki 3 channel yaitu red, green, blue. Masih ada beberapa lagi ruang warna yang dapat kita temui dalam pengolahan citra digital, di antaranya grayscale, CMYK, HSV, YCbCr. Berikut ini penjelasannya:

a. Model Warna RGB/BGR

Gambar pada model warna RGB terdiri dari tiga komponen gambar merah (*red*), hijau (*green*), dan biru (*blue*). Pada monitor yang menggunakan model warna RGB, ketiga gambar ini digabungkan untuk menghasilkan gambar warna komposit, dapat dilihat pada

BAB

III

RUANG WARNA PENGOLAHAN CITRA DIGITAL

Materi

- Konsep Ruang Warna
- Model Warna
- Transformasi Ruang Warna pada Citra Digital

Tujuan Pembelajaran

- Mahasiswa dapat menjelaskan konsep ruang warna pada pengolahan citra digital.
- Mahasiswa mampu menyebutkan dan menjelaskan model warna pada pengolahan citra digital.
- Mahasiswa mampu melakukan transformasi warna pada pengolahan citra digital menggunakan Python-openCV.

A. Penyajian

1. Konsep Warna

Pada dasarnya, warna pada objek yang dilihat manusia ditentukan oleh sifat cahaya yang dipantulkan dari objek tersebut. Seperti yang diilustrasikan pada Gambar 15, cahaya tampak terdiri dari pita frekuensi yang relatif sempit dalam spektrum elektromagnetik, yaitu berkisar antara 380-780 nanometer. Secara teknik, warna adalah spektrum tertentu yang terdapat di dalam suatu cahaya sempurna (berwarna putih). Benda yang mendukung pemantulan dalam rentang spektrum yang terlihat menunjukkan beberapa corak warna. Identitas suatu warna ditentukan panjang gelombang cahaya tersebut. Misalnya, objek berwarna hijau memantulkan cahaya dengan panjang gelombang terutama dalam kisaran

Keterangan:

- `dst` : *output* citra dengan size dan kedalaman yang sama dengan citra input
- `src` : citra input
- `code` : konversi ruang warna

Dalam OpenCV Ada lebih dari 150 metode atau konversi kode ruang warna yang tersedia di OpenCV, dapat dilihat pada link:

https://docs.opencv.org/4.1.2/d8/d01/group__imgproc__color__conversions.html

a. RGB/BGR ke *Grayscale*

Transformasi ruang RGB adalah konversi ke/dari warna RGB 16-bit (R5: G6: B5 atau R5: G5: B5), serta konversi ke/dari skala abu-abu menggunakan:

RGB[A] to Gray: $Y \leftarrow 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$

Gray to RGB[A]: $R \leftarrow Y, G \leftarrow Y, B \leftarrow Y, A \leftarrow \max(ChannelRange)$

Berikut ini kode konversi ruang warna yang dapat digunakan untuk transformasi ruang warna RGB/BGR ke *grayscale* atau sebaliknya [2].

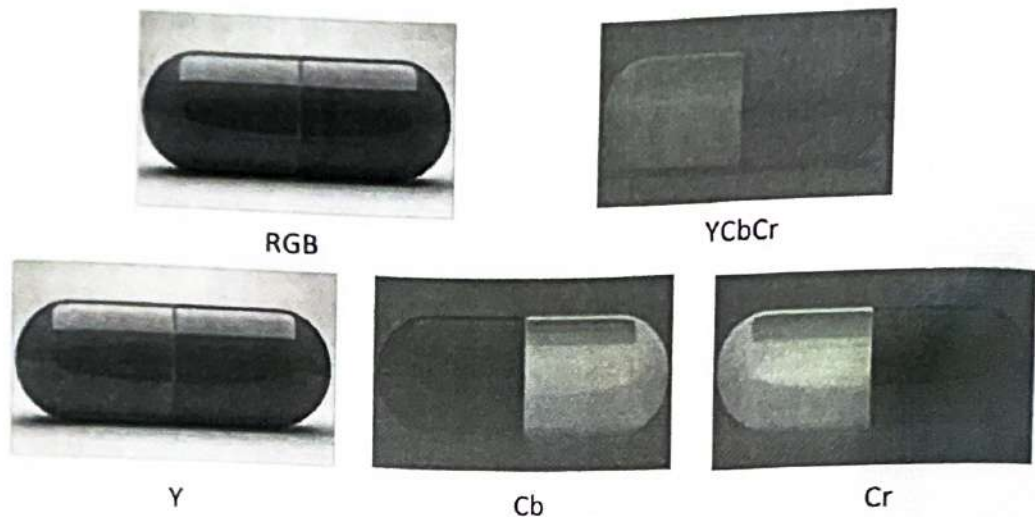
```
cv.COLOR_BGR2GRAY  
cv.COLOR_RGB2GRAY  
cv.COLOR_GRAY2BGR  
cv.COLOR_GRAY2RGB
```

b. RGB/BGR ke HSV dan HSV ke RGB/BGR

Berikut ini adalah formula konversi citra RGB ke HSV dengan citra RGB yang telah dikonversi ke format floating-point dan diskalakan agar sesuai dengan rentang 0 hingga 1.

d. Model Warna YCbCr

Komponen penyusun citra YCbCr antara lain Y merupakan komponen luma sedangkan Cb dan Cr merupakan komponen kroma. Perbedaan antara biru dan merah. Pada Gambar 18, menampilkan bahwa gambar Y merupakan citra skala abu-abu (greyscale) dari gambar aslinya. Pada gambar, salju ditampilkan dengan nilai yang sedang (tidak rendah dan tidak tinggi) antara Cr dan Cb; gundukan ditampilkan oleh Cb yang lemah dan Cr yang kuat; rumput hijau ditampilkan oleh Cb dan Cr yang sama-sama lemah; serta langit biru ditampilkan oleh Cb kuat dengan Cr lemah.



Gambar 18. Citra RGB dan Citra YCbCr dari Setiap Channel-nya.

3. Transformasi Ruang Warna pada Citra Digital

Transformasi ruang warna adalah proses mengubah ruang warna satu ke bentuk ruang warna lainnya. Dalam pengolahan citra digital akan sering sekali kita melakukan transformasi ini, misalnya, mengubah citra RGB ke grayscale, RGB ke HSV atau sebaliknya, RGB ke YCbCr atau sebaliknya. Berikut ini adalah fungsi untuk transformasi ruang warna pada citra digital:

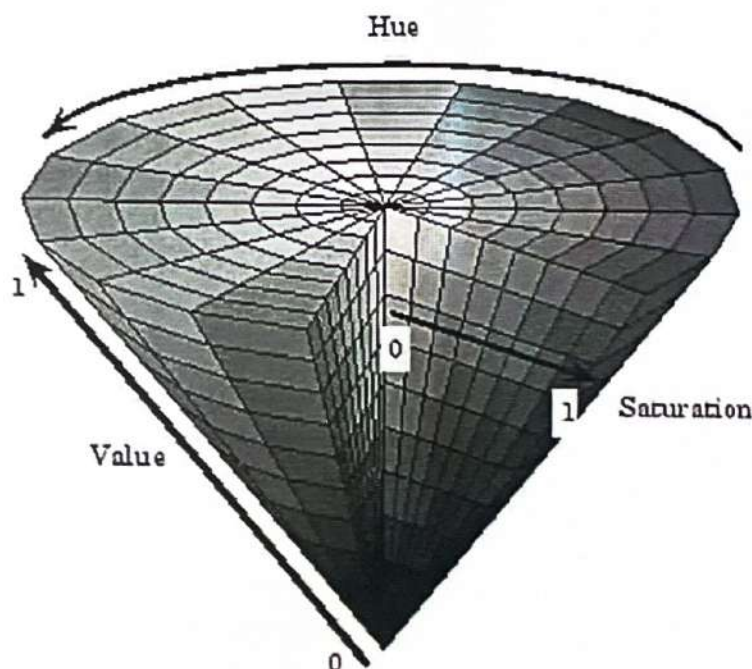
```
dst=cv.cvtColor(src, code)
```

input data CMY(K) atau melakukan konversi dari RGB ke CMY(K). Konversi ini dilakukan dengan menggunakan operasi sederhana.

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

c. Model Warna HSV

Ruang warna HSV (*Hue, Saturation, Value*). *Hue* merupakan suatu nilai yang merepresentasikan spektrum warna dari cahaya tampak (merah, jingga, kuning, hijau, biru, dan ungu). *Saturation* merupakan nilai yang menunjukkan tingkat kejenuhan atau kemurnian dari suatu warna. Semakin besar nilai saturasi maka semakin murni warna yang dihasilkan. Sedangkan *value* dapat didefinisikan sebagai nilai yang menunjukkan tingkat kecerahan warna. HSV dikembangkan menggunakan sistem koordinat polar dan banyak digunakan untuk fitur warna pada gambar. Gambar 17 adalah representasi ruang warna pada HSV.



Gambar 17. Representasi Ruang Warna pada HSV.

```
#menyimpan file citra hasil transformasi
cv.imwrite('Capsule Grayscale.jpg',imgGrayscale)
cv.imwrite('Capsule HSV.jpg',imgHSV)
cv.imwrite('Capsule YCbCr.jpg',imgYCrCb)

cv.imshow('RGB',imgCr)
cv.imshow('Grayscale',imgGrayscale)
cv.imshow('HSV',imgHSV)
cv.imshow('YCbCr',imgYCrCb)
cv.imshow('Y',imgY)
cv.imshow('Cb',imgCb)
cv.imshow('Cr',imgCr)
cv.waitKey(0)
cv.destroyAllWindows()
```

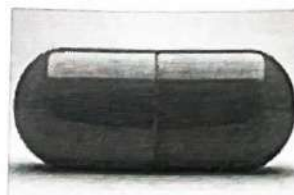
Keluaran Console:

Shape RGB : (800, 1480, 3)
 Shape Grayscale : (800, 1480)
 Shape HSV : (800, 1480, 3)
 Shape YcbCr : (800, 1480, 3)

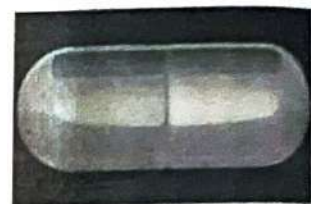
Keluaran Citra:



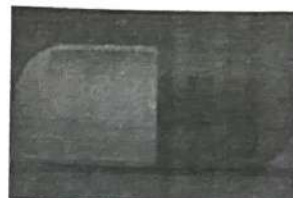
RGB



Grayscale



HSV



YCbCr

Berikut ini kode konversi ruang warna yang dapat digunakan untuk transformasi ruang warna RGB/BGR ke YCbCr atau sebaliknya [2].

```
cv.COLOR_BGR2YCrCb  
cv.COLOR_RGB2YCrCb  
cv.COLOR_YCrCb2BGR  
cv.COLOR_YCrCb2RGB
```

B. Latihan

1. Membuat program untuk mengubah citra RGB menjadi citra *grayscale*, HSV, YCbCr kemudian menampilkan citra dan disimpan ke dalam file baru dengan menggunakan *library* OpenCV.

```
import numpy as np  
import cv2 as cv  
  
#membaca citra RGB  
imgRgb = cv.imread('capsule.jpg')  
  
#transformasi citra RGB ke grayscale  
imgGrayscale = cv.cvtColor(imgRgb, cv.COLOR_BGR2GRAY)  
  
#transformasi citra RGB ke HSV  
imgHSV = cv.cvtColor(imgRgb, cv.COLOR_BGR2HSV)  
  
#transformasi citra RGB ke YCrCb  
imgYCrCb = cv.cvtColor(imgRgb, cv.COLOR_BGR2YCrCb)  
imgY = imgYCrCb[:, :, 0];  
imgCr = imgYCrCb[:, :, 1];  
imgCb = imgYCrCb[:, :, 2];  
  
#mencetak shape dari ke-empat ruang warna  
print('Shape RGB: '+str(imgRgb.shape))  
print('Shape Grayscale: '+str(imgGrayscale.shape))  
print('Shape HSV: '+str(imgHSV.shape))  
print('Shape YCbCr: '+str(imgYCrCb.shape))
```

$$\begin{aligned}
 V &\leftarrow \max(R, G, B) \\
 S &\leftarrow \begin{cases} \frac{V - \min(R, G, B)}{V} & \text{if } V \neq 0 \\ 0 & \text{otherwise} \end{cases} \\
 H &\leftarrow \begin{cases} 60(G - B)/(V - \min(R, G, B)) & \text{if } V = R \\ 120 + 60(B - R)/(V - \min(R, G, B)) & \text{if } V = G \\ 240 + 60(R - G)/(V - \min(R, G, B)) & \text{if } V = B \end{cases}
 \end{aligned}$$

Berikut ini kode konversi ruang warna yang dapat digunakan untuk transformasi ruang warna RGB/BGR ke HSV atau sebaliknya [2].

```

cv.COLOR_BGR2HSV
cv.COLOR_RGB2HSV
cv.COLOR_HSV2BGR
cv.COLOR_HSV2RGB

```

c. RGB/BGR ke YCbCr dan YCbCr ke RGB/BGR

Berikut ini adalah formula konversi citra RGB ke YCrCb

$$\begin{aligned}
 Y &\leftarrow 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \\
 Cr &\leftarrow (R - Y) \cdot 0.713 + \text{delta} \\
 Cb &\leftarrow (B - Y) \cdot 0.564 + \text{delta} \\
 R &\leftarrow Y + 1.403 \cdot (Cr - \text{delta}) \\
 G &\leftarrow Y - 0.714 \cdot (Cr - \text{delta}) - 0.344 \cdot (Cb - \text{delta}) \\
 B &\leftarrow Y + 1.773 \cdot (Cb - \text{delta})
 \end{aligned}$$

di mana:

$$\text{delta} = \begin{cases} 128 & \text{for 8-bit images} \\ 32768 & \text{for 16-bit images} \\ 0.5 & \text{for floating-point images} \end{cases}$$

melakukan konversi ruang warna terlebih dahulu dari ruangan warna RGB/BGR ke YCbCr, dan hasil pemrosesan citra tersebut menghasilkan dua *output* citra dengan ketentuan sebagai berikut:

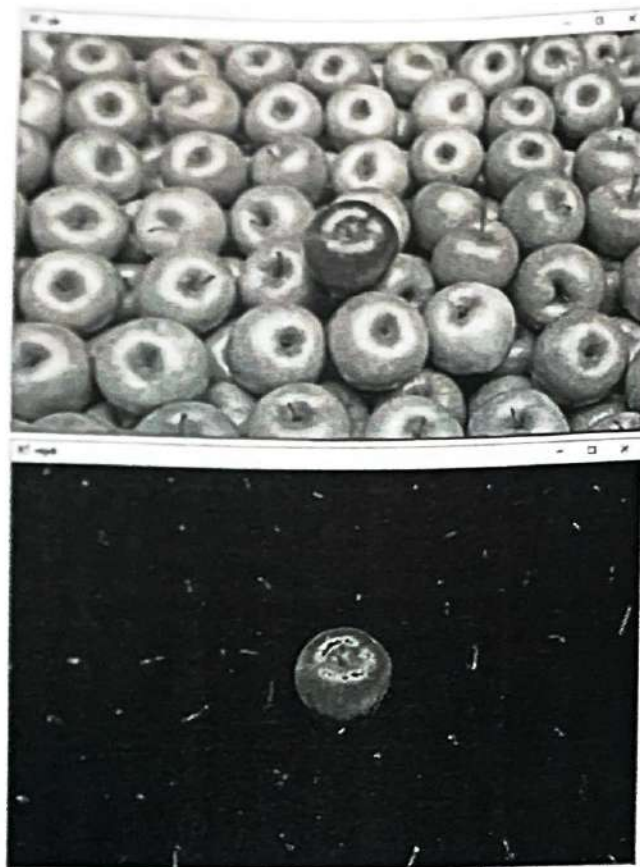
- a. *Output* citra pertama menghasilkan citra dengan latar belakang (*background*) citra menjadi hitam (atau bernilai 0) dan **objek citra dominan warna biru** yang kalian ambil menjadi warna sebenarnya (*rgb true color*).
- b. *Output* citra kedua menghasilkan citra dengan latar belakang (*background*) citra menjadi hitam (atau bernilai 0) dan **objek citra dominan warna merah/coklat** yang kalian ambil menjadi warna sebenarnya (*rgb true color*).

Referensi

- [1] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.
- [2] A. Mordvintsev and A. K, "OpenCV-Python Tutorials," 2019. https://docs.opencv.org/4.1.2/d6/d00/tutorial_py_root.html.

```
res = cv.bitwise_and(rgb, rgb, mask= mask)
cv.imshow('rgb', res)
cv.waitKey(0)
cv.destroyAllWindows()
```

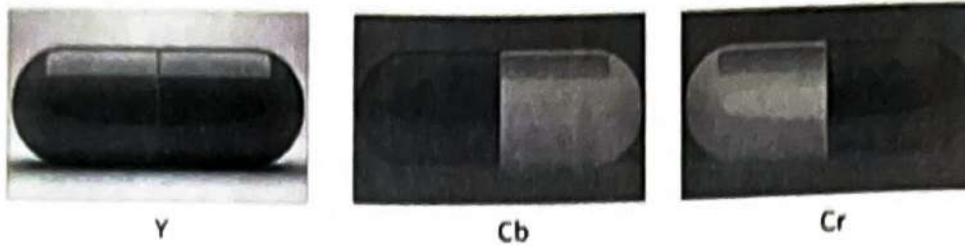
Keluaran:



Gambar 20. Output Citra Latihan No. 2.

C. Lembar Kerja Praktikum

1. Lakukan analisa singkat dari hasil program yang telah dibuat pada latihan nomor 2.
2. Lakukan akuisisi citra menggunakan kamera dengan objek bebas, dalam satu citra terdapat dua objek (satu objek dominan warna biru dan satu objek lainnya dominan warna merah/coklat). Kedua objek tersebut diakuisisi di atas kertas putih. Lakukan segmentasi dengan



Gambar 19. Output Citra Latihan No. 1.

2. Setelah kita mengetahui bagaimana mengkonversi gambar BGR/RGB ke *grayscale*, HSV, YCbCr/YCrCb, kita dapat menggunakannya untuk mengekstrak objek berwarna. Dalam YCbCr, lebih mudah untuk mewakili warna daripada di ruang warna BGR. Dalam aplikasi, Kita akan mencoba mengekstraksi objek apel berwarna merah. Jadi inilah caranya:
 - Baca citra dengan nama file 'red_apple.jpg'.
 - Konversi citra dari BGR ke ruang warna YCbCr/YCrCb.
 - Kami memberi ambang batas (*threshold*) gambar YCbCr/YCrCb untuk rentang warna merah.
 - Ekstrak objek merah saja, kita bisa melakukan apa saja pada gambar yang kita inginkan.
 - Di bawah ini adalah kode yang dikomentari secara rinci:

```
import numpy as np
import cv2 as cv

#membaca citra RGB
rgb = cv.imread('red_apple.jpg')
yrcrb = cv.cvtColor(rgb, cv.COLOR_BGR2YCrCb)

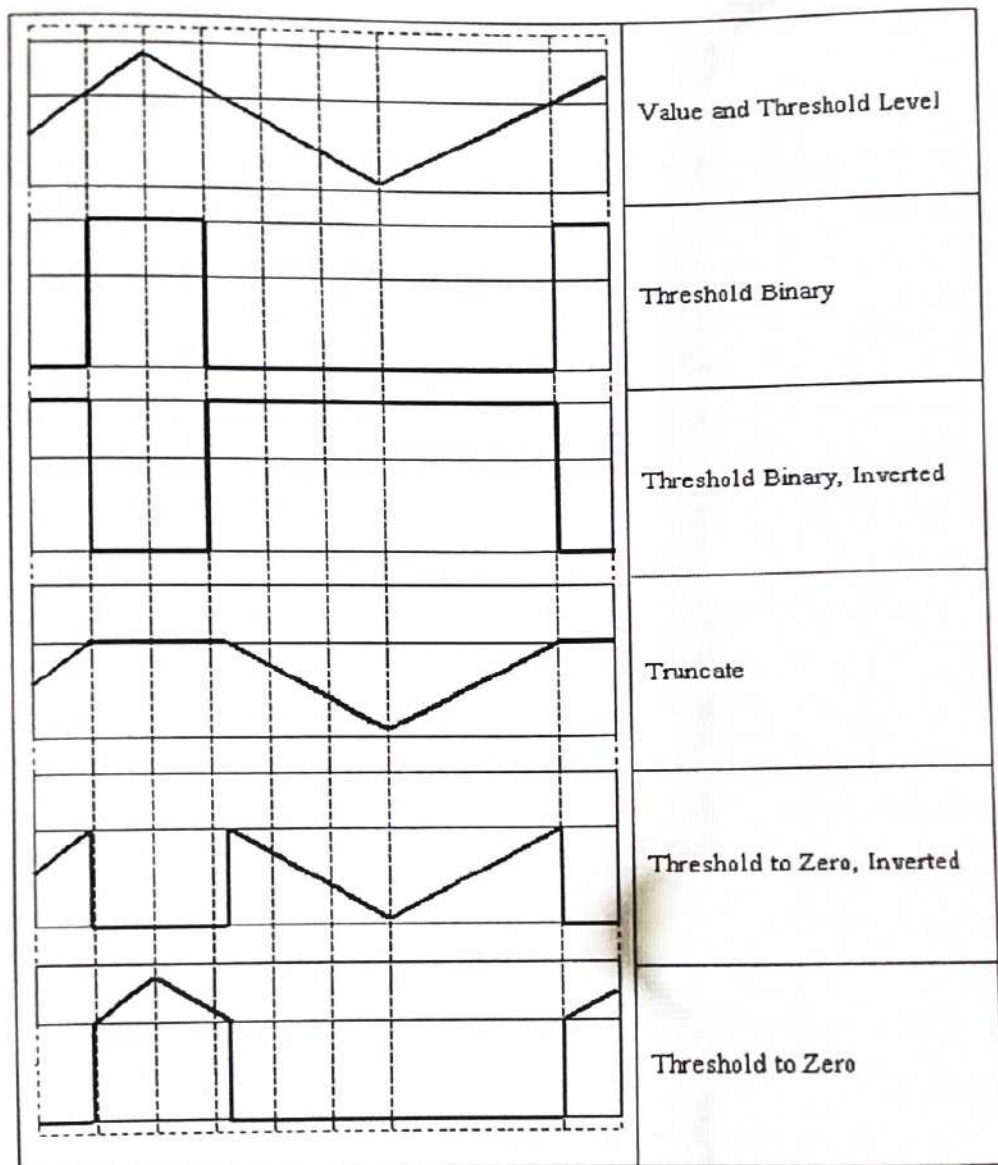
lower_blue = np.array([40,150,50])
upper_blue = np.array([200,255,150])

# Threshold the HSV image to get only red colors
mask = cv.inRange(yrcrb, lower_blue, upper_blue)
```

a. *Thresholding*

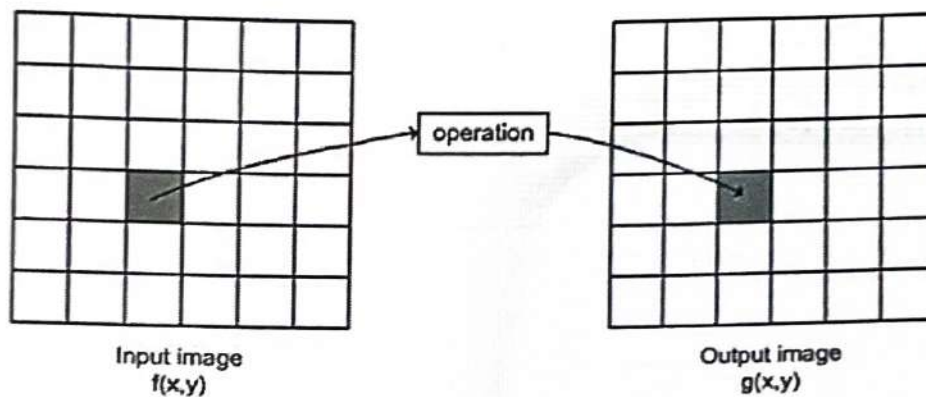
Thresholding merupakan teknik pengolahan citra berbasis piksel (*point processing*) yang banyak digunakan untuk segmentasi, yaitu dengan cara memberikan satu nilai (mungkin putih) jika piksel lebih besar dari nilai ambang batas (*threshold*) dan memberi nilai lain (mungkin hitam) jika piksel lebih kecil dari nilai ambang batas. Segmentasi adalah pemisahan objek dari latar belakang citra (*image background*).

Jenis operasi *threshold* (*threshold type*):



2. Point Processing

Point processing merupakan operasi yang paling sederhana, namun juga merupakan operasi pengolahan citra yang paling sering digunakan. Prinsip *point processing* yaitu sebuah piksel pada citra masukan diproses dan hasilnya disimpan pada posisi yang sama pada citra keluaran. Katakanlah kita memiliki citra masukan awal $f(x,y)$ dan ingin dilakukan manipulasi yang menghasilkan gambar yang berbeda $g(x,y)$. Pemrosesan titik didefinisikan sebagai operasi untuk menghitung nilai baru piksel $g(x,y)$ berdasarkan nilai piksel pada posisi yang sama di $f(x,y)$ dengan menggunakan operasi. Dalam *point processing*, tidak melihat nilai-nilai tetangga piksel di $f(x,y)$ [2]. Prinsip pemrosesan titik diilustrasikan pada Gambar 22.



Gambar 22. Gambaran *Point Processing*

Berikut ini adalah beberapa contoh *point processing* yang paling mendasar dalam pemrosesan citra digital yang biasa dilakukan:

- *Thresholding*
- *Image Negative*
- *Image Contrast*
- Operasi aritmetika (penjumlahan/pengurangan citra, perkalian/pembagian dua buah citra, perkalian/pembagian citra dengan skalar)
- Operasi Boolean
- Operasi Geometri (translasi, rotasi, penskalaan, dilatasi, distorsi, *image morphing*)

BAB IV PENGOLAHAN TITIK

Materi

- *Image enhancement*
- Pengolahan Titik (*point processing*)

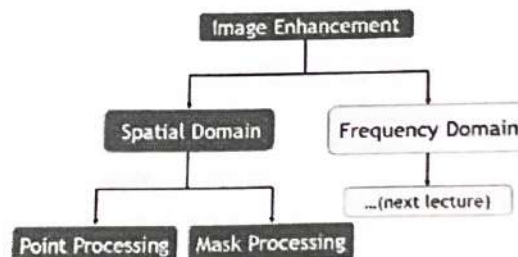
Tujuan Pembelajaran

- Mahasiswa dapat melakukan *image enhancement* pada citra dengan *point processing*.

A. Penyajian

1. Image Enhancement

Image enhancement adalah proses memanipulasi atau meningkatkan kualitas citra agar hasilnya dapat lebih **baik** dari citra awal untuk aplikasi tertentu. Kriteria **baik** tergantung pada aplikasi dan *problem* secara visual maupun secara otomatis (untuk aplikasi selanjutnya). Misalnya metode yang cukup berguna untuk menyempurnakan citra sinar-X, namun mungkin bukan pendekatan terbaik untuk meningkatkan kualitas citra satelit yang diambil dalam pita inframerah spektrum elektromagnetik [1]. Terdapat dua kategori pada *image enhancement* yang dapat dilihat pada Gambar 21.



Gambar 21. *Image Enhancement*

dihitung sebagai:
 $(double)dsize.width/src.cols$
fy : faktor skala sumbu Y; ketika sama dengan 0, itu dihitung sebagai:
 $(double)dsize.height/src.rows$
interpolation_flag : Metode untuk interpolasi, seluruh flag interpolasi dapat dilihat pada link sebagai berikut,
https://docs.opencv.org/4.1.2/da/d54/group__imgproc__transform.html

2) Translation

Translation adalah pergeseran lokasi objek. Untuk melakukan translasi citra dapat menggunakan matriks transformasi sebagai berikut:

$$M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$$

Pada OpenCV dapat dilakukan dengan memanggil fungsi `cv.warpAffine()`.

```
dst = cv.warpAffine(src, M, dsize)
```

src : Citra input
dst : Citra keluaran
M : matriks transformasi ukuran 2x3
dsize : ukuran gambar output, berupa ukuran baris dan kolom gambar output. Misal: (230, 300)

b. *Image Contrast*

Kontras menyatakan sebaran terang (*lightness*) dan gelap (*darkness*) di dalam sebuah gambar. Citra dengan kontras rendah dicirikan oleh sebagian besar komposisi citranya adalah terang atau sebagian besar gelap. Pada citra dengan kontras yang baik, komposisi gelap dan terang tersebar secara merata.

c. Operasi aritmetika

Brightness and contrast citra dapat dikendalikan oleh operasi aritmatika. Operasi aritmatika meliputi penambahan, pengurangan, perkalian, dan membagi nilai-nilai piksel dengan beberapa nilai konstan. Fungsi aritmatika pada nilai-nilai piksel RGB adalah:

- *Addition of a constant: Increases brightness*
- *Subtraction of a constant: Decreases brightness*
- *Multiplication by a constant: Increases contrast*
- *Division by a constant: Decreases contrast*

d. Operasi Boolean

Jenis operasi boolean atau *bitwise* telah dibahas pada BAB II.

e. Operasi Geometri

1) *Scaling*

Scaling adalah mengubah ukuran gambar lebih besar atau lebih kecil. Pada OpenCV dapat dilakukan dengan memanggil fungsi `cv.resize()`.

```
dst = cv.resize(src, dsize, fx, fy, interpolation_flag)
```

<code>src</code>	:	Citra input
<code>dst</code>	:	Citra keluaran
<code>dsize</code>	:	ukuran gambar output; jika sama dengan nol, itu dihitung sebagai: <code>dsize = Size(round(fx*src.cols), round(fy*src.rows))</code>
<code>fx</code>	:	faktor skala sumbu X; ketika sama dengan 0, itu

Dalam OpenCV Fungsi yang digunakan adalah `cv2.threshold`.

```
ret, thresh1 = cv2.threshold(img, thresholdVal, maxVal,
cv2.THRESH_BINARY)
```

Argumen pertama adalah gambar sumber dengan mode gambar adalah *grayscale*. Argumen kedua adalah nilai *threshold* yang digunakan untuk mengklasifikasikan nilai piksel. Argumen ketiga adalah `maxVal` mewakili nilai yang akan diberikan jika nilai piksel lebih dari (terkadang kurang dari) nilai *threshold*. Gambar hasil operasi thresholding akan disimpan dalam `thresh1`. OpenCV menyediakan gaya *thresholding* yang berbeda dan ditentukan oleh parameter keempat dari fungsi tersebut [3]. Jenis *threshold* dalam openCV antara lain:

Enumerator	
THRESH_BINARY Python: <code>cv.THRESH_BINARY</code>	$dst(x, y) = \begin{cases} \text{maxval} & \text{if } src(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$
THRESH_BINARY_INV Python: <code>cv.THRESH_BINARY_INV</code>	$dst(x, y) = \begin{cases} 0 & \text{if } src(x, y) > \text{thresh} \\ \text{maxval} & \text{otherwise} \end{cases}$
THRESH_TRUNC Python: <code>cv.THRESH_TRUNC</code>	$dst(x, y) = \begin{cases} \text{threshold} & \text{if } src(x, y) > \text{thresh} \\ src(x, y) & \text{otherwise} \end{cases}$
THRESH_TOZERO Python: <code>cv.THRESH_TOZERO</code>	$dst(x, y) = \begin{cases} src(x, y) & \text{if } src(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$
THRESH_TOZERO_INV Python: <code>cv.THRESH_TOZERO_INV</code>	$dst(x, y) = \begin{cases} 0 & \text{if } src(x, y) > \text{thresh} \\ src(x, y) & \text{otherwise} \end{cases}$
THRESH_MASK Python: <code>cv.THRESH_MASK</code>	
THRESH_OTSU Python: <code>cv.THRESH_OTSU</code>	flag. use Otsu algorithm to choose the optimal threshold value
THRESH_TRIANGLE Python: <code>cv.THRESH_TRIANGLE</code>	flag. use Triangle algorithm to choose the optimal threshold value

2. Buatlah program dengan menggunakan OpenCV-Python untuk transformasi citra digital. Citra yang digunakan adalah foto salah satu tokoh kemerdekaan yang lahir pada 24 Januari 1916 di Bodan Karangjati, Purbalingga, Jawa Tengah yaitu Jendral Sudirman "jenderalsoedirman.jpg".



Gambar 24. Foto Jendral Sudirman "jenderalsoedirman.jpg".

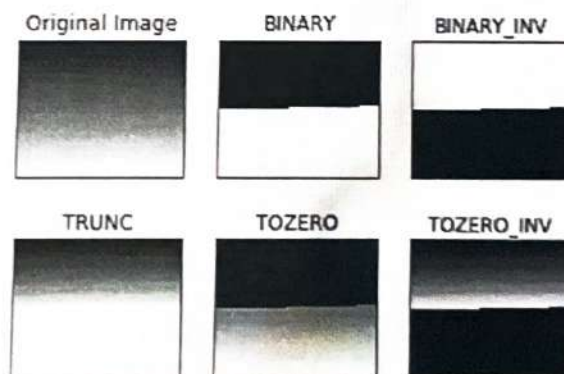
Lakukan operasi geometri *resize*, translasi, dan rotasi terhadap citra tersebut. Tampilkan dan simpan ketiga gambar hasil operasi geometri tersebut.

B. Latihan

1. Bacalah citra `gradient.jpg` dan lakukan *thresholding* terhadap citra tersebut menggunakan tipe: 1) `THRESH_BINARY`, 2) `THRESH_BINARY_INV`, 3) `THRESH_TRUNC`, 4) `THRESH_TOZERO`, 5) `THRESH_TOZERO_INV`. Lalu tampilkan enam citra tersebut (termasuk citra asal) menggunakan *library* `matplotlib`.

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt
img = cv.imread('gradient.jpg',0)
ret,thresh1 = cv.threshold(img,127,255,cv.THRESH_BINARY)
ret,thresh2 = cv.threshold(img,127,255,cv.THRESH_BINARY_INV)
ret,thresh3 = cv.threshold(img,127,255,cv.THRESH_TRUNC)
ret,thresh4 = cv.threshold(img,127,255,cv.THRESH_TOZERO)
ret,thresh5 = cv.threshold(img,127,255,cv.THRESH_TOZERO_INV)
titles = ['Original Image', 'BINARY', 'BINARY_INV', 'TRUNC', 'TOZERO', 'TOZERO_INV']
images = [img, thresh1, thresh2, thresh3, thresh4, thresh5]
for i in range(6):
    plt.subplot(2,3,i+1),plt.imshow(images[i],'gray')
    plt.title(titles[i])
    plt.xticks([],plt.yticks([]))
plt.show()
```

Keluaran:



Gambar 23. Citra Output Latihan No.1.

3) Rotation

Rotasi gambar untuk sudut θ dicapai oleh matriks transformasi bentuk:

$$M = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Tetapi OpenCV menyediakan rotasi berskala dengan pusat rotasi yang dapat disesuaikan sehingga Anda dapat memutar di lokasi mana pun yang Anda inginkan. Matriks transformasi yang dimodifikasi diberikan oleh:

$$\begin{bmatrix} \alpha & \beta & (1 - \alpha) \cdot \text{center.x} - \beta \cdot \text{center.y} \\ -\beta & \alpha & \beta \cdot \text{center.x} + (1 - \alpha) \cdot \text{center.y} \end{bmatrix}$$

Di mana:

$$\alpha = \text{scale} \cdot \cos \theta,$$

$$\beta = \text{scale} \cdot \sin \theta$$

```
M = cv.getRotationMatrix2D(center, angle, scale)
dst = cv.warpAffine(src, M, dsize)
```

- M : Matriks transformasi
- center : Pusat rotasi di gambar sumber
- angle : Sudut rotasi dalam derajat. Nilai positif berarti rotasi berlawanan arah jarum jam
- scale : Faktor skala

BAB V HISTOGRAM

Materi:

- Histogram Warna
- Perhitungan Histogram dan Plot Histogram
- *Contrast Stretching/Histogram Stretching*
- *Histogram Equalization*
- Histogram Warna

Tujuan Pembelajaran:

- Mahasiswa dapat menampilkan histogram untuk satu *channel grayscale* dan setiap *channel* warna.
- Mahasiswa dapat melakukan *image enhancement* menggunakan *contrast stretching* dan *histogram equalization*.
- Mahasiswa dapat menganalisis citra menggunakan histogram.

A. Penyajian

1. Histogram

Histogram merupakan representasi grafis dari distribusi data. Pada pengolahan citra, data yang ditampilkan pada *histogram* adalah persebaran frekuensi *pixel* yang memiliki nilai intensitas yang sama. Nilai intensitas pada histogram biasanya disebut “bin”. Untuk citra *grayscale*, ukuran bin di dalam histogram adalah 256 karena rentang intensitas berkisar 0 – 255. Contoh *histogram* dapat dilihat pada Gambar 25.

Referensi

- [1] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.
- [2] T. B. Moeslund, *Introduction to Video and Image Processing Building*, Ian Mackie. Springer, 2012.
- [3] A. Mordvintsev and A. K, "OpenCV-Python Tutorials," 2019, https://docs.opencv.org/4.1.2/d6/d00/tutorial_py_root.html.

```

import cv2 as cv
import numpy as np

img = cv.imread('jenderalsoedirman.jpg',0)
rows,cols = img.shape

img_resize = cv.resize(img,None,fx=2, fy=2, interpolation =
cv.INTER_CUBIC)

M = np.float32([[1,0,100],[0,1,50]])
img_translation = cv.warpAffine(img,M,(cols,rows))

M = cv.getRotationMatrix2D(((cols-1)/2.0,(rows-1)/2.0),45,1)
img_rotation = cv.warpAffine(img,M,(cols,rows))

cv.imwrite("jenderalsoedirman_resize.jpg", img_resize)
cv.imwrite("jenderalsoedirman_translation.jpg", img_translation)
cv.imwrite("jenderalsoedirman_rotation.jpg", img_rotation)

cv.imshow('image resize', img_resize)
cv.imshow('image translation', img_translation)
cv.imshow('image rotation', img_rotation)
cv.waitKey(0)
cv.destroyAllWindows()

```

C. Lembar Kerja Praktikum

1. Buatlah fungsi *image thresholding* (nama fungsi *my_threshold*) dengan ketentuan tidak menggunakan fungsi yang telah disediakan oleh OpenCV-Python. Dalam fungsi tersebut menerima salah satu parameter yaitu *type threshold*, dan *type threshold* yang dapat diterapkan dalam fungsi yang kalian buat antara lain:
 - Thresholding Binary (type = 1)
 - Thresholding Binary Inverted (type = 2)
 - Truncate (type = 3)
 - Truncate to Zero (type = 4)
 - Truncate to Zero Inverted (type = 5)

Perhitungan Histogram dalam Numpy:

```
img = cv.imread('home.jpg',0)
hist,bins = np.histogram(img.ravel(),256,[0,256])
```

b. Plotting Histogram

Untuk *plotting histogram* juga dapat dilakukan dengan berbagai cara. Matplotlib memiliki fungsi *plotting histogram* `matplotlib.pyplot.hist()` [1].

Contoh:

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('low.jpg',0)
plt.hist(img.ravel(),256,[0,256]); plt.show()
```

Pada openCV dapat hasil perhitungan histogram dengan menggunakan `cv.calcHist()` dapat langsung diplotkan histogramnya. Berikut ini adalah contoh kode:

```
import numpy as np
import cv2 as cv

img = cv.imread('low.jpg',0)
hist_full = cv.calcHist([img],[0],None,[256],[0,256])

plt.plot(hist_full)
plt.xlim([0,256])
plt.show()
```

2. Perhitungan Histogram dan Plot Histogram

a. Perhitungan *Histogram*

Untuk perhitungan histogram dapat menggunakan dua cara yaitu dengan openCV dan numpy. Dengan openCV dapat fungsi `cv.calcHist()` untuk menemukan histogram, untuk numpy menyediakan fungsi untuk menemukan histogram yaitu `np.histogram()` [1].

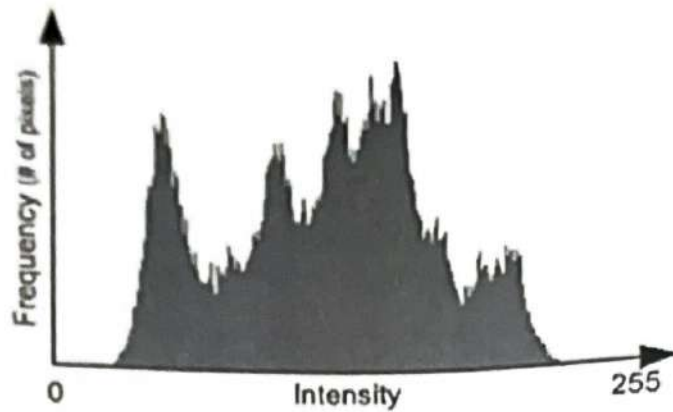
Perhitungan Histogram dalam OpenCV:

```
cv.calcHist(images, channels, mask, histSize, ranges)
```

- `images` : gambar sumber dengan tipe `uint8` atau `float32`, harus diberikan dalam tanda kurung siku, contoh “[img]”.
- `channels` : diberi dalam tanda kurung siku, parameter ini merupakan *channel* tempat menghitung histogram. Misalnya, jika input adalah gambar skala abu-abu, nilainya adalah [0]. Untuk gambar berwarna, dapat menggunakan [0], [1] atau [2] untuk menghitung histogram masing-masing *channel* biru, hijau atau merah.
- `mask` : mask image, untuk menemukan histogram dari gambar secara keseluruhan, isikan “none”.
- `histsize` : mewakili jumlah BIN, dan perlu diberikan dalam tanda kurung. Untuk skala penuh, gunakan [256].
- `ranges` : biasanya, [0,256].

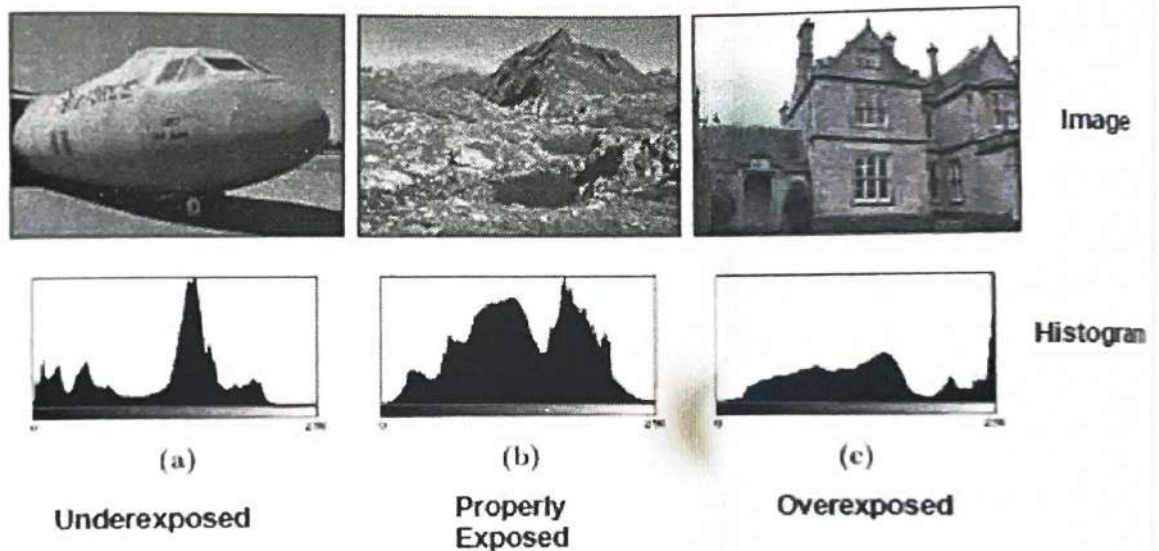
Contoh:

```
img = cv.imread('home.jpg',0)
hist = cv.calcHist([img],[0],None,[256],[0,256])
```



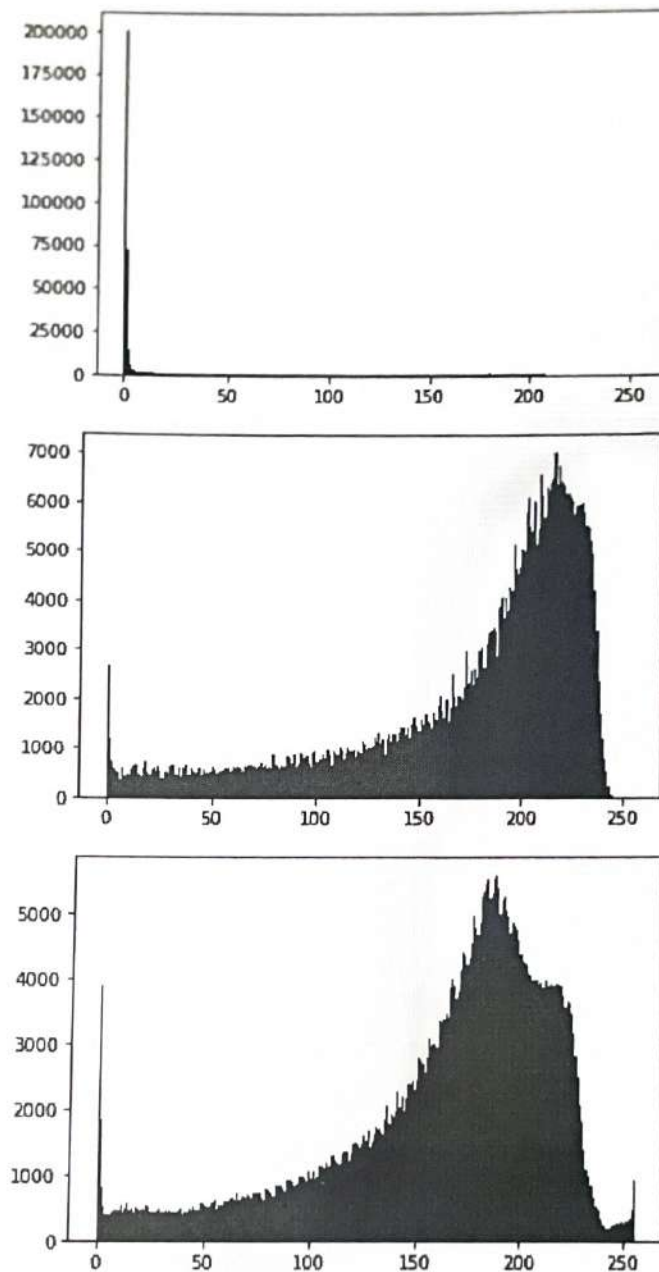
Gambar 25. Histogram Citra Grayscale 8-Bit.

istogram dapat digunakan untuk melakukan *image enhancement* atau menentukan nilai *threshold*. Histogram berguna untuk menganalisis *exposure*, *brightness* dan *kontras*, dan *dynamic range*. *Exposure* menunjukkan penyebaran nilai intensitas pixel di dalam suatu citra. Suatu citra dapat dikatakan ter-*expose* dengan baik ketika seluruh bin dari ujung kanan ke ujung kiri terisi tanpa ada *space* yang kosong. Sedangkan untuk *brightness*, kita dapat menggeser *histogram* untuk meningkatkan atau menurunkan *brightness*. Contoh *exposure* citra dapat dilihat pada Gambar 26.



Gambar 26. *Exposure* pada Citra

untuk tiap intensitas warna dalam citra. Untuk mendefinisikan *histogram*, warna di kuantisasi menjadi beberapa level diskret, kemudian untuk tiap level tersebut dihitung jumlah *pixel* yang nilainya sesuai. Contoh *histogram* warna *channel* RGB dapat dilihat pada Gambar 29.



Gambar 29. Histogram Warna pada *Channel* RGB

citra. Kontras rendah menggunakan sedikit bin, sedangkan kontras tinggi menggunakan banyak bin. Menambah atau mengurangi kontras dapat dilakukan dengan cara meregangkan atau memampatkan *histogram*.

Formula untuk *histogram stretching* yaitu dapat dilihat pada Persamaan berikut ini.

$$(x, y) = \frac{f(x, y) - f_{min}}{f_{max} - f_{min}} * 255$$

Keterangan:

$f(x, y)$: nilai *pixel* di titik (x, y)

f_{min} : nilai *pixel* terkecil pada citra

f_{max} : nilai *pixel* terbesar pada citra

4. Histogram Equalization

Histogram Equalization adalah suatu proses perataan *histogram*, di mana distribusi nilai derajat keabuan pada suatu citra dibuat rata. Sifat-sifat *histogram equalization* antara lain yaitu sebagai berikut:

- *Gray level* yang sering muncul lebih dijarangkan jaraknya dengan *gray level* sebelumnya.
- *Gray level* yang jarang muncul bisa lebih dirapatkan jaraknya dengan *gray level* sebelumnya.
- *Histogram* baru pasti mencapai nilai maksimal keabuan (contoh: 255).

Untuk dapat melakukan *histogram equalization* ini diperlukan suatu fungsi distribusi kumulatif yang merupakan kumulatif dari *histogram*. Adapun pada OpenCV proses *histogram equalization* dapat dilakukan dengan fungsi [1]:

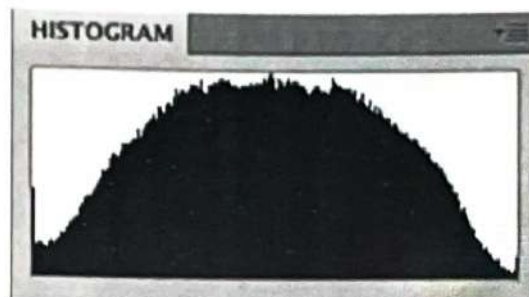
```
dst = cv.equalizeHist(src)
```

5. Histogram Warna

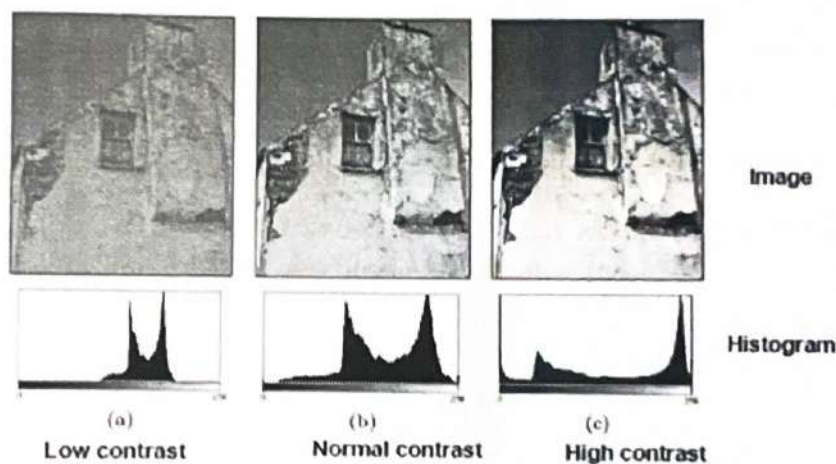
Komposisi warna pada citra dapat direpresentasikan dalam bentuk *histogram*. *Histogram warna* merepresentasikan distribusi jumlah piksel

3. Contrast Stretching/Histogram Stretching

Komputer dapat secara otomatis menilai apakah gambar terlalu gelap, terlalu terang atau memiliki kontras yang terlalu rendah, dan secara otomatis memperbaiki gambar menggunakan pemetaan tingkat keabuan, yaitu dengan menggunakan histogram gambar. Sebelum memproses gambar Kita harus selalu melihat histogram gambar, di mana *histogram* yang ideal yaitu nilai keabuan tersebar secara merata seperti yang dapat dilihat pada Gambar 27. Macam-macam kontras (*low contrast*, *normal contrast*, dan *high contrast*) pada citra dapat dilihat pada Gambar 28, terlihat bahwa citra yang *low contrast* yaitu bentuk histogram yang mengumpul di suatu tempat saja [2].



Gambar 27. Histogram Ideal



Gambar 28. Kontras pada Citra

Salah satu Teknik untuk meningkatkan kualitas citra dalam hal memperbaiki kontras yaitu dengan *contrast stretching*. Teknik ini adalah meningkatkan selisih nilai intensitas maksimal dan minimal pada sebuah

3. Lakukan *histogram equalization* terhadap gambar 'low.jpg' dengan menggunakan fungsi `cv.equalizeHist()`. Simpan kedua gambar (gambar sebelum dilakukan *histogram equalization* dan setelah dilakukan *histogram equalization*) dalam 1 file/gambar.

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('low.jpg',0)
equ = cv.equalizeHist(img)
res = np.hstack((img,equ)) #stacking images side-by-side

cv.imwrite('res.jpg',res)
```

2. Baca gambar 'low.jpg', lalu buatlah program untuk melakukan *histogram stretching* terhadap gambar tersebut. Tampilkan histogram awal sebelum dilakukan *histogram stretching* dan setelah dilakukan *histogram stretching* dengan matplotlib.

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('low.jpg',0)
hist,bins = np.histogram(img.flatten(),256,[0,256])
cdf = hist.cumsum()

cdf_m = np.ma.masked_equal(cdf,0)
cdf_m = (cdf_m-cdf_m.min())*255/(cdf_m.max()-cdf_m.min())
cdf = np.ma.filled(cdf_m,0).astype('uint8')
img2 = cdf[img]
cdf_normalized = cdf * float(hist.max())/cdf.max()

plt.subplot(2, 1, 1)
plt.plot(cdf, color = 'b')
plt.hist(img.flatten(),256,[0,256], color = 'r')
plt.xlim([0,256])
plt.legend(('cdf','histogram'), loc = 'upper left')
plt.show()

plt.subplot(2, 1, 2)
plt.plot(cdf_normalized, color = 'b')
plt.hist(img2.flatten(),256,[0,256], color = 'r')
plt.xlim([0,256])
plt.legend(('cdf','histogram'), loc = 'upper left')
plt.show()

cv.imshow('img1',img)
cv.imshow('img2',img2)
cv.waitKey(0)
cv.destroyAllWindows()
```

B. Latihan

1. Baca gambar berwarna, lalu buatlah tiga histogram dari setiap *channel* gambar tersebut. Tampilkan 3 hitogram dari setiap *channel* tersebut.

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('red_apple.jpg',cv.IMREAD_COLOR)

plt.subplot(1, 3, 1),plt.hist(img[:, :,0].ravel(),256,[0,256], color = 'b');
plt.subplot(1, 3, 2),plt.hist(img[:, :,1].ravel(),256,[0,256], color = 'g');
plt.subplot(1, 3, 3),plt.hist(img[:, :,2].ravel(),256,[0,256], color = 'r');
plt.show()
```

Berikut pada Gambar 32, contoh sebuah mask berukuran 3x3. Filter ini akan diterapkan/dikonvolusikan pada setiap jendela ketetanggaan 3x3 pada citra (Gambar 33).

W_{1C}	W_2	W_3
W_4	W_5	W_6
W_7	W_8	W_9

			G_{14}	G_{15}
	G_{22}		G_{24}	G_{25}
			G_{34}	G_{35}
G_{41}	G_{42}	G_{43}	G_{44}	G_{45}
G_{51}	G_{52}	G_{53}	G_{54}	G_{55}

Gambar 32. Mask Berukuran 3x3.

Gambar 33. Contoh Piksel Citra.

Dengan menggunakan rumus berikut, akan diperoleh nilai G_{22} yang baru hasil konvolusi.

$$G_{22}' = w_1 G_{11} + w_2 G_{12} + w_3 G_{13} + w_4 G_{21} + w_5 G_{22} + w_6 G_{23} + w_7 G_{31} + w_8 G_{32} + w_9 G_{33}$$

OpenCV menyediakan fungsi `cv.filter2D()` untuk melakukan konvolusi sebuah pada sebuah gambar [1].

`dst = cv.filter2D(src, ddepth, kernel, anchor, delta, borderType)`

Keterangan:

- src : citra input
- dst : citra keluaran
- ddepth : kedalaman yang diinginkan dari citra keluaran jika nilai diisi-1, maka citra output akan memiliki kedalaman yang sama dengan inputnya.
- kernel : kernel konvolusi, sebuah matriks *floating point* dengan *channel* tunggal.

BAB VI

IMAGE ENHANCEMENT DENGAN MASK PROCESSING

Materi

- *Mask Processing*/Konvolusi
- Filter Spasial
- Jenis Filter Spasial

Tujuan Pembelajaran

- Mahasiswa dapat melakukan *enhancement* pada citra dengan *mask processing* menggunakan konvolusi matriks dan filter spasial.

A. Penyajian

1. Mask Processing

Mask processing adalah operasi yang melibatkan pixel-pixel tetangganya dengan menggunakan jendela ketetanggaan. Operasi yang dilakukan adalah dengan mengoperasikan suatu *Mask* terhadap jendela tersebut $\oplus$ konvolusi. *Mask* sering juga disebut *filter*.

Gambar 31 merupakan contoh jendela ketetanggaan 3x3. Nilai pixel pada posisi X dipengaruhi oleh 8 nilai pixel tetangganya dan piksel itu sendiri, berbeda pada point processing yang tidak dipengaruhi oleh pixel tetangganya.

1	2	3
8	X	4
7	6	5

Gambar 31. Contoh Jendela Ketetanggaan

C. Lembar Kerja Praktikum

1. Buatlah penjelasan langkah demi langkah dari program yang telah dibuat pada latihan 1, 2, dan 3. Penjelasan dituliskan dalam file dokumen berformat pdf.
2. Gambar 32 merupakan foto Sutan Syahrir dengan kontras yang rendah, kita sebagai sivitas akademika UPN Veteran Jakarta yang cinta Tanah Air, dengan pengetahuannya di bidang informatika ingin meningkatkan kualitas foto Perdana Menteri pertama Republik Indonesia, Sutan Syahrir. Lakukan *histogram equalization* terhadap gambar 'SutanSjahrirLowContrast.jpg' dengan tanpa menggunakan fungsi pada openCV-Python (`cv.equalizeHist()`), dibuat dengan fungsi yang kalian buat sendiri dengan nama fungsi `myEqualizeHist`.



Gambar 30. SutanSjahrirLowContrast.jpg.

Referensi

- [1] A. Mordvintsev and A. K., "OpenCV-Python Tutorials," 2019. https://docs.opencv.org/4.1.2/d6/d00/tutorial_py_root.html.
- [2] T. B. Moeslund, *Introduction to Video and Image Processing Building*, Ian Mackie. Springer, 2012.

LPF biasa digunakan untuk membuang titik yang berbeda dengan titik-titik tetangganya (contohnya: penghilangan *noise* dan penghalusan citra).

- Penghilangan *noise* dengan LPF

Piksel yang mempunyai *noise* pada umumnya memiliki frekuensi yang tinggi. *Low Pass Filter* (LPF) mengambil komponen citra yang berfrekuensi rendah dan menahan komponen berfrekuensi tinggi.

- Penghalusan citra/bluring dengan LPF

LPF digunakan untuk melakukan proses efek *blur* dan reduksi *noise*.

b. High Pass Filter

High Pass Filter (HPF) akan memperkuat komponen citra yang berfrekuensi tinggi dan menahan komponen berfrekuensi rendah. HPF biasa digunakan untuk mempertahankan titik yang berbeda dengan titik-titik. Hasilnya HPF adalah berupa *edge* atau tepi citra terlihat lebih tajam. HPF digunakan untuk melakukan proses deteksi tepi. Ciri-ciri kernel dari HPF adalah nilai-nilainya terdiri positif, nol dan negatif, dan jumlah dari semua nilainya sama dengan nol.

$$\sum_{xy} H(x, y) = 0$$

Salah satu *high pass filter* adalah menggunakan **filter sobel**, berikut ini adalah filter sobel:

Urutan turunan (*derivative*) x:
$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Urutan turunan (*derivative*) y:
$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

OpenCV menyediakan fungsi `cv.sobel()` untuk melakukan konvolusi sebuah pada sebuah gambar [1].

3. Jenis Filter Spasial

Gambar dapat berada pada kondisi frekuensi rendah atau pada frekuensi tinggi. Kondisi gambar pada frekuensi rendah jika titik pada gambar mempunyai banyak kemiripan warna dengan titik-titik tetangganya, yaitu seperti gambar yang sering kita lihat pada umumnya. Bila suatu gambar menunjukkan frekuensi tinggi maka pada gambar tersebut banyak titik yang nilai grayscale-nya (warna) yang berbeda jauh dengan titik-titik tetangganya. Gambar 34 memperlihatkan perbandingan antara gambar yang memiliki frekuensi rendah dan frekuensi tinggi.



Gambar 34. Perbandingan Gambar Frekuensi Rendah dan Frekuensi Tinggi

Ada dua jenis filter spasial, yaitu *Low-Pass Filter* (LPF) dan *High-Pass Filter* (HPF). Sedangkan

a. *Low Pass Filter*

Low-Pass Filter (LPF) adalah suatu bentuk filter yang mengambil data pada frekuensi rendah dan membuang data pada frekuensi tinggi. Efek yang dihasilkan oleh *filter lowpass* adalah untuk menghaluskan gambar dengan memburamkannya [2]. Ciri-ciri kernel dari LPF adalah semua nilainya positif dan jumlah dari semua nilainya sama dengan satu.

$$H(x, y) \geq 0 \text{ dan } \sum_{xy} H(x, y) = 1$$

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad K = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Gambar 35. Kernel dari LPF

- anchor** : anchor kernel yang menunjukkan posisi relatif dari titik filter dalam kernel, anchor harus terletak di dalam kernel, nilai default (-1,-1) berarti jangkar berada di pusat kernel.
- delta** : nilai opsional ditambahkan ke piksel yang difilter sebelum menyimpannya di citra dst.
- borderType** : Tipe border, kombinasi tipe border dapat dilihat pada link: https://docs.opencv.org/4.1.2/d2/de8/group__core__array.html

2. Filter Spasial

Proses memanipulasi kumpulan piksel dari sebuah citra dengan melihat piksel ketetanggannya untuk menghasilkan citra baru. Salah satu tujuan dari filter spasial adalah untuk perbaikan kualitas citra (*image enhancement*). Perbaikan kualitas citra dengan filter spasial dapat dilakukan dengan:

a. Penghilangan derau (*noise*)

Noise adalah gambar atau piksel yang mengganggu kualitas citra yang dapat disebabkan oleh gangguan optik pada alat akuisisi maupun secara disengaja akibat proses pengolahan yang tidak sesuai. Ada empat macam *noise* yang sering kita temui, antara lain:

- *Salt and pepper noise*
- *Gaussian noise*
- *Speckle noise*
- *Periodic noise*

b. Penghalusan citra (*smoothing*)

Penghalusan citra (*image smoothing*) merupakan salah satu operasi perbaikan kualitas citra. Tujuan dari operasi ini adalah untuk menekan gangguan (*noise*) pada citra.

c. Penajaman citra/deteksi tepi (*sharpening*)

Deteksi tepi berfungsi untuk mengidentifikasi garis batas (*boundary*) dari suatu objek yang terdapat pada citra. Untuk proses segmentasi, diperlukan mendapatkan tepi dari objek yang ingin disegmentasi.

Keterangan:

- `kernel_3_3 = np.ones((3,3),np.float32)/9`, yaitu membuat matriks ukuran 3x3 yang berisikan nilai 1/9 di semua elemen matriksnya. Digunakan untuk kernel filter rata-rata.
 - `kernel_5_5 = np.ones((5,5),np.float32)/25`, yaitu membuat matriks ukuran 5x5 yang berisikan nilai 1/25 di semua elemen matriksnya. Digunakan untuk kernel filter rata-rata.
2. Buatlah program untuk percobaan dalam menghilangkan *salt and pepper noise* pada gambar Monas, Monumen Nasional (Monumen yang terletak di D.K.I. Jakarta, didirikan untuk mengenang perlawanan dan perjuangan rakyat Indonesia untuk merebut kemerdekaan dari pemerintahan kolonial Hindia Belanda). Program diawali dengan membaca gambar 'monas.jpg', lalu tambahkan noise jenis *salt and pepper*. Terapkan *smoothing image* dengan menggunakan fungsi `cv.blur`, `cv.GaussianBlur`, `cv.medianBlur` dengan jendela ketetanggan 3x3. Lalu tampilkan kelima gambar (gambar asli, gambar yang telah diberi *noise salt and pepper*, dan tiga gambar hasil filter dengan fungsi `cv.blur`, `cv.GaussianBlur`, `cv.medianBlur`) dengan menggunakan `cv.imshow` dan subplot `matplotlib`.



Gambar 36. Monas.jpg.

B. Latihan

1. Menerapkan *smoothing image* menggunakan *average filter* (filter rata-rata) menggunakan fungsi `cv.filter2D`. Gunakan 2 jendela ketetanggaan yang berbeda yaitu 3x3 dan 5x5. Lalu tampilkan ketiga gambar (gambar asli, gambar hasil filter dengan jendela ketetanggaan 3x3, dan gambar hasil filter dengan jendela ketetanggaan 5x5) dengan menggunakan `cv.imshow` dan subplot `matplotlib`.

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('upn.jpg')

kernel_3_3 = np.ones((3,3),np.float32)/9
kernel_5_5 = np.ones((5,5),np.float32)/25

#dst = cv.filter2D(img,cv.CV_8U,kernel_5_5)
dst_3_3 = cv.filter2D(img,cv.CV_8U,kernel_3_3,(-1,-1), delta = 0,
borderType = cv.BORDER_DEFAULT)
dst_5_5 = cv.filter2D(img,cv.CV_8U,kernel_5_5,(-1,-1), delta = 0,
borderType = cv.BORDER_DEFAULT)

plt.subplot(1, 3, 1),plt.imshow(img),plt.title('Original')
plt.xticks([], plt.yticks([]))
plt.subplot(1, 3, 2),plt.imshow(dst_3_3),plt.title('Averaging 3x3')
plt.xticks([], plt.yticks([]))
plt.subplot(1, 3, 3),plt.imshow(dst_5_5),plt.title('Averaging 5x5')
plt.xticks([], plt.yticks([]))
plt.show()

cv.imshow('Original', img)
cv.imshow('Averaging Kernel 3x3', dst_3_3)
cv.imshow('Averaging Kernel 5x5', dst_5_5)
cv.waitKey(0)
cv.destroyAllWindows()
```

```
dst = cv.Sobel(src, ddepth, dx, dy, ksize)
```

Keterangan:

dst : citra output
src : citra input
ddepth : kedalaman yang diinginkan dari citra keluaran
dx : urutan turunan (*derivative*) x
dy : urutan turunan (*derivative*) y
ksize : ukuran kernel Sobel. Ukuran yang bisa dipilih antara lain 1, 3, 5, atau 7.

```

cv.imshow('Original', img)
cv.imshow('Sobel X', sobelx_8u)
cv.imshow('Sobel Y', sobely_8u)
cv.imshow('Sobel Magn', sobel_8u)

cv.waitKey(0)
cv.destroyAllWindows()

```

Keterangan:

```

magnitude = cv.magnitude(sobelx_f, sobely_f)
abs_sobel64f = np.absolute(magnitude)
sobel_8u = np.uint8(abs_sobel64f)

```

Digunakan untuk mendapatkan nilai magnitude dari sobel *derivative* x dan sobel *derivative* y. Rumusan nilai magnitude yaitu sebagai berikut [2].

$$M(x, y) = \|\nabla f\| = \text{mag}(\nabla f) = \sqrt{g_x^2 + g_y^2}$$

C. Lembar Kerja Praktikum

1. Buatlah program dengan OpenCV-Python menerapkan *high pass filter*. Gunakan foto atau gambar dari Ki Hadjar Dewantara "Ki_Hadjar_Dewantara.jpg". Terapkan *high pass filter* menggunakan sobel, lakukan untuk *derivative* x, *derivative* y, dan *magnitude*-nya. Fungsi sobel dibuat sendiri tanpa menggunakan *library* OpenCV, ketentuan fungsi sobel adalah:
 - Nama fungsi mySobel(),
 - Fungsi menerima parameter citra input,
 - Proses fungsi mySobel terdiri dari perhitungan *derivative* x, *derivative* y, dan *magnitude*,
 - Fungsi mengembalikan 3 (tiga) nilai berupa citra hasil operasi sobel (citra hasil perhitungan *derivative* x, *derivative* y, dan *magnitude*).

- `noise_img = np.array(255*noise_img, dtype = 'uint8')`, yaitu untuk mengembalikan gambar *floating-point* pada rentang [0, 1], jadi perlu mengubahnya ke 'uint8' dan rentang nilai [0,255].
3. Baca gambar 'monas.jpg', lalu terapkan high pass filter menggunakan sobel, lakukan untuk *derivative x*, *derivative y*, dan *magnitude*-nya. Lalu tampilkan keempat gambar (gambar asli, gambar hasil filter sobel *derivative x*, gambar hasil filter sobel *derivative y*, dan gambar hasil *magnitude*) dengan menggunakan `cv.imshow` dan `subplot matplotlib`.

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('monas.jpg',0)

sobelx_f = cv.Sobel(img,cv.CV_64F,1,0,ksize=3)
sobely_f = cv.Sobel(img,cv.CV_64F,0,1,ksize=3)
sobelx_8u = cv.Sobel(img,cv.CV_8U,1,0,ksize=3)
sobely_8u = cv.Sobel(img,cv.CV_8U,0,1,ksize=3)

magnitude = cv.magnitude(sobelx_f,sobely_f)
abs_sobel64f = np.absolute(magnitude)
sobel_8u = np.uint8(abs_sobel64f)

plt.subplot(2,2,1),plt.imshow(img,cmap = 'gray')
plt.title('Original'), plt.xticks([], plt.yticks([]))
plt.subplot(2,2,2),plt.imshow(sobelx_8u,cmap = 'gray')
plt.title('Sobel X'), plt.xticks([], plt.yticks([]))
plt.subplot(2,2,3),plt.imshow(sobely_8u,cmap = 'gray')
plt.title('Sobel Y'), plt.xticks([], plt.yticks([]))
plt.subplot(2,2,4),plt.imshow(sobel_8u,cmap = 'gray')
plt.title('Sobel Y'), plt.xticks([], plt.yticks([]))

plt.show()
```

```

import cv2 as cv
from skimage.util import random_noise
from matplotlib import pyplot as plt

img = cv.imread('monas.jpg')
noise_img = random_noise(img, mode='s&p', amount=0.1)

noise_img = np.array(255*noise_img, dtype = 'uint8')

blur_average = cv.blur(noise_img,(3,3))
blur_gaussian = cv.GaussianBlur(noise_img,(3,3),0)
blur_median = cv.medianBlur(noise_img,3)

plt.subplot(3, 3, 1),plt.imshow(img),plt.title('Original')
plt.xticks([], plt.yticks([]))
plt.subplot(3, 3, 2),plt.imshow(noise_img),plt.title('Original with
Noise')
plt.xticks([], plt.yticks([]))
plt.subplot(3, 3, 3),plt.imshow(blur_average),plt.title('Average Filter')
plt.xticks([], plt.yticks([]))
plt.subplot(3, 3, 4),plt.imshow(blur_gaussian),plt.title('Gaussian
Blurred')
plt.xticks([], plt.yticks([]))
plt.subplot(3, 3, 5),plt.imshow(blur_median),plt.title('Median Filter')
plt.xticks([], plt.yticks([]))
plt.show()

cv.imshow('Original', img)
cv.imshow('Original with Noise', noise_img)
cv.imshow('Smoothing Average Filter', blur_average)
cv.imshow('Smoothing Gaussian', blur_gaussian)
cv.imshow('Smoothing Median Filter', blur_median)
cv.waitKey(0)
cv.destroyAllWindows()

```

Keterangan:

- `noise_img = random_noise(img, mode='s&p', amount=0.1)`, yaitu menerapkan noise berupa *salt and pepper noise* pada citra.

Degradasi citra dapat dimodelkan seperti berikut:

$$g(x, y) = h(x, y) * f(x, y) + n(x, y)$$

Keterangan:

$g(x, y)$	= Gambar yang terdegradasi
$h(x, y)$	= representasi spasial dari fungsi degradasi
$*$	= <i>convolution, spatial filtering</i>
$f(x, y)$	= input gambar
$n(x, y)$	= <i>additive noise term</i>

Pada model tersebut, dilakukan *blur filtering* $h(x, y)$ pada citra asli $f(x, y)$. Kemudian terdapat fungsi *random error* $n(x, y)$ atau kita kenal dengan *noise*. Maka, untuk menghasilkan kembali citra asli kita dapat melakukannya seperti berikut,

$$f(x, y) = g(x, y) - n(x, y) / h(x, y)$$

Contoh model degradasi di atas dapat direstorasi pada domain spasial, tetapi ada juga *noise* yang hanya bisa direstorasi di domain frekuensi. Biasanya restorasi citra dilakukan karena kita sudah mengetahui dan menduga model *noise* yang terdapat pada citra, sehingga penanganannya akan berbeda-beda tergantung model *noise*.

2. Noise

Sumber utama *noise* dalam gambar digital muncul selama akuisisi dan/atau transmisi gambar. Kinerja sensor pencitraan dipengaruhi oleh berbagai faktor lingkungan selama akuisisi gambar. Misalnya, dalam memperoleh gambar dengan kamera, level cahaya dan suhu sensor merupakan faktor utama yang mempengaruhi jumlah *noise* pada gambar yang dihasilkan. Gambar rusak selama transmisi terutama karena gangguan pada saluran transmisi. Misalnya, gambar yang dikirim menggunakan jaringan nirkabel mungkin rusak oleh petir atau gangguan atmosfer lainnya [1].

Noise merupakan degradasi pada sinyal citra yang terjadi akibat gangguan eksternal. *Error* yang terlihat akan bergantung pada jenis

BAB VII RESTORASI CITRA

Materi:

- Restorasi Citra
- *Noise*

Tujuan Pembelajaran:

- Mahasiswa dapat melakukan mengembalikan citra yang terkena *noise* ke citra semula

A. Penyajian

1. Restorasi Citra

Seperti dalam *image enhancement*, tujuan utama dari teknik restorasi adalah untuk meningkatkan citra dalam arti yang telah ditentukan sebelumnya. Perbedaan dengan *image enhancement* yaitu untuk *image enhancement* lebih subjektif, sementara restorasi citra merupakan proses yang objektif. Restorasi citra merupakan proses untuk mengurangi atau menghilangkan degradasi pada citra. Degradasi yang muncul dapat berupa *noise* atau efek optik atau distorsi saat melakukan akuisisi citra, seperti tidak fokusnya kamera yang menghasilkan citra *blur* [1]. Adapun perbedaan *image enhancement* dan *image restoration* dapat dilihat pada tabel berikut ini.

<i>Image Enhancement</i>	<i>Image Restoration</i>
Bersifat subjektif	Bersifat objektif
Digunakan untuk memanipulasi citra	Merekonstruksi citra untuk memperbaiki kualitas citra
Memperbaiki tampilan citra untuk tujuan tertentu	Memperbaiki citra yang sudah terkena <i>noise</i>

Lalu tampilkan keempat gambar (gambar asli, gambar hasil filter sobel *derivative* x, gambar hasil filter sobel *derivative* y, dan gambar hasil *magnitude*) dengan menggunakan `cv.imshow` dan `subplot` `matplotlib`.



Gambar 37. Ki Hadjar Dewantara, Bapak Pendidikan Nasional

Referensi

- [1] A. Mordvintsev and A. K, "OpenCV-Python Tutorials," 2019.
https://docs.opencv.org/4.1.2/d6/d00/tutorial_py_root.html.
- [2] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.

BAB VII RESTORASI CITRA

Materi:

- Restorasi Citra
- *Noise*

Tujuan Pembelajaran:

- Mahasiswa dapat melakukan mengembalikan citra yang terkena *noise* ke citra semula

A. Penyajian

1. Restorasi Citra

Seperti dalam *image enhancement*, tujuan utama dari teknik restorasi adalah untuk meningkatkan citra dalam arti yang telah ditentukan sebelumnya. Perbedaan dengan *image enhancement* yaitu untuk *image enhancement* lebih subjektif, sementara restorasi citra merupakan proses yang objektif. Restorasi citra merupakan proses untuk mengurangi atau menghilangkan degradasi pada citra. Degradasi yang muncul dapat berupa *noise* atau efek optik atau distorsi saat melakukan akuisisi citra, seperti tidak fokusnya kamera yang menghasilkan citra *blur* [1]. Adapun perbedaan *image enhancement* dan *image restoration* dapat dilihat pada tabel berikut ini.

<i>Image Enhancement</i>	<i>Image Restoration</i>
Bersifat subjektif	Bersifat objektif
Digunakan untuk memanipulasi citra	Merekonstruksi citra untuk memperbaiki kualitas citra
Memperbaiki tampilan citra untuk tujuan tertentu	Memperbaiki citra yang sudah terkena <i>noise</i>

Degradasi citra dapat dimodelkan seperti berikut:

$$g(x, y) = h(x, y) * f(x, y) + n(x, y)$$

Keterangan:

$g(x, y)$	= Gambar yang terdegradasi
$h(x, y)$	= representasi spasial dari fungsi degradasi
$*$	= <i>convolution, spatial filtering</i>
$f(x, y)$	= input gambar
$n(x, y)$	= <i>additive noise term</i>

Pada model tersebut, dilakukan *blur filtering* $h(x, y)$ pada citra asli $f(x, y)$. Kemudian terdapat fungsi *random error* $n(x, y)$ atau kita kenal dengan *noise*. Maka, untuk menghasilkan kembali citra asli kita dapat melakukannya seperti berikut,

$$f(x, y) = g(x, y) - n(x, y) / h(x, y)$$

Contoh model degradasi di atas dapat direstorasi pada domain spasial, tetapi ada juga *noise* yang hanya bisa direstorasi di domain frekuensi. Biasanya restorasi citra dilakukan karena kita sudah mengetahui dan menduga model *noise* yang terdapat pada citra, sehingga penanganannya akan berbeda-beda tergantung model *noise*.

2. Noise

Sumber utama *noise* dalam gambar digital muncul selama akuisisi dan/atau transmisi gambar. Kinerja sensor pencitraan dipengaruhi oleh berbagai faktor lingkungan selama akuisisi gambar. Misalnya, dalam memperoleh gambar dengan kamera, level cahaya dan suhu sensor merupakan faktor utama yang mempengaruhi jumlah *noise* pada gambar yang dihasilkan. Gambar rusak selama transmisi terutama karena gangguan pada saluran transmisi. Misalnya, gambar yang dikirim menggunakan jaringan nirkabel mungkin rusak oleh petir atau gangguan atmosfer lainnya [1].

Noise merupakan degradasi pada sinyal citra yang terjadi akibat gangguan eksternal. *Error* yang terlihat akan bergantung pada jenis

gangguan yang terjadi. Sumber dari *noise* pada citra dapat dilihat sebagai berikut:

- *Error* muncul ketika suatu citra dikirim secara elektronik dari suatu tempat ke tempat lain melalui satelit, kabel, atau nirkabel
- Sensor panas ketika mengambil gambar
- Nilai ISO yang tinggi pada kamera yang menyebabkan sensor kamera lebih cepat menangkap cahaya
- Ukuran sensor pada kamera.

Noise merupakan *random error* yang dapat dimodelkan sebagai fungsi aditif seperti yang dimodelkan pada model degradasi di atas. Beberapa tipe *noise* yang diperkenalkan adalah *salt and pepper noise*, *gaussian noise*, *speckle noise*, dan *periodic noise*.

2.1. Salt and Pepper Noise

Secara visual, *noise salt & pepper* terdiri dari piksel hitam dan putih yang tersebar pada seluruh citra. Sering disebut *noise impulse*, karena disebabkan oleh gangguan yang tiba-tiba pada sinyal citra. *Noise* ini dapat diatasi dengan *filter mean*, dan *filter median*. Citra *salt and pepper noise* dapat dilihat pada Gambar 38.



Gambar 38. Citra Salt and Pepper Noise

2.2. Gaussian Noise

Penyebab adanya *gaussian noise* adalah fluktuasi acak pada sinyal yang ditambahkan pada citra. *Gaussian noise* merupakan *white noise* ideal yang disebabkan oleh fluktuasi acak dalam sinyal karena penyebarannya terdistribusi normal. *Noise* ini dapat dikurangi dengan *filter* spasial seperti *median filtering*, *gaussian smoothing*, dan *mean filtering*. Jika citra dinotasikan dengan I dan *gaussian noise* dengan N , maka model degradasinya yaitu $(I + N)$. Citra *gaussian noise* dapat dilihat pada Gambar 39.



Gambar 39. Citra *Gaussian Noise*

2.3. Speckle Noise

Meskipun secara visual *gaussian* dan *speckle noise* terlihat mirip, tetapi keduanya memiliki pemodelan yang berbeda. *Speckle noise* dapat dimodelkan dengan nilai acak yang dikalikan dengan nilai pixel pada citra. Model degradasinya dapat dituliskan dengan persamaan $I(1+N)$. Penanganan untuk *speckle noise* dengan menggunakan *filter* spasial. Citra *speckle noise* dapat dilihat pada Gambar 40.



Gambar 40. Citra *Speckle Noise*

2.4. Periodic Noise

Berbeda dengan *noise* lain yang acak, *periodic noise* merupakan *noise* yang memiliki pola. Citra yang terdegradasi oleh *noise* ini akan tampak terdapat garis-garis pada citra. Untuk menghilangkan *noise* ini, dibutuhkan *filtering* pada domain frekuensi seperti *band reject filtering* dan *notch filter*. Citra *periodic noise* dapat dilihat pada Gambar 41.



Gambar 41. Citra *Periodic Noise*

B. Latihan

Latihan 1: Domain spasial

Berikut ini merupakan latihan membuat program yang dapat menambahkan dan merestorasi *noise* di domain spasial berupa *salt and pepper noise*, *gaussian noise*, dan *speckle noise*.

1. Membuat *Noise* pada Citra

```
import cv2 as cv
import numpy as np
from skimage.util import random_noise

# Load the image
img = cv.imread("upn.jpg")

# Add salt-and-pepper noise to the image.
noise_img_snp = random_noise(img, mode='s&p', amount=0.2)

# Add gaussian noise to the image.
noise_img_gaussian = random_noise(img, mode='gaussian', mean=0,
var=0.01)

# Add speckle noise to the image.
noise_img_speckle = random_noise(img, mode='speckle')

# The above function returns a floating-point image on the range [0, 1], thus
we changed it to 'uint8' and from [0,255]
noise_img_snp = np.array(255*noise_img_snp, dtype = 'uint8')
noise_img_gaussian = np.array(255*noise_img_gaussian, dtype = 'uint8')
noise_img_speckle = np.array(255*noise_img_speckle, dtype = 'uint8')

# Display the noise image
cv.imshow('img snp',noise_img_snp)
cv.imshow('img gaussian',noise_img_gaussian)
cv.imshow('img speckle',noise_img_speckle)
cv.waitKey(0)
cv.destroyAllWindows()
```

2. Merestorasi Citra yang Telah Diberi Noise

```
import cv2 as cv
import numpy as np
from skimage.util import random_noise

# Load the image
img = cv.imread("monas.jpg")

# Add salt-and-pepper noise to the image.
noise_img_snp = random_noise(img, mode='s&p', amount=0.05)

# Add gaussian noise to the image.
noise_img_gaussian = random_noise(img, mode='gaussian', mean=0,
var=0.01)

# Add speckle noise to the image.
noise_img_speckle = random_noise(img, mode='speckle')

# The above function returns a floating-point image on the range [0, 1],
thus we changed it to 'uint8' and from [0,255]
noise_img_snp = np.array(255*noise_img_snp, dtype = 'uint8')
noise_img_gaussian = np.array(255*noise_img_gaussian, dtype =
'uint8')
noise_img_speckle = np.array(255*noise_img_speckle, dtype = 'uint8')

# image reduction
kernel_3_3 = np.ones((3,3),np.float32)/9

img_snp_average_filter =
cv.filter2D(noise_img_snp,cv.CV_8U,kernel_3_3,(-1,-1), delta
= 0, borderType = cv.BORDER_DEFAULT)
img_snp_median_median = cv.medianBlur(noise_img_snp,3)

img_gaussian_average_filter =
cv.filter2D(noise_img_gaussian,cv.CV_8U,kernel_3_3,(-1,-1),
delta = 0, borderType = cv.BORDER_DEFAULT)
img_gaussian_median_median = cv.medianBlur(noise_img_gaussian,3)
```

```

img_speckle_average_filter =
cv.filter2D(noise_img_speckle,cv.CV_8U,kernel_3_3,(-1,-1),
delta = 0, borderType = cv.BORDER_DEFAULT)
img_speckle_median_median = cv.medianBlur(noise_img_speckle,3)

# Display the noise image
cv.imshow('img snp',noise_img_snp)
cv.imshow('img gaussian',noise_img_gaussian)
cv.imshow('img speckle',noise_img_speckle)

# Display the image reduction
cv.imshow('img s&p reduction with average filter',
img_snp_average_filter)
cv.imshow('img s&p reduction with median filter',
img_snp_median_median)

cv.imshow('img gaussian reduction with average filter',
img_gaussian_average_filter)
cv.imshow('img gaussian reduction with median filter',
img_gaussian_median_median)

cv.imshow('img speckle reduction with average filter',
img_speckle_average_filter)
cv.imshow('img speckle reduction with median filter',
img_speckle_median_median)

cv.waitKey(0)
cv.destroyAllWindows()

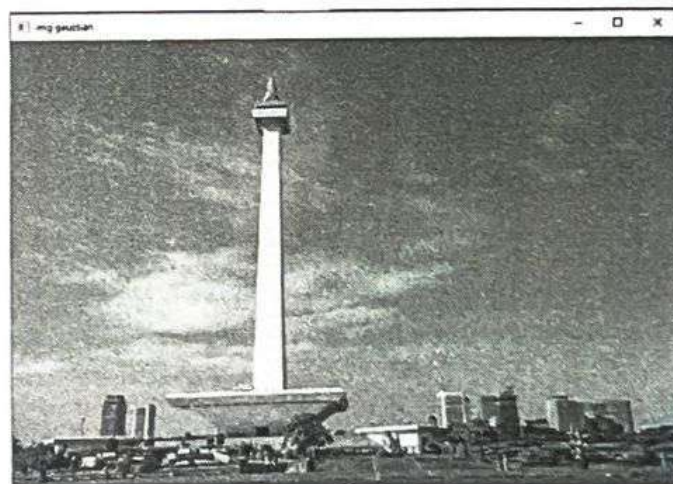
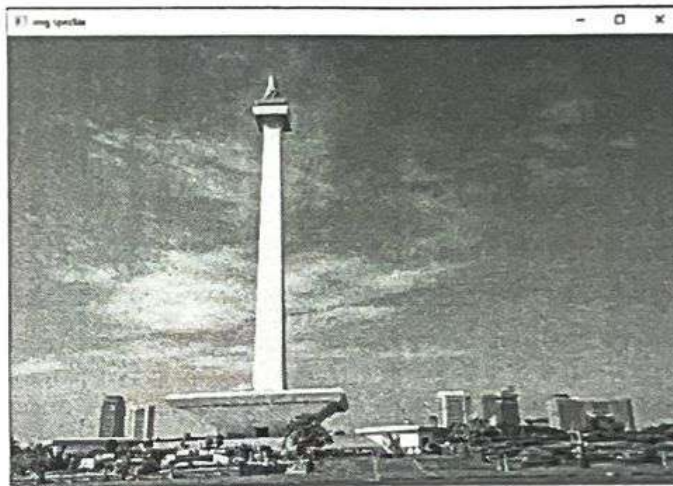
```

Restorasi yang digunakan pada latihan ini ialah *mean filter*, dan *median filter*. Hasil menjalankan program di atas dapat dilihat pada Gambar 42, Gambar 43, Gambar 44, Gambar 45, dan Gambar 46.

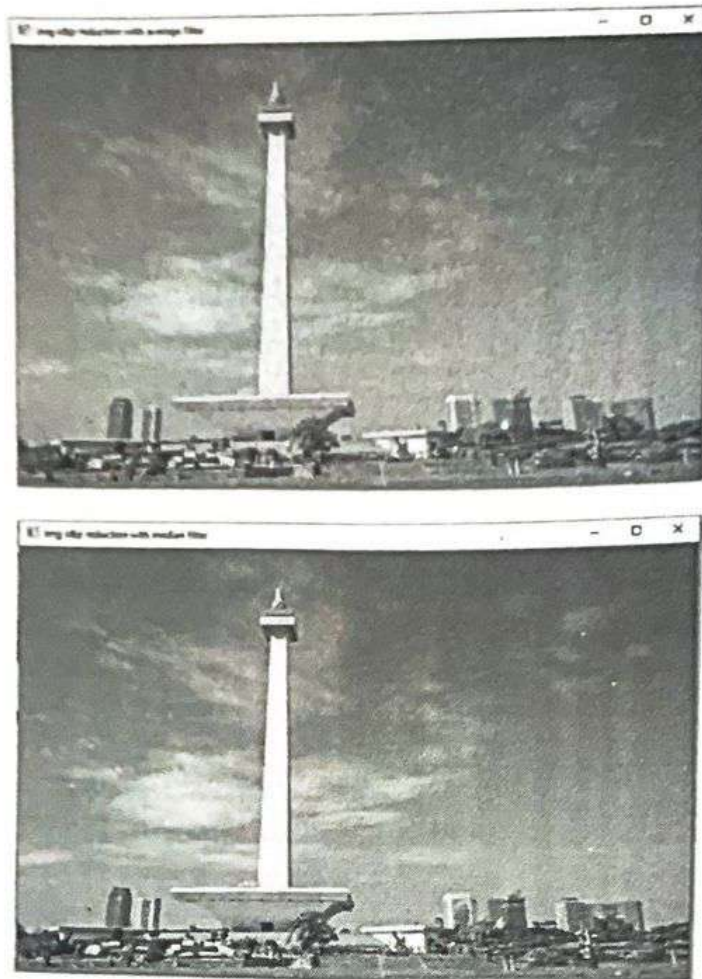
Output



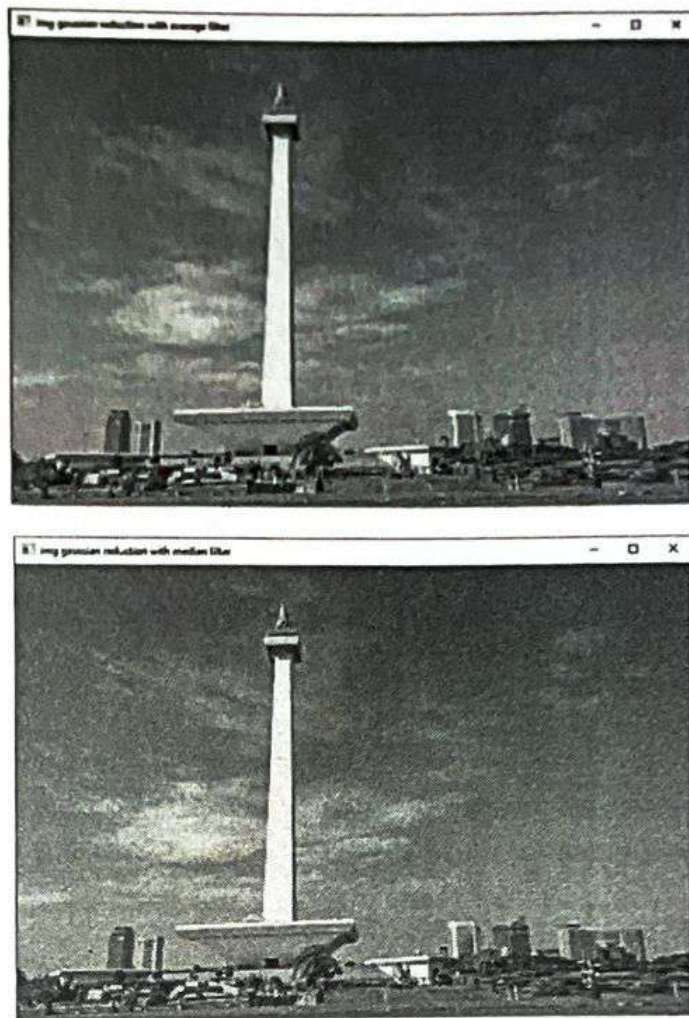
Gambar 42. Citra Asli.



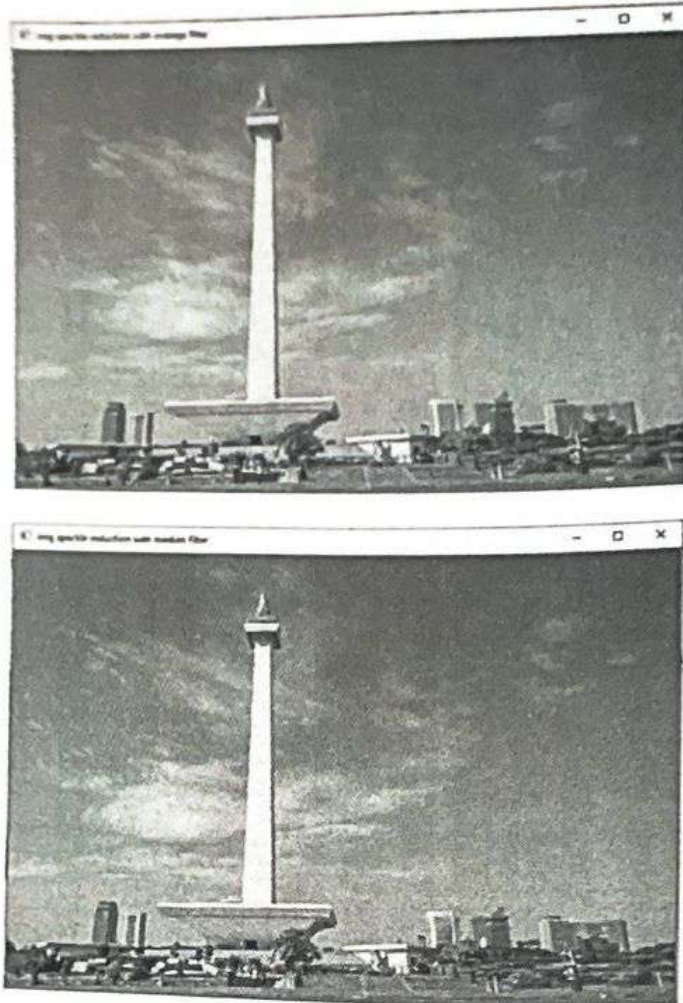
Gambar 43. Citra Hasil Penambahan *Noise*.



Gambar 44. Citra Hasil Restorasi Pada *Salt and Pepper Noise*.



Gambar 45. Citra Hasil Restorasi pada *Gaussian Noise*.



Gambar 46. Citra Hasil Restorasi pada *Speckle Noise*.

Latihan 2: Domain frekuensi

Berikut ini merupakan latihan membuat program untuk melakukan operasi low pass filter (LPF), high pass filter (HPF), dan juga merestorasi *periodic noise* di domain frekuensi menggunakan Discrete Fourier Transform (DFT).

```
import numpy as np
import cv2
from matplotlib import pyplot as plt

img = cv2.imread('halftone.png',0)

img_float32 = np.float32(img)

dft = cv2.dft(img_float32, flags = cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft)

rows, cols = img.shape
crow, ccol = int(rows/2), int(cols/2) # center

mask_lpf = np.zeros((rows, cols, 2), np.uint8)
mask_lpf[crow-30:crow+30, ccol-30:ccol+30] = 1

mask_hpf = np.ones((rows, cols, 2), np.uint8)
mask_hpf[crow-50:crow+50, ccol-50:ccol+50] = 0

mask_red = np.ones((rows, cols, 2), np.uint8)

for i in range(60, rows, 135):
    for j in range(100, cols, 200):
        if not (i == 330 and j == 500):
            mask_red[i-10:i+10, j-10:j+10] = 0
    for i in range(0, rows, 135):
        for j in range(200, cols, 200):
            if not (i == 330 and j == 500):
                mask_red[max(0,i-15):min(rows,i+15), max(0,j-15):min(cols,j+15)] = 0
```

```

fshift_lpf = dft_shift*mask_lpf
f_ishift_lpf = np.fft.ifftshift(fshift_lpf)
img_back_lpf = cv2.idft(f_ishift_lpf)
img_back_lpf = cv2.magnitude(img_back_lpf[:,0],img_back_lpf[:,1])

fshift_hpf = dft_shift*mask_hpf
f_ishift_hpf = np.fft.ifftshift(fshift_hpf)
img_back_hpf = cv2.idft(f_ishift_hpf)
img_back_hpf = cv2.magnitude(img_back_hpf[:,0],img_back_hpf[:,1])

fshift_red = dft_shift*mask_red
f_ishift_red = np.fft.ifftshift(fshift_red)
img_back_red = cv2.idft(f_ishift_red)
img_back_red = cv2.magnitude(img_back_red[:,0],img_back_red[:,1])

I_out_lpf =
cv2.normalize(img_back_lpf,None,0,255,cv2.NORM_MINMAX,cv2.CV_8U)
I_out_hpf =
cv2.normalize(img_back_hpf,None,0,255,cv2.NORM_MINMAX,cv2.CV_8U)
I_out_red =
cv2.normalize(img_back_red,None,0,255,cv2.NORM_MINMAX,cv2.CV_8U)

```

```

magnitude_spectrum =
20*np.log(cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1]))

plt.subplot(421), plt.imshow(img, cmap = 'gray')
plt.title('Input Image'), plt.xticks([]), plt.yticks([])
plt.subplot(422), plt.imshow(magnitude_spectrum, cmap = 'gray')
plt.title('Magnitude Spectrum Input Image'), plt.xticks([]), plt.yticks([])

plt.subplot(423), plt.imshow(img_back_lpf, cmap = 'gray')
plt.title('Result Image LPF'), plt.xticks([]), plt.yticks([])
plt.subplot(424), plt.imshow(mask_lpf[:, :, 0], cmap = 'gray')
plt.title('Filter LPF'), plt.xticks([]), plt.yticks([])

plt.subplot(425), plt.imshow(img_back_hpf, cmap = 'gray')
plt.title('Result Image HPF'), plt.xticks([]), plt.yticks([])
plt.subplot(426), plt.imshow(mask_hpf[:, :, 0], cmap = 'gray')
plt.title('Filter HPF'), plt.xticks([]), plt.yticks([])

plt.subplot(427), plt.imshow(img_back_red, cmap = 'gray')
plt.title('Result Image Noise Reduction'), plt.xticks([]), plt.yticks([])
plt.subplot(428), plt.imshow(mask_red[:, :, 0], cmap = 'gray')
plt.title('Filter Noise Reduction'), plt.xticks([]), plt.yticks([])

plt.show()

cv2.imshow('Result Image Filter LPF', I_out_lpf)
cv2.imshow('Result Image Filter HPF', I_out_hpf)
cv2.imshow('Result Image Noise Reduction', I_out_red)
cv2.imshow('Input Image', img)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

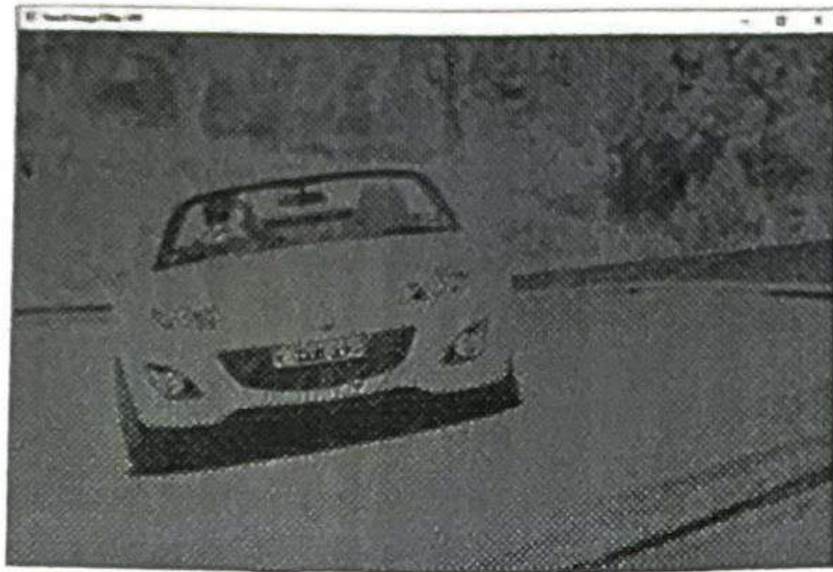
Output



Gambar 47. Citra Input dengan *Periodic Noise*.



Gambar 48. Citra Hasil *Low Pass Filter Noise* Menggunakan DFT.



Gambar 49. Citra Hasil *High Pass Filter Noise* Menggunakan DFT.



Gambar 50. Citra Hasil Restorasi *Periodic Noise* Menggunakan DFT.

C. Lembar Kerja Praktikum

1. Bung Tomo adalah pahlawan yang terkenal karena peranannya dalam membangkitkan semangat rakyat untuk melawan kembalinya penjajah Belanda melalui tentara NICA, yang berakhir dengan pertempuran 10 November 1945 yang hingga kini diperingati sebagai Hari Pahlawan [2]. Di mesin pencari terdapat salah satu foto Bung Tomo yang telah terdegradasi dengan *noise periodic*. Tugas

Kita sebagai mahasiswa informatika yang paham akan ilmu pengolahan citra digital yaitu lakukan restorasi citra terhadap foto Bung Tomo tersebut. Buatlah program dengan menggunakan Bahasa Pemrograman Python dan *library* Open-CV untuk proses *image reduction* terhadap foto Bung Tomo yang telah terdegradasi dengan *periodic noise*.



Gambar 51. Sutomo.jpg.

Referensi

- [1] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.
- [2] Wikipedia, "Sutomo," <https://id.wikipedia.org/>, 2020.
<https://id.wikipedia.org/wiki/Sutomo>.

BAB VIII

SEGMENTASI CITRA (*IMAGE SEGMENTATION*)

Materi:

- Segmentasi Citra
- *Thresholding*
- Deteksi Tepi (*Edge Detection*)

Tujuan Pembelajaran:

- Mahasiswa dapat mengimplementasikan teknik *thresholding* dan *edge detection* menggunakan OpenCV-Python.

A. Penyajian

1. Segmentasi Citra

Segmentasi citra merupakan proses mempartisi citra menjadi beberapa daerah atau objek. Umumnya segmentasi bertujuan untuk memisahkan suatu objek citra dengan objek lainnya dan atau memisahkan objek citra dengan latar belakang citra. Segmentasi citra didasarkan pada dua hal yaitu diskontinuitas dan kemiripan intensitas piksel. Pendekatan diskontinuitas disebut juga pendekatan berbasis *edge* (*edge-based*), sementara pendekatan kemiripan intensitas, seperti *thresholding*, *clustering* (contoh: k-means), klasifikasi, pendekatan teori graf (contoh: GrabCut), *region growing*, dan *region splitting & merging*.

2. Thresholding

Thresholding merupakan teknik yang paling dasar untuk melakukan segmentasi citra. Teknik *thresholding* dapat digunakan untuk melakukan segmentasi citra berdasarkan warna tertentu atau berdasarkan suatu nilai piksel tertentu.

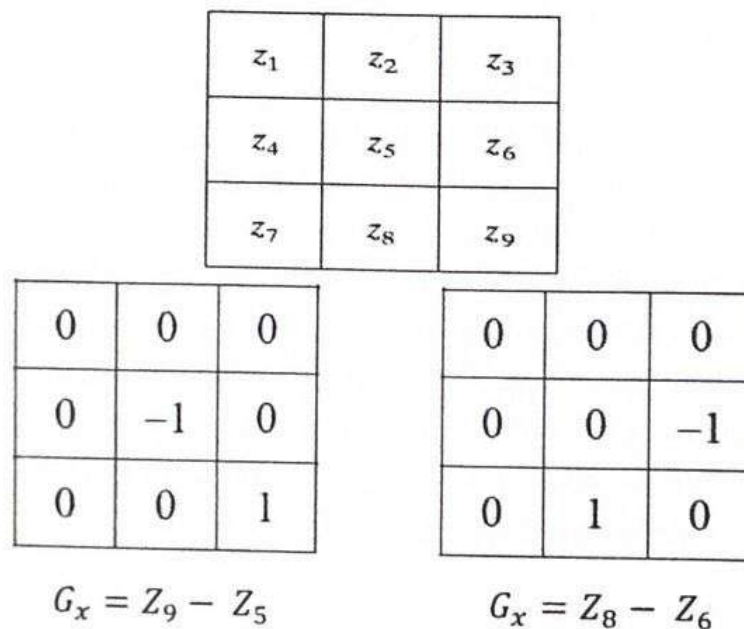
3. Deteksi Tepi (*Edge Detection*)

Deteksi tepi adalah operasi yang dijalankan untuk mendeteksi garis tepi yang membatasi dua wilayah citra homogen yang memiliki tingkat kecerahan yang berbeda. *Edge* adalah bagian dari citra di mana intensitas kecerahan berubah secara drastis. Tujuan dari deteksi tepi pada umumnya yaitu untuk mengurangi jumlah data pada citra pada langkah *image processing* berikutnya. Deteksi tepi adalah pendekatan yang sering digunakan untuk segmentasi citra digital dengan berdasarkan perubahan intensitas yang lokal. Ada beberapa pendekatan atau metode untuk deteksi tepi, yaitu *first order edge detection* dan *second order edge detection* [1].

a. *First Order Derivative Edge Detection*

Metode deteksi tepi yang termasuk ke dalam *first order derivative* adalah *Roberts operators*, *Prewitt operators*, *Sobel operators*, dan *First-order of gaussian* (FDOG). Pada modul ini akan dicoba untuk operator Sobel, Prewitt, dan Roberts. Setiap operator memiliki *row gradient* dan *column gradient*.

1) Roberts Operator



Gambar 52. Roberts Operator

2) Prewitt Operator

-1	-1	-1
0	0	0
1	1	1

$$G_x = (Z_7 + Z_8 + Z_9) - (Z_1 + Z_2 + Z_3) - G_x$$

1	1	1
0	0	0
-1	-1	-1

$$G_y = (Z_7 + Z_8 + Z_9) - (Z_1 + Z_2 + Z_3) - G_x$$

Gambar 53. Prewitt Operator

OpenCV-Python tidak menyediakan fungsi khusus untuk operator **roberts** dan **prewitt**. Namun dapat menggunakan fungsi **filter2D** yang terdapat pada OpenCV-Python dengan mendefinisikan sendiri kernel-nya, sesuai dengan operator yang ingin digunakan. Fungsi **filter2D** telah dijelaskan pada Modul VI: *Image Enhancement* dengan *Mask Processing*:

```
dst = cv.filter2D(src, ddepth, kernel, anchor, delta,  
borderType)
```

3) Sobel Operator

Operator Sobel adalah operasi smoothing Gaussian bersama diferensiasi, sehingga lebih tahan terhadap noise. Arah turunan dapat kita tentukan, baik secara vertikal maupun horizontal (masing-masing dengan argumen x order dan y order). Dokumen lebih detail terkait kernel yang digunakan dapat dilihat pada:

- https://docs.opencv.org/4.1.2/d4/d86/group_imgproc_filter.html#gacea54f142e81b6758cb6f375ce782c8d.

Berikut contoh dengan parameter sobel (xorder = 1, yorder = 0, ksize = 3) atau (xorder = 0, yorder = 1, ksize = 3).

-1	-2	-1
0	0	0
1	2	1

$$G_x = (Z_7 + 2Z_8 + Z_9) - (Z_1 + 2Z_2 + Z_3) - G_x$$

1	2	1
0	0	0
-1	-2	-1

$$G_y = (Z_7 + 2Z_8 + Z_9) - (Z_1 + 2Z_2 + Z_3) - G_y$$

Gambar 54. Sobel Operator

Pada OpenCV-Python dapat menggunakan fungsi sebagai berikut [2]:

```
dst = cv.Sobel(src, ddepth, dx, dy, ksize)
```

keterangan

dst : citra *output* dengan ukuran yang sama dan jumlah *channel* yang sama dengan src.
src : citra input
dx : order dari *derivative* x.
dy : order dari *derivative* y.
ksize : ukuran kernel Sobel (1, 3, 5, atau 7).

b. Second Order Derivative Edge Detection

1) Laplacian Derivatives

Laplacian gambar dapat dihitung dengan,

$$\Delta src = \frac{\partial^2 src}{\partial x^2} + \frac{\partial^2 src}{\partial y^2}$$

di mana setiap turunan ditemukan menggunakan turunan Sobel. Jika ksize = 1, maka kernel berikut digunakan untuk memfilter:

$$kernel = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Pada OpenCV-Python dapat menggunakan fungsi sebagai berikut:

`dst = cv.Laplacian(src, ddepth, ksize)`

keterangan

- dst : citra *output* dengan ukuran yang sama dan jumlah *channel* yang sama dengan src.
- src : citra input.
- ksize : Ukuran aperture yang digunakan untuk menghitung filter *derivative* kedua. Ukuran harus bernilai positif dan ganjil, untuk nilai default adalah 1.

2) Canny detection

Materi lebih lanjut tentang Canny dapat dilihat pada file Canny Operator di LMS. Untuk melakukan canny, dilakukan 5 langkah berikut [2]:

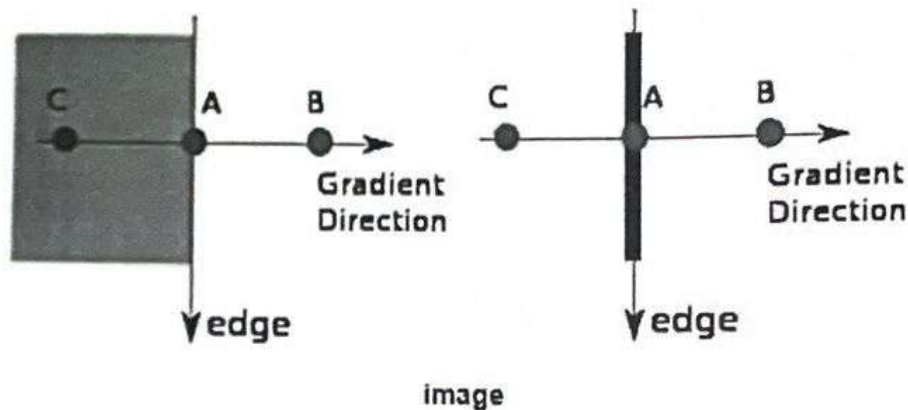
- a) *Smoothing*, melakukan *blurring* untuk menghilangkan *noise*.
Karena deteksi tepi rentan terhadap noise pada gambar, langkah pertama adalah menghilangkan noise pada gambar dengan filter Gaussian 5x5.
- b) Menemukan gradien
Gambar yang dihaluskan kemudian dilakukan filter dengan kernel Sobel pada order horizontal (G_x) dan vertikal (G_y) untuk mendapatkan turunan pertama. Dari dua gambar ini, kita dapat menemukan gradien tepi dan arah untuk setiap piksel sebagai berikut:

$$Edge_Gradient (G) = \sqrt{G_x^2 + G_y^2}$$

$$Angle (\theta) = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

c) *Non-maximum suppression*

Tujuan dari step ini adalah untuk meng-convert edge 'blur' dari step sebelumnya menjadi *edge* yang 'sharp'. Algoritma ini dilakukan untuk setiap pixel pada citra gradient. Hanya maksimum lokal yang ditandai sebagai edge. Setiap piksel diperiksa jika itu adalah maksimum lokal di lingkungannya (*neighborhood*) dalam arah gradien (*gradient directions*). Lihat gambar di bawah ini.



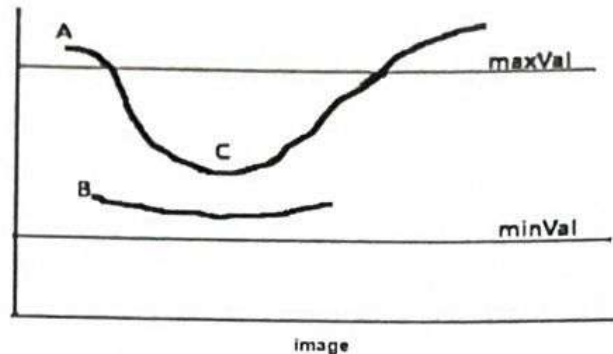
Gambar 55. *Non-Maximum Suppression*

Titik A ada di edge (arah vertikal). Titik B dan C berada dalam *gradient directions*. Jadi titik A diperiksa dengan titik B dan C untuk melihat apakah itu membentuk maksimum lokal. Jika demikian, itu dipertimbangkan untuk tahap berikutnya, jika tidak, itu ditekan (nol). Singkatnya, hasil yang didapatkan pada tahapan ini adalah gambar biner dengan "tepi tipis".

d) *Hysteresis Thresholding*

Tahap ini menentukan mana benar-benar sebagai tepi dan mana yang tidak. Untuk ini, kita membutuhkan dua nilai *threshold*, yaitu minVal dan maxVal. Setiap tepi dengan nilai intensitas gradien lebih dari maxVal adalah tepi dan yang di bawah minVal pasti non-edge. Intensitas yang berada di antara dua *threshold* ini dapat diklasifikasikan sebagai tepi

atau non-tepi berdasarkan konektivitas mereka. Jika mereka terhubung ke piksel dapat diklasifikasikan sebagai tepi. Kalau tidak, mereka dapat dibuang. Lihat gambar di bawah ini.



Gambar 56. *Hysteresis Thresholding*

Edge A berada di atas *maxVal*, sehingga dianggap sebagai “tepi”. Meskipun *edge C* berada di bawah *maxVal*, namun *edge C* terhubung ke *edge A*, sehingga dianggap sebagai *edge* yang valid dan didapatkan kurva penuh tersebut sebagai *edge*. Untuk *edge B*, meskipun berada di atas *minVal* dan berada di wilayah yang sama dengan tepi *C*, namun tidak terhubung ke “tepi”, sehingga *edge* tersebut dibuang. Maka sangat penting memilih *minVal* dan *maxVal* yang sesuai untuk mendapatkan hasil yang benar.

Tahap ini juga menghilangkan noise piksel kecil dengan asumsi tepi adalah garis panjang.

Pada OpenCV-Python dapat menggunakan fungsi sebagai berikut:

```
edges = cv.Canny(image, threshold1, threshold2)
```

keterangan

edges : output tepi (edge), channel tunggal 8-bit yang memiliki ukuran yang sama dengan citra input.

image : citra input 8-bit.

threshold1 : *threshold* pertama untuk prosedur hysteresis.
threshold2 : *threshold* kedua untuk prosedur hysteresis.

B. Latihan

1. Baca gambar 'monas.jpg', lalu terapkan deteksi *edge* menggunakan operator sobel pada order horizontal (G_x), vertical (G_y), dan magnitude-nya, serta lakukan deteksi *edge* dengan menggunakan operator *Laplacian Derivatives*. Lalu tampilkan kelima gambar (gambar asli, gambar hasil filter sobel derivative x, gambar hasil filter sobel derivative y, gambar hasil magnitude, serta hasil filter Laplacian Derivatives) dengan menggunakan `cv.imshow` dan `subplot matplotlib`.

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('monas.jpg',0)

laplacian64f = cv.Laplacian(img,cv.CV_64F)
abs_laplacian64f = np.absolute(laplacian64f)
laplacian_8u = np.uint8(abs_laplacian64f)

sobelx64f = cv.Sobel(img,cv.CV_64F,1,0,ksize=3)
abs_sobelx64f = np.absolute(sobelx64f)
sobelx_8u = np.uint8(abs_sobelx64f)

sobely64f = cv.Sobel(img,cv.CV_64F,0,1,ksize=3)
abs_sobely64f = np.absolute(sobely64f)
sobely_8u = np.uint8(abs_sobely64f)

magnitudesobel = cv.magnitude(sobelx64f,sobely64f)
abs_sobel64f = np.absolute(magnitudesobel)
sobel_8u = np.uint8(abs_sobel64f)
```

```

plt.subplot(3,2,1),plt.imshow(img,cmap = 'gray')
plt.title('Original'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,2),plt.imshow(laplacian_8u,cmap = 'gray')
plt.title('Laplacian'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,3),plt.imshow(sobelx_8u,cmap = 'gray')
plt.title('Sobel X'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,4),plt.imshow(sobely_8u,cmap = 'gray')
plt.title('Sobel Y'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,5),plt.imshow(sobel_8u,cmap = 'gray')
plt.title('Sobel Magnitude'), plt.xticks([]), plt.yticks([])
plt.show()

cv.imshow('Original', img)
cv.imshow('Laplacian', laplacian_8u)
cv.imshow('Sobel X', sobelx_8u)
cv.imshow('Sobel Y', sobely_8u)
cv.imshow('Sobel Magnitude', sobel_8u)
cv.waitKey(0)
cv.destroyAllWindows()

```

2. Baca gambar 'monas.jpg', lalu terapkan deteksi edge menggunakan *canny detection*.

```

import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('monas.jpg',0)
edges = cv.Canny(img,100,200)
plt.subplot(121),plt.imshow(img,cmap = 'gray')
plt.title('Original Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(edges,cmap = 'gray')
plt.title('Edge Image'), plt.xticks([]), plt.yticks([])
plt.show()

cv.imshow('Original', img)
cv.imshow('Canny', edges)

cv.waitKey(0)
cv.destroyAllWindows()

```

C. Lembar Kerja Praktikum

1. Indonesia terkenal dengan keanekaragaman hayatinya, baik hewan dan tumbuhan. Banyak jenis tumbuhan khas Indonesia yang mungkin ada di sekitar Kita. Tugas Anda adalah ambil foto di sekitar terkait tumbuhan khas Indonesia. Foto yang diambil adalah bagian daun tumbuhan dengan latar belakang warna putih (daun dapat dipetik terlebih dahulu, lalu letakan di atas kertas putih). Daftar tumbuhan dapat dilihat pada halaman website ini (<https://rimbakita.com/daftar-nama-tumbuhan-di-indonesia/>).
 - a. Lakukan deteksi tepi dengan menggunakan operator sobel pada *order horizontal* (G_x), *vertical* (G_y), dan *magnitude*-nya, lakukan juga deteksi *edge* dengan menggunakan operator *laplacian derivatives* dan *canny detection*.
 - b. Lakukan segmentasi terhadap foto tersebut sedemikian sehingga latar foto bernilai 0 dan objek daun sesuai dengan intensitas objek tersebut. Pemrosesan citra menggunakan *grayscale*.

Referensi

- [1] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.
- [2] A. Mordvintsev and A. K, "OpenCV-Python Tutorials," 2019. https://docs.opencv.org/4.1.2/d6/d00/tutorial_py_root.html.

BAB IX MORFOLOGI

Materi:

- Definisi dan Fungsi Morfologi
- Dilasi
- Erosi
- *Opening*
- *Closing*

Tujuan Pembelajaran:

- Mahasiswa dapat mengimplementasikan operasi-operasi morfologi di OpenCV-Python.

A. Penyajian

1. Definisi dan Fungsi Morfologi

Kata morfologi biasa kita dengar di cabang biologi yaitu yang berhubungan dengan bentuk dan struktur hewan dan tumbuhan. Sama halnya dalam konteks morfologi matematis yaitu sebagai alat untuk mengekstraksi komponen gambar yang berguna dalam representasi dan deskripsi bentuk wilayah, seperti batas dan kerangka. Teknik morfologi juga berguna untuk *pre-processing* dan *post-processing* citra, seperti penipisan dan pemangkasan, misalnya untuk memperbaiki hasil segmentasi [1]. Morfologi adalah proses mengidentifikasi bentuk dengan basis region (citra bertipe biner & *grayscale*). Adapun operasi-operasi morfologi yang sering digunakan ialah dilasi, erosi, *opening*, dan *closing*.

2. Dilasi

Merupakan proses penggabungan titik-latar 0 menjadi bagian dari objek 1 berdasarkan *structuring element* (SE) yang digunakan. Notasi dilasi dapat ditulis berdasarkan persamaan di bawah ini.

$$D(A, S) = A \oplus S$$

Keterangan:

$D(A, S)$: Fungsi dilasi

A : Citra

S : *Structure element*

$\oplus$: Simbol dilasi

Fungsi Dilasi pada OpenCV [2]:

```
dst = cv.dilate(src, kernel, anchor, iterations, borderType, borderValue)
```

Keterangan:

src : input image; jumlah channel bisa berapa saja, tetapi kedalamannya harus salah satu dari CV_8U, CV_16U, CV_16S, CV_32F or CV_64F.

dst : output image dengan ukuran dan tipe yang sama dengan src.

kernel : *structuring element* yang digunakan untuk dilasi; jika *element* = Mat (), *structuring element* persegi 3 x 3 digunakan. Kernel dapat dibuat menggunakan `getStructureElement`.

(https://docs.opencv.org/4.1.2/d4/d86/group_imgproc_filter.html)

anchor : posisi jangkar di dalam elemen; nilai default (-1,-1) berarti bahwa jangkar berada di pusat elemen.

iterations : *number of times dilation is applied*.

borderType : metode ekstrapolasi piksel, lihat `BorderTypes` (https://docs.opencv.org/4.1.2/d2/de8/group_core_array.html)

borderValue : *border value* untuk nilai border yang konstan

3. Erosi

Merupakan proses penghapusan titik-titik objek (1) menjadi bagian dari latar (0), berdasarkan *structuring element* (SE) yang digunakan. Notasi erosi dapat ditulis berdasarkan persamaan di bawah ini.

$$E(A, S) = A \otimes S$$

Keterangan:

$E(A, S)$: Fungsi erosi

A : Citra

S : *Structure element*

$\otimes$: Simbol erosi

Fungsi Erosi pada OpenCV [2]:

`Dst = cv.erode(src, kernel, anchor, iterations, borderType, borderValue)`

Keterangan:

src : input image; jumlah channel bisa berapa saja, tetapi kedalamannya harus salah satu dari CV_8U, CV_16U, CV_16S, CV_32F or CV_64F.

dst : output image dengan ukuran dan tipe yang sama dengan src.

kernel : *structuring element* yang digunakan untuk dilasi; jika element = Mat (), *structuring element* persegi 3 x 3 digunakan. Kernel dapat dibuat menggunakan `getStructureElement`.

(https://docs.opencv.org/4.1.2/d4/d86/group_imgproc_filter.html)

anchor : posisi jangkar di dalam elemen; nilai default (-1,-1) berarti bahwa jangkar berada di pusat elemen.

iterations : number of times dilation is applied.

borderType : metode ekstrapolasi piksel, lihat `BorderTypes` (https://docs.opencv.org/4.1.2/d2/de8/group_core_array.html)

borderValue : *border value* untuk nilai border yang konstan

4. Opening dan Closing

Opening adalah proses erosi yang diikuti dengan dilasi. Efek yang dihasilkan adalah menghilangkan objek-objek kecil dan kurus, memecah objek pada titik-titik yang kurus, dan secara umum menghaluskan batas objek besar tanpa mengubah area objek secara signifikan. *Opening* berguna untuk menghaluskan citra, menghilangkan tonjolan yang tipis, dan memisahkan objek. Notasi *opening* dapat ditulis berdasarkan persamaan di bawah ini.

$$A \circ S = (A \otimes S) \oplus S$$

Closing adalah proses dilasi yang diikuti dengan erosi. Efek yang dihasilkan adalah mengisi lubang kecil pada objek, menggabungkan objek-objek yang berdekatan, dan secara umum menghaluskan batas dari objek besar tanpa mengubah area objek secara signifikan. Teknik ini berguna untuk menghaluskan citra, menghilangkan lubang yang kecil. Notasi *closing* dapat ditulis berdasarkan persamaan di bawah ini.

$$A \circ S = (A \oplus S) \otimes S$$

Fungsi Opening pada OpenCV [2]:

```
dst = cv.morphologyEx(src, op, kernel, anchor, iterations, borderType,
borderValue)
```

Keterangan:

- src : input image; jumlah channel bisa berapa saja, tetapi kedalamannya harus salah satu dari CV_8U, CV_16U, CV_16S, CV_32F or CV_64F.
- dst : output image dengan ukuran dan tipe yang sama dengan src.
- op : tipe dari operasi morphological, untuk closing: cv.MORPH_CLOSE dan untuk opening: cv.MORPH_OPEN, lihat **MorphTypes** (https://docs.opencv.org/4.1.2/d4/d86/group_imgproc_filter.html)

- kernel** : *structuring element* yang digunakan untuk dilasi; jika *element* = `Mat()`, *structuring element* persegi 3 x 3 digunakan. Kernel dapat dibuat menggunakan `getStructureElement`.
(https://docs.opencv.org/4.1.2/d4/d86/group_imgproc_filter.html)
- anchor** : posisi jangkar di dalam elemen; nilai default (-1,-1) berarti bahwa jangkar berada di pusat elemen.
- iterations** : number of times dilation is applied.
- borderType** : metode ekstrapolasi piksel, lihat `BorderTypes`
(https://docs.opencv.org/4.1.2/d2/de8/group_core_array.html)
- borderValue** : *border value* untuk nilai border yang konstan

Contoh Structuring Element

```
# Rectangular Kernel
>>> cv.getStructuringElement(cv.MORPH_RECT,(5,5))
array([[1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1]], dtype=uint8)
# Elliptical Kernel
>>> cv.getStructuringElement(cv.MORPH_ELLIPSE,(5,5))
array([[0, 0, 1, 0, 0],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [0, 0, 1, 0, 0]], dtype=uint8)
# Cross-shaped Kernel
>>> cv.getStructuringElement(cv.MORPH_CROSS,(5,5))
array([[0, 0, 1, 0, 0],
       [0, 0, 1, 0, 0],
       [1, 1, 1, 1, 1],
       [0, 0, 1, 0, 0],
       [0, 0, 1, 0, 0]], dtype=uint8)
```

B. Latihan

1. Berikut ini merupakan latihan membuat program yang dapat melakukan dilasi, erosi, *opening*, *closing*.

Morfologi.py

```
import numpy as np
import cv2 as cv

def morfologi(img, se):
    dst_erosi = cv.erode(img, se, iterations = 1)
    dst_dilate = cv.dilate(img, se, iterations = 1)
    dst_opening = cv.morphologyEx(img, cv.MORPH_OPEN, se, iterations = 1)
    dst_closing = cv.morphologyEx(img, cv.MORPH_CLOSE, se, iterations = 1)
    return dst_erosi, dst_dilate, dst_opening, dst_closing

def main():
    img1 = cv.imread('circbw.tif', 0)
    s_element = cv.getStructuringElement(cv.MORPH_RECT, (3, 3))
    dst_erosi, dst_dilate, dst_opening, dst_closing = morfologi(img1, s_element)

    cv.imshow('dst_erosi', dst_erosi)
    cv.imshow('dst_dilate', dst_dilate)
    cv.imshow('dst_opening', dst_opening)
    cv.imshow('dst_closing', dst_closing)
    cv.waitKey(0)
    cv.destroyAllWindows()
```

2. Ingat kembali pada latihan di Modul 3: Ruang Warna Pengolahan Citra, dalam latihan tersebut kita mencoba mensegmentasi objek apel warna merah. Berikut ini adalah kode programnya:

```
import numpy as np
import cv2 as cv

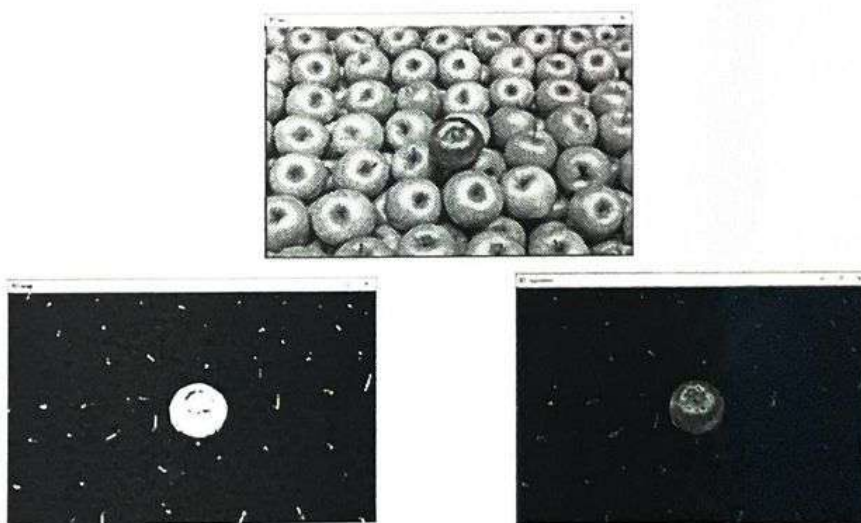
#membaca citra RGB
rgb = cv.imread('red_apple.jpg')
ycrCb = cv.cvtColor(rgb, cv.COLOR_BGR2YCrCb)

lower_blue = np.array([40,150,50])
upper_blue = np.array([200,255,150])
# Threshold the HSV image to get only blue colors
mask = cv.inRange(ycrCb, lower_blue, upper_blue)

res = cv.bitwise_and(rgb,rgb, mask= mask)

cv.imshow('src', rgb)
cv.imshow('rgb', mask)
cv.imshow('objek', res)
cv.waitKey(0)
cv.destroyAllWindows()
```

Output:



Gambar 57. Citra Output Hasil Segmentasi tanpa Dilakukan Morfologi.

Dari hasil menggunakan segmentasi masih terlihat belum sempurna, yaitu masih ada noise pada latar belakang dan bentuk apel yang masih ada lubang kosong di dalamnya. Hasil segmentasi tersebut dapat diperbaiki dengan morfologi sebagai berikut.

```
import numpy as np
import cv2 as cv

#membaca citra RGB
rgb = cv.imread('red_apple.jpg')
ycrcb = cv.cvtColor(rgb, cv.COLOR_BGR2YCrCb)

lower_blue = np.array([40,150,50])
upper_blue = np.array([200,255,150])
# Threshold the HSV image to get only blue colors
img_biner = cv.inRange(ycrcb, lower_blue, upper_blue)

#mendefinisikan structuring element 3x3 dan 5x5
se_3 = cv.getStructuringElement(cv.MORPH_RECT,(3,3))
se_5 = cv.getStructuringElement(cv.MORPH_RECT,(5,5))
#operasi morfologi
dst_dilate = cv.dilate(img_biner, se_3, iterations = 1)
dst_erosi = cv.erode(dst_dilate, se_3, iterations = 2)
dst_dilate2 = cv.dilate(dst_erosi, se_5, iterations = 2)
dst_erosi2 = cv.erode(dst_dilate2, se_5, iterations = 3)
dst_dilate3 = cv.dilate(dst_erosi2, se_3, iterations = 1)

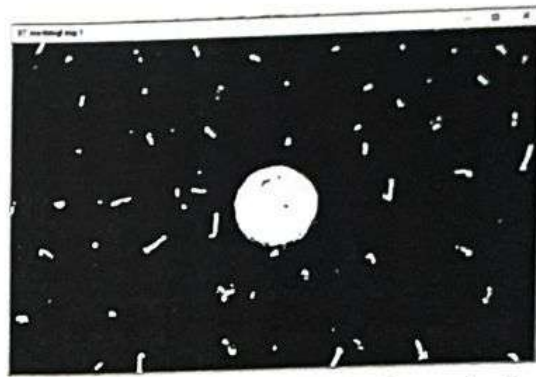
res = cv.bitwise_and(rgb,rgb, mask= dst_dilate3)

cv.imshow('src', rgb)
cv.imshow('rgb', dst_dilate3)
cv.imshow('objek', res)
cv.waitKey(0)
cv.destroyAllWindows()
```

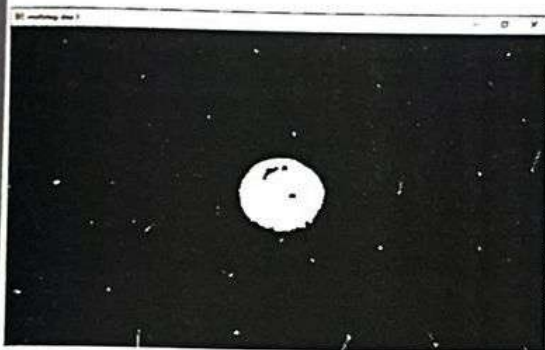
Keluaran:



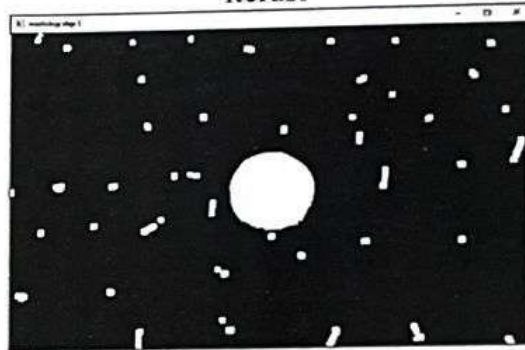
Citra awal



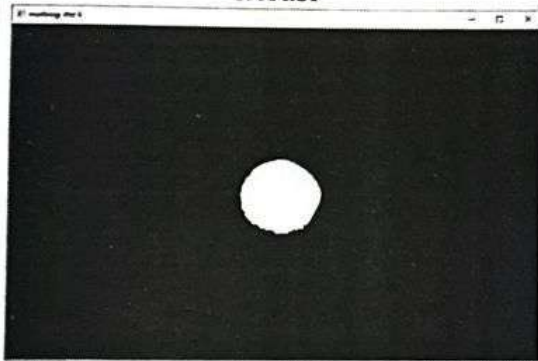
Hasil Dilasi dengan SE 3x3 satu kali iterasi



Hasil Erosi dengan SE 3x3 dua kali iterasi



Hasil Dilasi dengan SE 5x5 dua kali iterasi



Hasil Erosi dengan SE 5x5 tiga kali iterasi



Citra Hasil

Gambar 58. Citra Output Latihan No.2.

C. Lembar Kerja Praktikum

1. Burung jalak bali merupakan satu-satunya satwa endemik Pulau Bali yang masih tersisa. Sebagai mahasiswa informatika yang memiliki jiwa bela Negara, ingin berkontribusi terhadap kelestarian burung tersebut di Indonesia. Maka akan dibuat aplikasi berbasis pengolahan citra digital untuk mendeteksi jumlah burung jalak bali yang ada di suatu wilayah. Program membaca gambar, dan keluarannya adalah jumlah burung jalak bali yang ada di foto tersebut. Gunakan Teknik image enhancement, image restoration, segmentasi citra, post-processing dengan morfologi, dan hitung jumlah burung yang ada di gambar tersebut.



Gambar 59. Foto Burung Jalak Bali

Referensi

- [1] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.
- [2] A. Mordvintsev and A. K, "OpenCV-Python Tutorials," 2019. https://docs.opencv.org/4.1.2/d6/d00/tutorial_py_root.html.

BAB

X

EKSTRAKSI CIRI TEKSTUR

Materi:

- Konsep Tekstur pada Citra Digital
- *Local Binary Pattern*

Tujuan Pembelajaran:

- Mahasiswa dapat memahami konsep tekstur pada citra digital
- Mahasiswa dapat menerapkan *Local Binary Pattern* pada citra digital

A. Penyajian

1. Konsep Tekstur pada Citra

Salah satu tahapan yang harus dilakukan dalam pengenalan pola citra digital, adalah ekstraksi ciri atau disebut juga fitur. Ciri-ciri umum yang dapat diekstrak dalam sebuah citra digital yakni, ekstraksi ciri bentuk, ekstraksi ciri ukuran dan geometri, ekstraksi ciri tekstur dan ekstraksi ciri warna. Pada modul ini, kita akan mempelajari salah satu metode ekstraksi ciri, yakni ciri tekstur.

Ciri tekstur merupakan salah satu ciri penting dalam pengenalan pola. Misal Anda diminta untuk mendeteksi sebuah bola berwarna merah di lapangan rumput yang berwarna hijau maka dengan mudah Anda melakukannya dengan cara menggunakan ciri warna dari bola tersebut. Namun, jika Anda diminta untuk mendeteksi seekor macan tutul melewati padang rumput yang kering maka ciri warna tidak cukup untuk dapat mendeteksi macan tutul tersebut dengan akurat. Oleh karena itu, dalam kasus ini, ciri tekstur menjadi penting karena tekstur dari kulit macan tutul yang berbintik berbeda dengan tekstur rumput yang bergaris. Pada modul

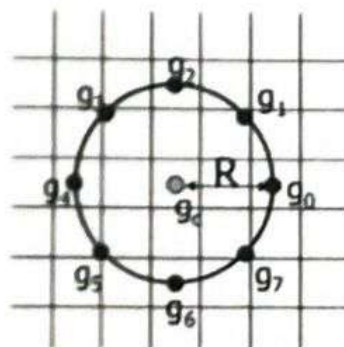
ini kita akan membahas salah satu metode ekstraksi ciri tekstur pada citra digital yakni *Local Binary Pattern* (LBP).

2. Local Binary Pattern

Local Binary Pattern (LBP) merupakan salah satu *texture descriptor* yang sederhana dan efisien dengan cara memberikan label pada piksel dengan melakukan *thresholding* pada setiap piksel tetangga dan mempertimbangkan hasilnya sebagai bilangan biner.

LBP menggunakan citra *grayscale* dengan ketetanggaan 3×3 , di mana nilai piksel pada posisi tengah digunakan sebagai ambang batas (*threshold*). Piksel-piksel tetangga kemudian dilakukan binerisasi dengan ketentuan apabila nilai piksel tetangga lebih kecil dibandingkan dengan ambang batas maka akan bernilai 0, sebaliknya akan bernilai 1. Setelah itu, nilai binerisasi pada setiap piksel dikalikan dengan bobot yang telah ditentukan pada piksel yang bersesuaian, yakni dikalikan dengan 2^n , di mana n adalah urutan piksel tetangga yang dimulai dari pojok kiri atas dengan mengitari piksel pada posisi tengah dengan arah dari kiri sampai dengan kanan bawah dengan urutan dimulai dari 0-7. Hasil perkalian pada seluruh piksel tetangga kemudian dilakukan penjumlahan.

LBP merupakan *descriptor* yang efisien yang mendeskripsikan pola tekstur lokal pada citra *grayscale*. LBP didefinisikan sebagai sekumpulan *pixel* ketetanggaan yang tersebar secara melingkar (*circular neighborhoods*) dengan *pixel* pusat berada di tengah seperti ditunjukkan Gambar 60. Notasi g_p merupakan nilai *pixel* tetangga ke- p . g_c merupakan *pixel* pusat yang digunakan sebagai nilai *threshold* agar *pixel* ketetanggaannya menjadi kode biner.



Gambar 60. *Circular Neighborhood Delapan Sampling Points*

Secara matematis, operator LBP dapat ditulis menggunakan persamaan di bawah ini:

$$LBP_{P,R}(x_c, y_c) = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p$$

$$s(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

di mana,

x_c dan y_c = koordinat *pixel* pusat

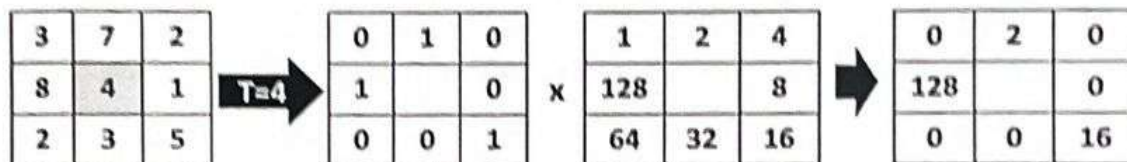
R = radius

P = titik sampling

g_p = nilai derajat keabuan pada piksel tetangga ke- p

g_c = nilai derajat keabuan pada piksel di posisi tengah

Contoh penggunaan LBP pada citra dapat dilihat pada Gambar 61.



Gambar 61. Contoh perhitungan nilai LBP.

$$\begin{aligned} \text{Pola} \rightarrow 01001001 &= 0(2^0) + 1(2^1) + 0(2^2) + 0(2^3) + 1(2^4) + 0(2^5) + 0(2^6) \\ &\quad + 1(2^7) \\ &= 0 + 2 + 0 + 0 + 16 + 0 + 0 + 128 = 146 \end{aligned}$$

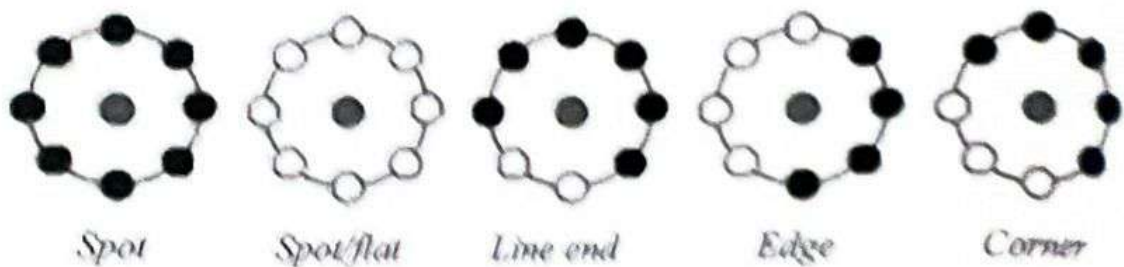
Untuk keperluan klasifikasi/pengenalan objek, nilai-nilai LBP selanjutnya direpresentasikan dalam bentuk histogram. Histogram menunjukkan frekuensi kejadian berbagai nilai LBP. Untuk ukuran citra $N \times M$, keseluruhan nilai LBP dapat direpresentasikan dengan membentuk histogram sebagai berikut:

$$H(k) = \sum_{i=0}^N \sum_{j=1}^M f(LBP_{P,R}(i, j), k), k \in [0, K]$$

$$f(x, y) = \begin{cases} 1, & x = y \\ 0, & \text{selainnya} \end{cases}$$

dengan k merupakan nilai LBP maksimum.

Ojala *et al.* (2002) melakukan observasi bahwa beberapa pola LBP tertentu memiliki informasi penting dari suatu tekstur. Pola-pola yang memiliki informasi penting ini disebut *uniform patterns*. LBP dikatakan *uniform* jika *discontinuities* atau transisi bit 0/1 paling banyak adalah dua. Sebagai contoh 00000000 (0 transisi), 11001111 (2 transisi), dan 11001111 (2 transisi) merupakan *uniform patterns*, sedangkan 11001001 (4 transisi) dan 10101001 (6 transisi) bukan merupakan *uniform patterns*. *Uniform patterns* berfungsi untuk mengidentifikasi noda (*spot*), *flat area* atau *dark spot*, sudut, dan tepi. Hampir 90 persen dari tekstur merupakan *uniform patterns* [1].



Gambar 62. Tekstur *Uniform Patterns*.

Gambar 62 menunjukkan makna dari *uniform patterns*. Secara matematis, *uniform patterns* dapat diekspresikan sebagai berikut:

$$U(LBP_{p,R}) = |s(g_{p-1} - g_c) - s(g_0 - g_c)| + \sum_{p=1}^{P-1} |s(g_p - g_c) - s(g_{p-1} - g_c)| \quad (7)$$

dengan $U(LBP_{p,R})$ merupakan *uniform patterns* dari jumlah *sampling points* (P) dan radius (R), p adalah *circular sampling point*, g_p adalah nilai keabuan dari p , dan g_c adalah nilai keabuan rata-rata seluruh *pixel neighborhood*. Jumlah pola yang dihasilkan *uniform patterns* adalah $P(P-1) + 2$ him

Sebelum kita mengimplementasikan LBP pada sebuah citra digital, kode program di bawah ini akan menampilkan tekstur uniform pattern yang terdapat dalam LBP [2].

```
import numpy as np
import matplotlib.pyplot as plt

METHOD = 'uniform'
plt.rcParams['font.size'] = 9

def plot_circle(ax, center, radius, color):
    circle = plt.Circle(center, radius, facecolor=color, edgecolor='0.5')
    ax.add_patch(circle)

def plot_lbp_model(ax, binary_values):
    """Draw the schematic for a local binary pattern."""
    # Geometry spec
    theta = np.deg2rad(45)
    R = 1
    r = 0.15
    w = 1.5
    gray = '0.5'

    # Draw the central pixel.
    plot_circle(ax, (0, 0), radius=r, color=gray)
    # Draw the surrounding pixels.
    for i, facecolor in enumerate(binary_values):
        x = R * np.cos(i * theta)
        y = R * np.sin(i * theta)
        plot_circle(ax, (x, y), radius=r, color=str(facecolor))

    # Draw the pixel grid.
    for x in np.linspace(-w, w, 4):
        ax.axvline(x, color=gray)
        ax.axhline(x, color=gray)

    # Tweak the layout.
    ax.axis('image')
    ax.axis('off')
    size = w + 0.2
```

```

ax.set_xlim(-size, size)
ax.set_ylim(-size, size)

fig, axes = plt.subplots(ncols=5, figsize=(7, 2))

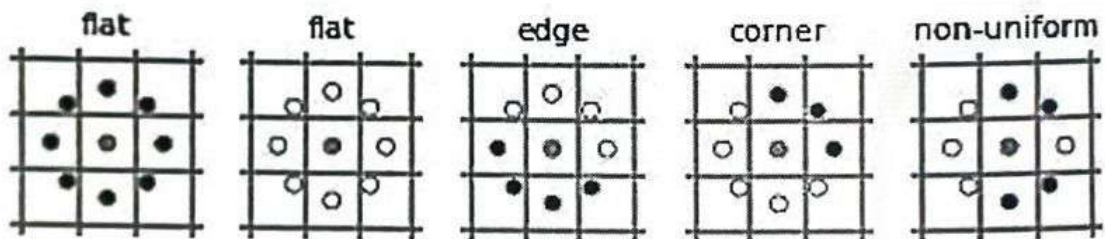
titles = ['flat', 'flat', 'edge', 'corner', 'non-uniform']

binary_patterns = [np.zeros(8),
                    np.ones(8),
                    np.hstack([np.ones(4), np.zeros(4)]),
                    np.hstack([np.zeros(3), np.ones(5)]),
                    [1, 0, 0, 1, 1, 1, 0, 0]]

for ax, values, name in zip(axes, binary_patterns, titles):
    plot_lbp_model(ax, values)
    ax.set_title(name)

```

Keluaran:



Gambar 63. Keluaran Kode Program untuk Menampilkan *Tekstur Uniform Pattern LBP*

Pada gambar di atas menunjukkan bahwa warna hitam atau putih mewakili piksel yang memiliki intensitas kurang atau lebih dibandingkan piksel pusatnya. Ketika piksel di sekitar piksel pusat semuanya berwarna hitam atau putih, maka maknanya wilayah tersebut datar (tidak ada tekstur pada wilayah tersebut). Sekumpulan piksel hitam atau putih yang muncul berturut-turut dianggap memiliki pola "*uniform*" yang diartikan bahwa terdapat sudut (*corner*) atau tepian (*edge*) pada wilayah tersebut. Namun jika piksel hitam dan putih muncul secara bergantian, maka wilayah tersebut memiliki pola "*non-uniform*".

Untuk implementasi LBP menggunakan Python, Anda dapat menggunakan *library* scikit-image yang mana merupakan *library* pengolahan citra digital yang bersifat *open source*. Pada *library* ini terdapat modul/function LBP yang dapat Anda gunakan langsung. Berikut definisi function LBP pada *library* scikit-image:

```
local_binary_pattern(image, P, R, method='default')
```

Terdapat 4 parameter pada function LBP, yakni:

image : (N, M) array, citra *grayscale*

P (int) : Titik sampling atau *Circular neighborhood sampling point*

R (float) : Radius

Method {'default', 'ror', 'uniform', 'var'}

Nama Method	Keterangan
'default'	local binary pattern biasa yang tidak bersifat rotation invariant.
'ror'	ekstensi dari LBP biasa yang bersifat rotation invariant.
'uniform'	memperbaiki LBP rotation invariance dengan menambahkan uniform patterns
'nri_uniform'	LBP dengan non rotation-invariant uniform patterns variant

B. Latihan

Berikut contoh penerapan LBP pada citra tekstur bata.

```
from skimage.transform import rotate
from skimage.feature import local_binary_pattern
from skimage import data
from skimage.color import label2rgb

# settings for LBP
radius = 3
n_points = 8 * radius

def overlay_labels(image, lbp, labels):
```

```

mask = np.logical_or.reduce([lbp == each for each in labels])
return label2rgb(mask, image=image, bg_label=0, alpha=0.5)

def highlight_bars(bars, indexes):
    for i in indexes:
        bars[i].set_facecolor('r')

image = data.brick()
lbp = local_binary_pattern(image, n_points, radius, METHOD)

def hist(ax, lbp):
    n_bins = int(lbp.max() + 1)
    return ax.hist(lbp.ravel(), density=True, bins=n_bins, range=(0, n_bins),
        facecolor='0.5')

# plot histograms of LBP of textures
fig, (ax_img, ax_hist) = plt.subplots(nrows=2, ncols=3, figsize=(9, 6))
plt.gray()

titles = ('edge', 'flat', 'corner')
w = width = radius - 1
edge_labels = range(n_points // 2 - w, n_points // 2 + w + 1)
flat_labels = list(range(0, w + 1)) + list(range(n_points - w, n_points + 2))
i_14 = n_points // 4      # 1/4th of the histogram
i_34 = 3 * (n_points // 4) # 3/4th of the histogram
corner_labels = (list(range(i_14 - w, i_14 + w + 1)) +
    list(range(i_34 - w, i_34 + w + 1)))

label_sets = (edge_labels, flat_labels, corner_labels)

for ax, labels in zip(ax_img, label_sets):
    ax.imshow(overlay_labels(image, lbp, labels))

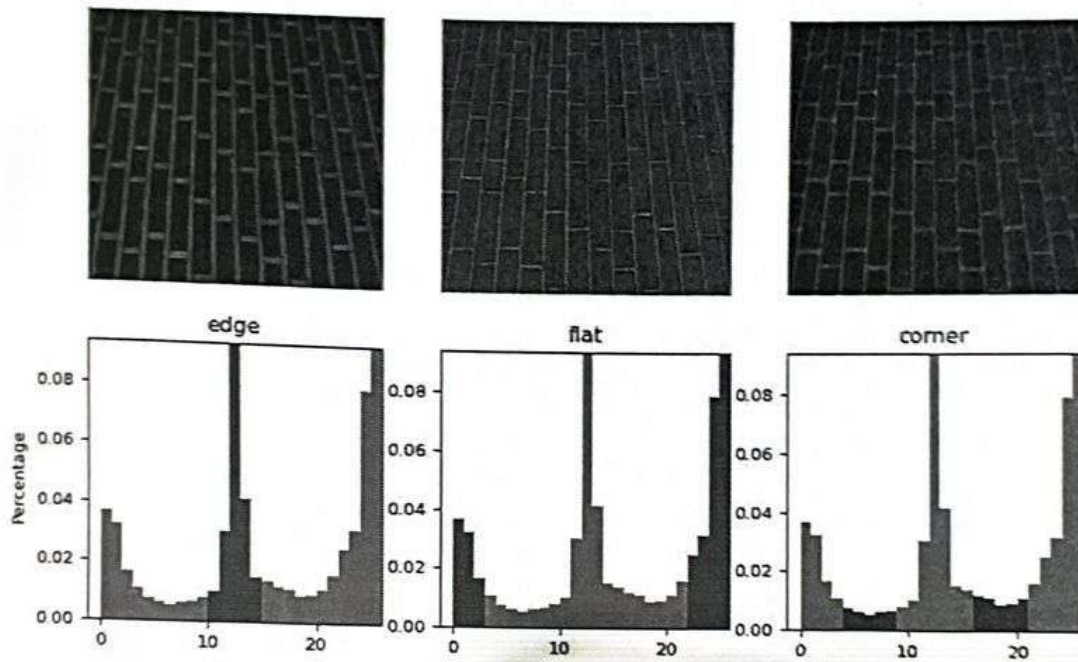
for ax, labels, name in zip(ax_hist, label_sets, titles):
    counts, _ , bars = hist(ax, lbp)
    highlight_bars(bars, labels)
    ax.set_ylim(top=np.max(counts[:-1]))
    ax.set_xlim(right=n_points + 2)

```

```
ax.set_title(name)

ax_hist[0].set_ylabel('Percentage')
for ax in ax_img:
    ax.axis('off')
```

Keluaran:



Gambar 64. Keluaran Kode Program pada Latihan No.1.

Penjelasan:

Pada gambar di atas, dengan menggunakan LBP, kita dapat mendeteksi area tepian, area datar dan sudut pada citra tekstur bata (lihat *highlight* merah pada gambar).

C. Lembar Kerja Praktikum

1. Batik merupakan warisan budaya Indonesia yang harus dilindungi. Kita mungkin sering sekali menggunakan batik baik dalam acara formal atau informal, namun kebanyakan dari Kita tidak mengetahui jenis atau motif batik yang kita kenakan. Batik di Indonesia memiliki berbagai macam jenis corak atau pola Batik. Maka,

sebagai wujud cinta Tanah Air dan Bela Negara, Mahasiswa Fakultas Ilmu Komputer UPN Veteran Jakarta ingin menciptakan sebuah inovasi dalam bentuk aplikasi untuk mengidentifikasi jenis-jenis batik yang ada di Indonesia berbasis citra digital. Tahap pertama adalah mempelajari jenis-jenis batik yang ada di Indonesia dari berbagai sumber (minimal 2 jenis batik). Selanjutnya, sebelum ke tahap klasifikasi/identifikasi lakukan ekstraksi ciri tekstur dengan LBP menggunakan citra batik yang telah Anda pilih. Lakukan percobaan dengan berbagai macam jenis LBP dan lakukan analisis hasilnya.



Gambar 65. Contoh Motif Batik Kawung dan Sidomukti.

Referensi

- [1] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, 2002, doi: 10.1109/TPAMI.2002.1017623.
- [2] the scikit-image development team, "Local Binary Pattern for texture classification," <https://scikit-image.org/>. https://scikit-image.org/docs/dev/auto_examples/features_detection/plot_local_binary_pattern.html (accessed Nov. 03, 2020).

BAB XI KLASIFIKASI CITRA

Materi:

- *Image Classification* dan *Image Recognition*
- Memahami k-Nearest Neighbor (kNN)
- Contoh Perhitungan kNN

Tujuan Pembelajaran:

- Mahasiswa dapat menerapkan metode klasifikasi citra sederhana menggunakan OpenCV-Python.

A. Penyajian

1. *Image Classification* dan *Image Recognition*

Image classification (klasifikasi citra) adalah proses mengidentifikasi atau mengenali sebuah objek pada sebuah citra digital. *Image classification* merupakan bagian dari *image recognition*. *Image recognition* adalah proses mengidentifikasi serta mendeteksi objek pada sebuah citra digital. *Image recognition* biasa disebut juga dengan *Computer Vision*.

Sebagai contoh, asumsikan pada sistem terdapat sekumpulan/himpunan label/kelas seperti anjing, kucing, truk, pesawat, dll. Kemudian, Anda memasukkan sebuah citra pada sistem tersebut, dan sistem akan memberitahu Anda bahwa citra yang Anda inputkan tersebut adalah kucing. Pekerjaan seperti ini lah yang disebut *image classification*, lihat Gambar 67.



kucing

Gambar 66. *Image Classification*

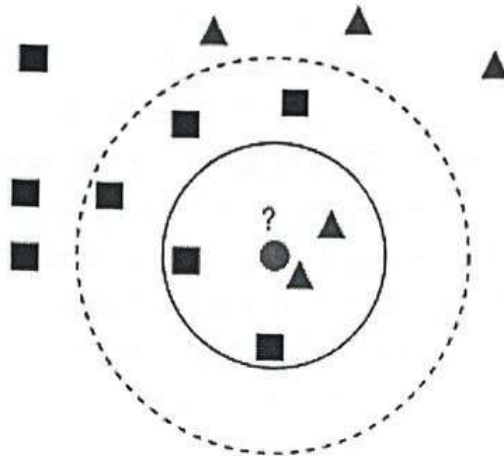
Image recognition merupakan tahapan yang lebih *advance* dibandingkan *image classification*. Pada *image classification*, sistem hanya mampu memprediksi/mengidentifikasi label/kelas dari objek, pada *image recognition*, selain memprediksi namun juga dapat mendeteksi posisi dari objek itu pada sebuah citra digital [1]. *Image recognition* dapat dilihat pada Gambar 67.



Gambar 67. *Image Recognition*

2. Memahami k-Nearest Neighbor (kNN)

Algoritma K-Nearest Neighbor (k-NN) adalah salah satu algoritma klasifikasi paling sederhana berdasarkan pembelajaran data. Algoritma ini termasuk dalam *supervised learning*. Idennya adalah untuk mencari kecocokan terdekat (*closest match*) dari data uji dalam ruang fitur. Untuk lebih jelasnya dapat dilihat pada Gambar 68.



Gambar 68. Visualisasi tentang Klasifikasi menggunakan kNN

Gambar 68, terdapat sejumlah data dengan dua kelas, yaitu kelas dengan kotak biru dan kelas dengan segitiga merah. Data-data tersebut diproyeksikan dalam ruang fitur. Misalnya, dapat dilihat pada contoh Gambar 68, data-data diproyeksikan pada ruang koordinat 2D, yang artinya untuk setiap data memiliki dua fitur, yaitu koordinat x dan y. Jika ditambah jumlah fitur, misalnya dengan tiga fitur, maka diperlukan ruang 3D. Dan jika menggunakan sebanyak N fitur, maka membutuhkan ruang N-D (N-Dimensi), ruang N-Dimensi ini adalah ruang fiturnya.

Pada Gambar 68, terdapat anggota baru atau data baru yang ditampilkan dengan lingkaran hijau. Data tersebut akan ditambahkan ke salah satu kelas biru atau kelas merah. Proses tersebut dinamakan klasifikasi. Lalu, bagaimana cara untuk mengklasifikasikan data tersebut?

Salah satu metode adalah dengan cara memeriksa siapa tetangga terdekatnya. Dari gambar 68, terlihat jelas bahwa data baru tersebut lebih dekat pada data dengan kelas segitiga merah. Maka data baru tersebut dapat diklasifikasikan untuk masuk ke kelas segitiga merah. Metode ini

disebut **Nearest Neighbor**, karena klasifikasi hanya bergantung pada tetangga terdekat.

Data dengan kelas segitiga merah mungkin yang terdekat tetapi bagaimana jika ada banyak data dengan kelas kotak biru yang dekat dengannya. Jadi mengecek yang terdekat saja tidak cukup namun dapat dilakukan pengecekan untuk beberapa k tetangga terdekat. Dalam Gambar 68, ambil $k = 3$, yaitu 3 tetangga terdekat. Data baru tersebut memiliki dua data pada kelas merah dan satu pada kelas biru. Jadi dengan $k = 3$, data baru tersebut diklasifikasikan untuk masuk ke kelas segitiga merah. Tetapi bagaimana jika kita menggunakan $k = 7$, terdapat 5 tetangga terdekat di kelas biru dan 2 tetangga terdekat di kelas merah. Jadi dengan $k = 7$ data baru tersebut diklasifikasikan untuk masuk ke kelas kotak biru. Maka data dilihat hasil klasifikasi dapat berubah dengan perubahan nilai k . Metode ini disebut **k-Nearest Neighbor (kNN)** karena klasifikasi tergantung pada k tetangga terdekat.

3. Contoh Perhitungan kNN

Untuk tahapan awal klasifikasi, adalah membagi data latih dan data uji. Data latih digunakan untuk membangun model dan data uji digunakan untuk menerapkan model. Untuk memahami proses klasifikasi kNN, dataset yang akan digunakan memiliki 2 fitur atau atribut (X_1 dan X_2). Setiap record terdapat label atau kelasnya (*supervised learning*) dengan jumlah kelas adalah 2 (kelas 1 dan kelas 2). Berikut ini merupakan data latih (*data training*).

	X_1	X_2	Kelas
Data Training 1	8	5	1
Data Training 2	7	3	1
Data Training 3	9	8	1
Data Training 4	5	5	2
Data Training 5	6	7	2

Berikut ini adalah data uji (*data testing*) yang ingin diketahui kelasnya. Akan dilakukan klasifikasi dengan menggunakan kNN dengan $k = 3$.

	X1	X2	Kelas
Data Testing	4	6	???

Untuk algoritma kNN, tahap awal adalah menghitung jarak antara data testing terhadap keseluruhan data training. Berikut adalah tabel hasil perhitungan jarak dengan menggunakan Square distance.

	X1	X2	Square distance to query distance (4,6)
Data Training 1	8	5	$(8-4)^2 + (5-6)^2 = 17$
Data Training 2	7	3	$(7-4)^2 + (3-6)^2 = 18$
Data Training 3	9	8	$(9-4)^2 + (8-6)^2 = 29$
Data Training 4	5	5	$(5-4)^2 + (5-6)^2 = 2$
Data Training 5	6	7	$(6-4)^2 + (7-6)^2 = 5$

Ambil k data dengan jarak terdekat, di mana $k = 3$. Pada tabel di bawah ini 3 tetangga terdekat terhadap data testing ditandai dengan $Y = 1$. Lihat kelas dominan dari 3 tetangga terdekat.

	X1	X2	Square distance	Rank	K=3	Y
Data Training 1	8	5	$(8-4)^2 + (5-6)^2 = 17$	3	Yes	1
Data Training 2	7	3	$(7-4)^2 + (3-6)^2 = 18$	4	No	-
Data Training 3	9	8	$(9-4)^2 + (8-6)^2 = 29$	5	No	-
Data Training 4	5	5	$(5-4)^2 + (5-6)^2 = 2$	1	Yes	2
Data Training 5	6	7	$(6-4)^2 + (7-6)^2 = 5$	2	Yes	2

Kelas dominan dari 3 tetangga terdekat adalah kelas 2, dengan jumlah data sebanyak 2 untuk kelas 2 dan jumlah data sebanyak 1 untuk kelas 1. Maka data testing terklasifikasi ke kelas 2.

	X1	X2	Class
Data Testing	4	6	2

B. Latihan

1. Klasifikasi kNN Menggunakan Dataset Random Number Generator

Berikut ini adalah contoh sederhana untuk klasifikasi kNN, dengan dua kelas. Kedua kelas tersebut diberi label kelas merah (dilambangkan dengan 0) dan kelas biru (dilambangkan dengan 1). Dibuat data pelatihan

sebanyak 25 record, dan memberi label mereka Kelas-0 atau Kelas-1. Generate dataset menggunakan Random Number Generator di Numpy dan menggunakan *library* Matplotlib untuk visualisasi data. Kelas merah ditampilkan dengan Segitiga Merah dan kelas Biru ditampilkan sebagai Kotak Biru.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

# fitur set yang berisi nilai (x, y) dari 25 data latih
trainData = np.random.randint(0,100,(25,2)).astype(np.float32)

# Beri label masing-masing merah atau biru dengan angka 0 dan 1
responses = np.random.randint(0,2,(25,1)).astype(np.float32)

# Ambil kelas merah dan plotkan
red = trainData[responses.ravel()==0]
plt.scatter(red[:,0],red[:,1],80,'r','^')

# Ambil kelas biru dan plotkan
blue = trainData[responses.ravel()==1]
plt.scatter(blue[:,0],blue[:,1],80,'b','s')

#buat data baru yang akan diklasifikasikan dan plotkan dengan warna hijau
newcomer = np.random.randint(0,100,(1,2)).astype(np.float32)
plt.scatter(newcomer[:,0],newcomer[:,1],80,'g','o')

knn=KNeighborsClassifier(n_neighbors=3) #define K=1
knn.fit(trainData,responses)

res = knn.predict(newcomer)

plt.show()
```

2. Klasifikasi kNN Menggunakan Dataset pada Sub-bab "Contoh Perhitungan kNN"

Lakukan *training* dengan kNN dengan menggunakan dataset di bawah ini.

	X1	X2	Kelas
Data Training 1	8	5	1
Data Training 2	7	3	1
Data Training 3	9	8	1
Data Training 4	5	5	2
Data Training 5	6	7	2

Lakukan *testing* dengan data di bawah ini.

	X1	X2	Kelas
Data Testing	4	6	???

Source Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

trainData = np.array([[8, 5], [7, 3], [9, 8], [5, 5], [6, 7]]).astype(np.float32)
responses = np.array([1, 1, 1, 2, 2]).astype(np.float32)

red = trainData[responses.ravel()==1]
plt.scatter(red[:,0],red[:,1],80,'r','^')

blue = trainData[responses.ravel()==2]
plt.scatter(blue[:,0],blue[:,1],80,'b','s')

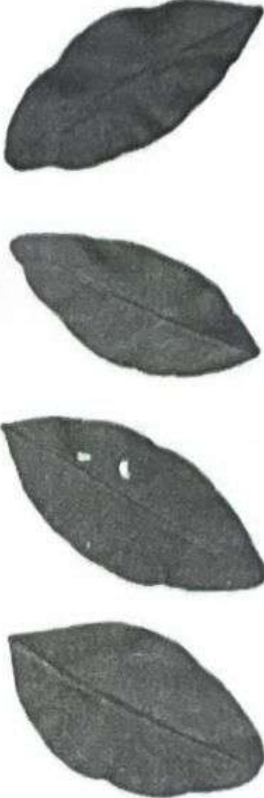
newcomer = np.array([[4, 6]]).astype(np.float32)
plt.scatter(newcomer[:,0],newcomer[:,1],80,'g','o')

knn=KNeighborsClassifier(n_neighbors=3) #define K=1
knn.fit(trainData,responses)

res = knn.predict(newcomer)
print (res)
plt.show()
```

3. Klasifikasi Citra Daun Menggunakan kNN

Lakukan klasifikasi data citra daun dengan menggunakan kNN dan ekstraksi fitur/ciri dengan *Local Binary Pattern*. Data citra daun memiliki 2 kelas. Masing-masing kelas terdapat 10 data, jadi total data daun berjumlah 20. Contoh data citra daun dapat lihat pada tabel berikut.

Kelas 1	Kelas 2
	

Gambar 69. Contoh Data Citra Daun.

Source Code:

```
import numpy as np
import cv2 as cv
from skimage.feature import local_binary_pattern
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

# settings for LBP
radius = 2
n_points = 8 * radius
METHOD = 'uniform'
plt.rcParams['font.size'] = 9

#membaca gambar kelas 2 dan lakukan konversi ke grayscale, lalu lakukan
feature extraction dengan LBP
data1_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/1.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)
data2_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/2.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)
data3_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/3.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)
data4_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/4.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)
data5_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/5.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)
data6_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/6.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)
data7_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/7.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)
data8_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/8.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)
data9_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/9.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)
data10_kelas2_lbp = local_binary_pattern(cv.imread('DATASET/KELAS2/10.jpg',
cv.IMREAD_GRAYSCALE), n_points, radius, METHOD)

#dapatkan histogram dari LBP untuk data kelas 1
data1_kelas1_lbp_hist, bins = np.histogram(data1_kelas1_lbp.ravel(), 256, [0, 256])
data2_kelas1_lbp_hist, bins = np.histogram(data2_kelas1_lbp.ravel(), 256, [0, 256])
data3_kelas1_lbp_hist, bins = np.histogram(data3_kelas1_lbp.ravel(), 256, [0, 256])
data4_kelas1_lbp_hist, bins = np.histogram(data4_kelas1_lbp.ravel(), 256, [0, 256])
data5_kelas1_lbp_hist, bins = np.histogram(data5_kelas1_lbp.ravel(), 256, [0, 256])
data6_kelas1_lbp_hist, bins = np.histogram(data6_kelas1_lbp.ravel(), 256, [0, 256])
data7_kelas1_lbp_hist, bins = np.histogram(data7_kelas1_lbp.ravel(), 256, [0, 256])
```

```

data8_kelas1_lbp_hist, bins = np.histogram(data8_kelas1_lbp.ravel(), 256, [0, 256])
data9_kelas1_lbp_hist, bins = np.histogram(data9_kelas1_lbp.ravel(), 256, [0, 256])
data10_kelas1_lbp_hist, bins = np.histogram(data10_kelas1_lbp.ravel(), 256, [0, 256])

```

#dapatkan histogram dari LBP untuk data kelas 2

```

data1_kelas2_lbp_hist, bins = np.histogram(data1_kelas2_lbp.ravel(), 256, [0, 256])
data2_kelas2_lbp_hist, bins = np.histogram(data2_kelas2_lbp.ravel(), 256, [0, 256])
data3_kelas2_lbp_hist, bins = np.histogram(data3_kelas2_lbp.ravel(), 256, [0, 256])
data4_kelas2_lbp_hist, bins = np.histogram(data4_kelas2_lbp.ravel(), 256, [0, 256])
data5_kelas2_lbp_hist, bins = np.histogram(data5_kelas2_lbp.ravel(), 256, [0, 256])
data6_kelas2_lbp_hist, bins = np.histogram(data6_kelas2_lbp.ravel(), 256, [0, 256])
data7_kelas2_lbp_hist, bins = np.histogram(data7_kelas2_lbp.ravel(), 256, [0, 256])
data8_kelas2_lbp_hist, bins = np.histogram(data8_kelas2_lbp.ravel(), 256, [0, 256])
data9_kelas2_lbp_hist, bins = np.histogram(data9_kelas2_lbp.ravel(), 256, [0, 256])
data10_kelas2_lbp_hist, bins = np.histogram(data10_kelas2_lbp.ravel(), 256, [0, 256])

```

#ubah vektor ke matriks dan lakukan transpose matriks untuk data kelas 1

```

data1_kelas1_lbp_hist = np.transpose(data1_kelas1_lbp_hist[0:18, np.newaxis])
data2_kelas1_lbp_hist = np.transpose(data2_kelas1_lbp_hist[0:18, np.newaxis])
data3_kelas1_lbp_hist = np.transpose(data3_kelas1_lbp_hist[0:18, np.newaxis])
data4_kelas1_lbp_hist = np.transpose(data4_kelas1_lbp_hist[0:18, np.newaxis])
data5_kelas1_lbp_hist = np.transpose(data5_kelas1_lbp_hist[0:18, np.newaxis])
data6_kelas1_lbp_hist = np.transpose(data6_kelas1_lbp_hist[0:18, np.newaxis])
data7_kelas1_lbp_hist = np.transpose(data7_kelas1_lbp_hist[0:18, np.newaxis])
data8_kelas1_lbp_hist = np.transpose(data8_kelas1_lbp_hist[0:18, np.newaxis])
data9_kelas1_lbp_hist = np.transpose(data9_kelas1_lbp_hist[0:18, np.newaxis])
data10_kelas1_lbp_hist = np.transpose(data10_kelas1_lbp_hist[0:18, np.newaxis])

```

#ubah vektor ke matriks dan lakukan transpose matriks untuk data kelas 2

```

data1_kelas2_lbp_hist = np.transpose(data1_kelas2_lbp_hist[0:18, np.newaxis])
data2_kelas2_lbp_hist = np.transpose(data2_kelas2_lbp_hist[0:18, np.newaxis])
data3_kelas2_lbp_hist = np.transpose(data3_kelas2_lbp_hist[0:18, np.newaxis])
data4_kelas2_lbp_hist = np.transpose(data4_kelas2_lbp_hist[0:18, np.newaxis])
data5_kelas2_lbp_hist = np.transpose(data5_kelas2_lbp_hist[0:18, np.newaxis])
data6_kelas2_lbp_hist = np.transpose(data6_kelas2_lbp_hist[0:18, np.newaxis])
data7_kelas2_lbp_hist = np.transpose(data7_kelas2_lbp_hist[0:18, np.newaxis])
data8_kelas2_lbp_hist = np.transpose(data8_kelas2_lbp_hist[0:18, np.newaxis])
data9_kelas2_lbp_hist = np.transpose(data9_kelas2_lbp_hist[0:18, np.newaxis])
data10_kelas2_lbp_hist = np.transpose(data10_kelas2_lbp_hist[0:18, np.newaxis])

```

#gabungkan data citra menjadi satu matriks data training

```

trainData = np.concatenate((data1_kelas1_lbp_hist, data2_kelas1_lbp_hist,
data3_kelas1_lbp_hist, data4_kelas1_lbp_hist,
data5_kelas1_lbp_hist, data6_kelas1_lbp_hist,

```

```

data7_kelas1_lbp_hist, data8_kelas1_lbp_hist,
data1_kelas2_lbp_hist, data2_kelas2_lbp_hist,
data3_kelas2_lbp_hist, data4_kelas2_lbp_hist,
data5_kelas2_lbp_hist, data6_kelas2_lbp_hist,
data7_kelas2_lbp_hist, data8_kelas2_lbp_hist), axis=0).astype(np.float32)

#gabungkan data citra menjadi satu matriks data testing
testData = np.concatenate((data9_kelas1_lbp_hist, data10_kelas1_lbp_hist,
data9_kelas2_lbp_hist, data10_kelas2_lbp_hist), axis=0).astype(np.float32)

#target
responses = np.array([1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2]).astype(np.float32)

#kNN
knn=KNeighborsClassifier(n_neighbors=3) #define K=3
knn.fit(trainData,responses)
res = knn.predict(testData)
print(res)

```

C. Lembar Kerja Praktikum

1. Dari LKP BAB X, lakukan proses klasifikasi citra batik dengan minimal 2 kelas batik. Kumpulkan dataset 2 jenis batik tersebut, boleh ambil kumpulan citra batik dari internet. Silahkan lakukan eksperimen terhadap fitur dan metode klasifikasinya. Boleh gunakan fitur lain selain LBP atau kombinasi dari beberapa fitur (misal fitur tekstur dan warna). Juga boleh menerapkan metode klasifikasi lain selain KNN, misal ANN, SMV, Random Forest, dan lain-lain. Lakukan analisis terhadap hasilnya, tampilkan dalam visualisasi yang baik.

Referensi

- [1] Renukasoni, "Image detection, recognition and image classification with machine learning.," <https://medium.com/>, 2019. <https://medium.com/ai-techsystems/image-detection-recognition-and-image-classification-with-machine-learning-92226ea5f595> (accessed Nov. 04, 2020).

BAB

XII

EVALUASI HASIL KLASIFIKASI CITRA

Materi:

- Akurasi Klasifikasi
- Evaluasi Kinerja Model

Tujuan Pembelajaran:

- Mahasiswa dapat melakukan evaluasi model klasifikasi citra menggunakan OpenCV-Python

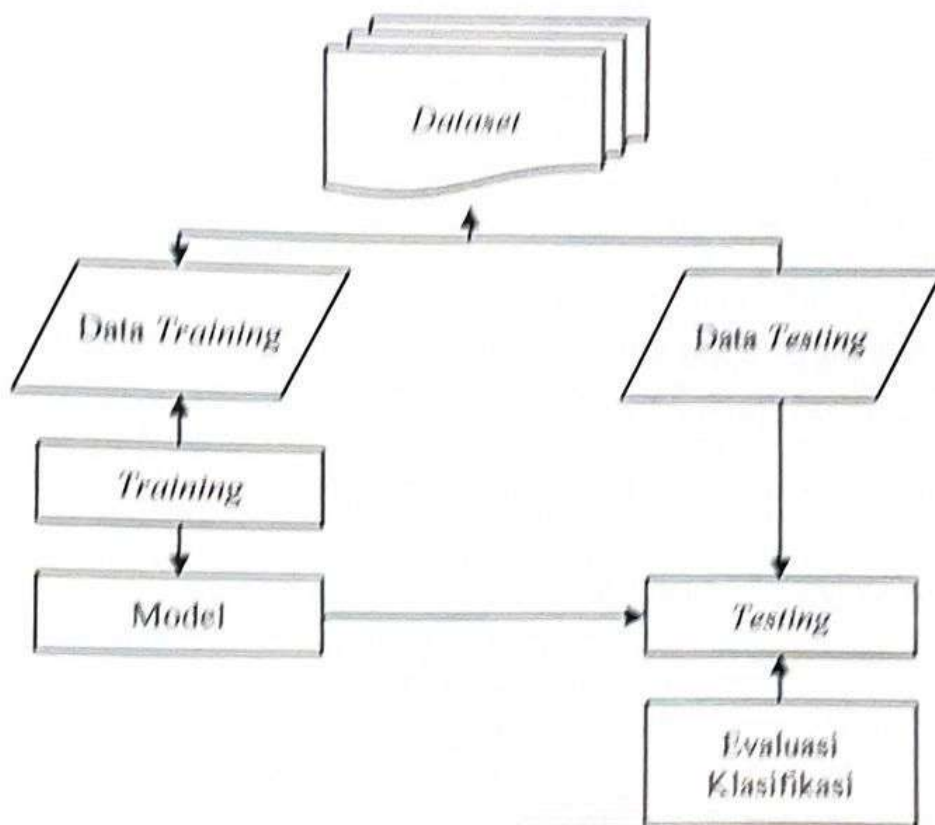
A. Penyajian

Setelah membangun model, Kita perlu mengetahui seberapa bagus dan seberapa akurat model yang sudah kita bangun. Misal, dalam mengklasifikasikan tumbuhan, seberapa akurat model dapat mengenali tumbuhan berdasarkan ciri-cirinya. Kita ingin mengestimasi, kira-kira seberapa akurat model dalam mengklasifikasikan tumbuhan berdasarkan ciri daun jika diberikan data baru yang berisi ciri-cirinya di mana data baru ini belum pernah dipakai untuk membangun model. Bahkan kita mungkin ingin membandingkan beberapa model dan membandingkan seberapa baik model-model tersebut dengan membandingkan akurasi. Apa itu akurasi? Bagaimana kita mengestimasi akurasi? adakah pengukuran yang lain selain akurasi? bagaimana kita mendapatkan akurasi yang *reliable*?

Menggunakan data *training* untuk membangun model, kemudian menggunakan data yang sama untuk menghitung akurasi model dapat membawa kita kepada estimasi yang “*overoptimistic*” karena data tersebut sudah dipakai untuk *training* dalam membangun model. Bisa Anda bayangkan jika Anda diminta mengerjakan latihan supaya memahami suatu konsep, jika Anda diuji dengan data yang sama dengan data saat

latihan, kemudian nilai Anda bagus, apakah Anda yakin jika Anda sudah memahami konsepnya? untuk menguji pemahaman Anda, maka akan diberikan soal tes yang belum dipakai untuk latihan.

Maka dari itu, dalam proses klasifikasi dibagi data ke dalam 2 bagian, yaitu data *training* dan data *testing*. Data *training* digunakan untuk membangun model, dan data *testing* digunakan untuk menguji model. Alur kerja proses klasifikasi secara umum dapat dilihat pada Gambar 70 berikut:



Gambar 70. Alur Kerja Klasifikasi Citra Digital.

Data dibagi menjadi data *training* dan data *testing*, kemudian model dibangun dengan menggunakan data *training*. Model *classifier* yang telah dibangun selanjutnya diuji dengan data *testing*, hasil dari pengujian inilah yang akan kita cari akurasi.

1. Akurasi Klasifikasi

Akurasi merupakan rasio prediksi benar (positif dan negatif) dengan keseluruhan data. Dalam kasus klasifikasi seseorang positif virus Covid-19 atau negatif virus Covid-19 berdasarkan citra CT Scan paru-paru, akurasi menjawab pertanyaan “berapa persen pasien yang benar diprediksi positif virus Covid-19 dan negatif virus Covid-19 dari keseluruhan pasien”. Untuk mendapatkan nilai akurasi, diperlukan suatu matriks yang dinamakan *confusion matrix*.

		Hasil Prediksi		
		+1 (yes)	-1 (no)	Total
Hasil yang Diharapkan (<i>ground truth</i>)	+1 (yes)	TP	FN	P
	-1 (no)	FP	TN	N
	Total	P (Prediksi)	N (Prediksi)	

Tuple positif dan *tuple* negatif, misal kita punya kasus deteksi bahwa seseorang terpapar virus Covid-19 yang memiliki kelas Covid = yes (sakit) dan Covid = no (tidak sakit alias sehat). Kelas Covid = yes kita sebut *Positive Tuple*, sedangkan Covid = no kita sebut *Negative Tuple*. Misal jumlah kelas positif = P, dan jumlah kelas negatif = N.

- True Positives* (TP) : Jumlah *tuple* positif yang benar diklasifikasi oleh *classifier* (model klasifikasi yang telah kita bangun) atau *tuple* positif yang diprediksi juga sebagai positif.
- True Negative* (TN) : Jumlah *tuple* negatif yang benar diprediksi oleh *classifier* atau *tuple negative* diprediksi juga sebagai negatif.
- False Positive* (FP) : Jumlah *tuple* negatif yang salah diprediksi sebagai positif. misal seharusnya kelasnya Covid = no, tapi model kita memprediksi sebagai Covid = yes.
- False Negative* (FN) : adalah jumlah *tuple* positif yang salah diprediksi sebagai negatif. misal, seharusnya kelasnya (dari data) Covid = yes, tapi model kita memprediksi sebagai Covid = no.

Ada beberapa pengukuran performance model klasifikasi dengan berdasarkan *confusion matrix*, antara lain:

Measure	Formula
accuracy, recognition rate	$\frac{TP+TN}{P+N}$
error rate, misclassification rate	$\frac{FP+FN}{P+N}$
sensitivity, true positive rate, recall	$\frac{TP}{P}$
specificity, true negative rate	$\frac{TN}{N}$
precision	$\frac{TP}{TP+FP}$
$F, F_1, F\text{-score}$, harmonic mean of precision and recall	$\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$
F_β , where β is a non-negative real number	$\frac{(1 + \beta^2) \times \text{precision} \times \text{recall}}{\beta^2 \times \text{precision} + \text{recall}}$

P merupakan jumlah kelas positif ($P = TP + FN$), dan N merupakan jumlah kelas negatif ($N = FP + TN$). *Sensitivity* merupakan *True Positive Recognition Rate*, yaitu akurasi untuk kelas positif. *Specificity* adalah *True Negative Recognition Rate*, yaitu akurasi untuk kelas negatif. perhitungan ini diperlukan untuk mengetahui seberapa bagus model/classifier untuk mengklasifikasikan data.

Sebagai contoh, perhatikan tabel *confusion matriks* di bawah ini:

Classes	yes	no	Total	Recognition (%)
yes	90	210	300	30.00
no	140	9560	9700	98.56
Total	230	9770	10,000	96.40

Dapat dilihat bahwa akurasi keseluruhan untuk pengujian tersebut adalah 96,40%. Namun, kelas *yes* hanya benar diprediksi (TP) sebanyak 90 dari total kelas *yes* ($P = 300$) atau dengan *sensitivity* = 30%, sedangkan kelas negatif memiliki TN sebanyak 9560 dari total negatif (N) sebanyak 9700 sehingga *specificity* = 98,56%. Kasus perbedaan drastis ini umum terjadi pada kasus *class imbalance* data di mana jumlah kelas positif (P)

dan jumlah kelas negatif (N) memiliki proporsi yang jauh berbeda (kalau di decision tree, kita bisa bilang table dengan impurity rendah).

2. Evaluasi Kinerja Model

Setelah kita mengetahui teknik pengukuran akurasi, teman-temannya akurasi, dan properti-propertinya, kita akan membahas teknik pembagian data. Seperti yang kita bahas di paragraf-paragraf awal modul ini, data perlu dibagi menjadi data training dan data testing. pertanyaannya, bagaimana membaginya? seberapa besar untuk training, dan seberapa besar untuk testing?

a. *Hold-out Method* dan *Random sub-sampling*

Pada metode *hold-out*, data secara random dibagi menjadi dua himpunan (set) yang independen, training set dan test set. Secara fisik dapat digambarkan pada Gambar 1. Secara umum $2/3$ data dialokasikan untuk *training* (membangun model), dan sisanya ($1/3$) untuk *testing* untuk mengestimasi akurasi. estimasi dari model ini masih pesimistik karena hanya satu porsi data digunakan untuk membangun model.

Random subsampling merupakan varian dari *hold-out method* di mana pada *random subsampling*, dilakukan pengulangan pada *hold-out* sebanyak k -kali. akurasi keseluruhan dihitung dengan mengambil rata-rata akurasi dari k pengulangan tersebut.

b. *Cross Validation*

Cross-validation (CV) adalah metode statistik yang dapat digunakan untuk mengevaluasi kinerja model atau algoritma di mana data dipisahkan menjadi dua subset yaitu data proses pembelajaran dan data validasi/evaluasi. Model atau algoritma dilatih oleh subset pembelajaran dan divalidasi oleh subset validasi. Selanjutnya pemilihan jenis CV dapat didasarkan pada ukuran dataset. Biasanya *K-fold cross-validation* digunakan karena dapat mengurangi waktu komputasi dengan tetap menjaga keakuratan estimasi [1]. Pada *K-fold cross-validation*, data awal dibagi secara random ke dalam k bagian, $D_1, D_2, \dots, D_k$, di mana setiap data kira-kira memiliki ukuran yang sama. *Training* dan *testing* dilakukan sebanyak k kali. Pada iterasi ke i , partisi data D_i digunakan sebagai *testing* dan

sisanya digunakan untuk *training*. Ilustrasi *k-fold cross validation* dapat dilihat pada gambar 71.

Iterasi-1	→	D1	D2	D3	D4	D5
Iterasi-2	→	D1	D2	D3	D4	D5
Iterasi-3	→	D1	D2	D3	D4	D5
Iterasi-4	→	D1	D2	D3	D4	D5
Iterasi-5	→	D1	D2	D3	D4	D5

Gambar 71. 5-Fold Cross Validation

Gambar 71 merupakan metode pembagian data *5-fold cross validation*. saat iterasi pertama, Data partisi D1 digunakan untuk test, dan sisanya (D2, D3, D4, D5) untuk training model. itersi ke-2, data D2 digunakan untuk tes, dan sisanya untuk training, dst hingga iterasi ke5 di mana data D5 digunakan untuk test, dan D1 hingga D4 digunakan untuk training model.

Tidak seperti *hold-out* dan *random subsampling*, pada metode *k-fold cross validation* setiap sampel data pernah digunakan untuk test sekali, dan digunakan untuk *training* dengan jumlah yang sama (misal pada 5-fold, tiap sampel digunakan untuk training sebanyak 4 kali, dan 1 kali untuk test). Untuk klasifikasi, estimasi akurasi adalah jumlah keseluruhan dari klasifikasi yang benar dari k iterasi, dibagi jumlah seluruh sampel data (karena semua data digunakan untuk test).

Metode *cross validation* ini sebenarnya ada banyak. untuk di Python bisa di cek di https://scikitlearn.org/stable/modules/cross_validation.html.

B. Latihan

1. Klasifikasi Citra Daun Menggunakan Cross Validation

Lakukan klasifikasi data citra daun dengan menggunakan kNN dan ekstraksi fitur/ciri dengan *Local Binary Pattern*. Lakukan evaluasi model dengan menggunakan *cross validation*. Untuk menggunakan *cross validation* untuk memvalidasi model klasifikasi dapat menambahkan *code* pada modul ke-11 latihan ke-3 sebagai berikut:

- Tambahkan *library*.

```
from sklearn.model_selection import cross_val_score
```

- Data tidak dipisah ke dalam data training dan data testing, tapi digabungkan keseluruhan data.

```
Dataset = np.concatenate((data1_kelas1_lbp_hist,
data2_kelas1_lbp_hist, data3_kelas1_lbp_hist, data4_kelas1_lbp_hist,
data5_kelas1_lbp_hist, data6_kelas1_lbp_hist, data7_kelas1_lbp_hist,
data8_kelas1_lbp_hist, data9_kelas1_lbp_hist,
data10_kelas1_lbp_hist, data1_kelas2_lbp_hist,
data2_kelas2_lbp_hist, data3_kelas2_lbp_hist, data4_kelas2_lbp_hist,
data5_kelas2_lbp_hist, data6_kelas2_lbp_hist, data7_kelas2_lbp_hist,
data8_kelas2_lbp_hist, data9_kelas2_lbp_hist,
data10_kelas2_lbp_hist), axis=0).astype(np.float32)
```

- Karena dataset tidak dipisah ke dalam data training dan data testing, maka variable responses dibuat seperti ini.

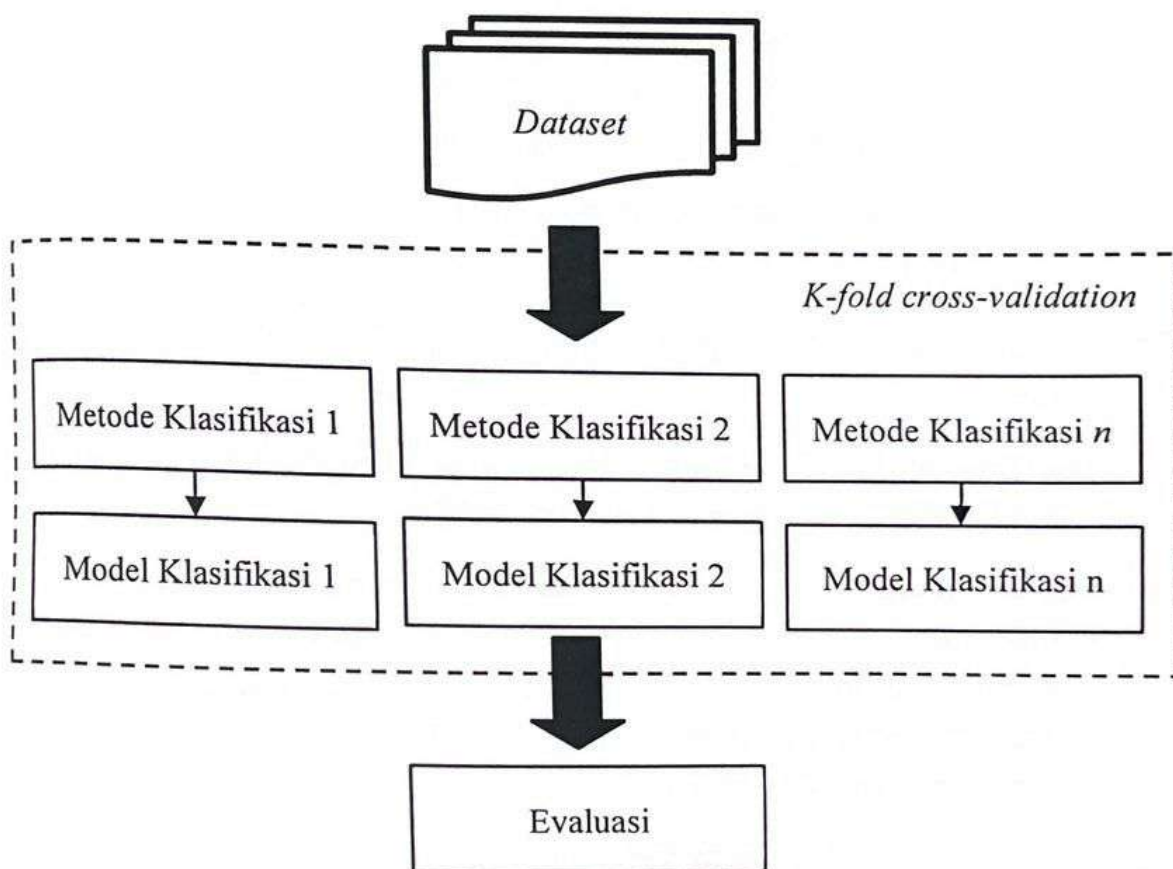
```
responses = np.array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]).astype(np.float32)
```

- Klasifikasi kNN dengan $k = 3$, dan hitung score dengan cross validation menggunakan perhitungan akurasi dengan parameter *cross validation* = 5.

```
knn=KNeighborsClassifier(n_neighbors=3)
score= cross_val_score(knn, Dataset, responses, cv=5,
scoring='accuracy')
print(score.mean())
```

C. Lembar Kerja Praktikum

1. Dari LKP BAB XI, yaitu program klasifikasi citra batik yang telah dibuat. Lakukan eksperimen dengan membandingkan beberapa metode klasifikasi, misalnya ANN, SVM, kNN, Naïve Bayes, dan lain-lain (minimal 2). Untuk pengujian dan validasi model klasifikasi menggunakan *K-fold cross-validation*. Berikut ini adalah alur dari *design* eksperimen untuk klasifikasi citra batik.



Gambar 72. Alur dari Design Eksperimen untuk Klasifikasi Citra Batik.

Referensi

- [1] A. Wibowo, "10 Fold-Cross Validation," <https://mti.binus.ac.id/>, 2017. [https://mti.binus.ac.id/2017/11/24/10-fold-cross-validation/#:~:text=Cross-validation \(CV\) adalah,pembelajaran dan data validasi %2F evaluasi.](https://mti.binus.ac.id/2017/11/24/10-fold-cross-validation/#:~:text=Cross-validation%20(CV) adalah,pembelajaran dan data validasi%2F evaluasi.) (accessed Nov. 10, 2020).